

Effective and Scalable Algorithms for Influence Estimation and Maximization under Hypergraph Threshold Cascades

Tianyou Gao

Kyoto University

Kyoto, Japan

gao.tianyou.66z@st.kyoto-u.ac.jp

Shaowen Peng

Nara Institute of Science and

Technology

Nara, Japan

peng.shaowen@naist.ac.jp

Takayuki Ito

Kyoto University

Kyoto, Japan

ito@i.kyoto-u.ac.jp

ABSTRACT

Influence estimation (IE) and influence maximization (IM) have been extensively studied under pairwise network diffusion, while real-world diffusion often also involves higher-order group effects. For example, a user may be exposed to a product or opinion through a follow link, but may adopt it only after several members in the same group have discussed or recommended it. This motivates a hybrid diffusion setting where pairwise propagation coexists with threshold-based activation within hyperedges. To formalize this setting, this paper studies the *Hypergraph Threshold Cascade (HTC)* model, where each hyperedge has a given threshold and a node can be activated through the hyperedge only when at least the threshold number of other nodes in the same hyperedge have already become active. Based on the HTC model, we formulate two problems, i.e., *IE under HTC (HTC-IE)* and *IM under HTC (HTC-IM)*. These two problems pose two main technical challenges. First, the well-studied reverse reachable set coverage view no longer applies because root activation may require multiple prerequisite nodes. Second, threshold-based hyperedges create seed complementarity, making the HTC influence function non-submodular. To address these challenges, we propose the *Reverse Threshold Witness (RTW)*, a reverse sampling object that records both live-edge dependencies and hyperedge threshold gates. Using RTWs, we design *RTW-Est* for HTC-IE, which uses an unbiased influence estimator and provides an (ϵ, δ) -relative approximation guarantee. For HTC-IM, we develop the *RTW-Greedy* algorithm with a solution-dependent approximation guarantee and further propose *RTW-Greedy-P* as a scalable practical variant. Experiments on seven real-world datasets with up to millions of nodes demonstrate the effectiveness and efficiency of the proposed algorithms.

PVLDB Reference Format:

Tianyou Gao, Shaowen Peng, and Takayuki Ito. Effective and Scalable Algorithms for Influence Estimation and Maximization under Hypergraph Threshold Cascades. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://anonymous.4open.science/r/codeHTC>.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Influence maximization (IM), which aims to select up to k seed nodes to maximize the expected spread, is a fundamental problem in data mining and social network analysis [26]. Classical IM studies mainly focus on networks where influence propagates through pairwise relationships between users. However, users also interact through higher-order structures in many real-world scenarios [4], e.g., group chats [39], online discussion communities [54], and events [31]. In such scenarios, influence may propagate not only through ordinary pairwise links, but also through higher-order interactions that can be represented by hyperedges [4, 15]. To capture such higher-order interactions, recent studies have extended IM from ordinary graphs to hypergraphs [52, 55, 57]. However, existing hypergraph IM studies still leave efficient and scalable algorithms for hybrid cascades underexplored, especially when pairwise propagation coexists with threshold-based hyperedge activation. This hybrid setting is algorithmically challenging, as discussed below, and addressing it is the focus of this paper.

This setting is motivated by viral marketing, a classical application of IM. In such scenarios, influence may propagate through both pairwise relations (follow links) and group-based structures (group chats). For example, one user may adopt a product or opinion after being influenced by a followed user, whereas another may adopt it only after sufficiently many members of the same close-knit group, such as family members or close friends, have recommended it. The former behavior is naturally modeled by probabilistic pairwise propagation, while the latter is captured by threshold-based activation within hyperedges.

This paper studies this hybrid setting, referred to as the *Hypergraph Threshold Cascade (HTC)* model. Unlike pure hypergraph threshold diffusion, HTC combines pairwise probabilistic propagation with threshold-based hyperedge activation. In this model, each pairwise edge transmits influence probabilistically, while each hyperedge h is associated with a threshold τ_h . A node $v \in h$ can be activated through h only when at least τ_h other nodes in the same hyperedge have already become active. Figure 1 illustrates a simple example of the HTC model.

The threshold mechanism is motivated by classical threshold models and complex contagion, where adoption may require reinforcement from multiple active sources [7, 18, 51]. Hypergraphs naturally capture such group-level reinforcement beyond pairwise links [4, 15, 25]. Together, these observations support HTC as a natural model for diffusion processes in which pairwise exposure and threshold-based group reinforcement coexist.

Given the HTC model, this paper studies two basic computational tasks. The first task is influence estimation (IE) under HTC, referred

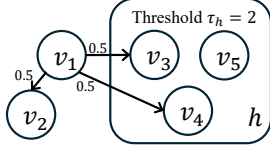


Figure 1: A simple example of the HTC model. Ordinary directed edges represent pairwise probabilistic propagation, where node v_1 can activate v_2 , v_3 , and v_4 independently with probability 0.5. The hyperedge $h = \{v_3, v_4, v_5\}$ has threshold $\tau_h = 2$. If v_1 is selected as the seed, then v_5 can be activated through h only when both v_3 and v_4 have been activated.

to as *HTC-IE*, which estimates the expected spread of a given seed set and is #P-hard for exact computation. The second task is IM under HTC, referred to as *HTC-IM*, which selects up to k seed nodes to maximize the expected spread and is NP-hard. These tasks pose two main challenges due to threshold-based hyperedge activation.

- **Challenge 1: Invalid RR-set coverage view.** A standard tool for classical IM is the reverse influence sampling (RIS) technique, introduced by Borgs et al. [5]. Classical RIS works because a seed set activates a sampled root if and only if it intersects the corresponding reverse reachable set (RR-set). However, under the HTC model, activating a root may require several prerequisite nodes to be jointly activated before a hyperedge threshold is satisfied. Therefore, the RR-set coverage view cannot be directly applied to either HTC-IE or HTC-IM.
- **Challenge 2: Non-submodularity.** Threshold-based hyperedge activation introduces complementarity among seed nodes. Several nodes may have no marginal contribution when selected individually, but may jointly satisfy a hyperedge threshold when selected together. As a result, the HTC influence function is monotone but non-submodular, so classical submodular maximization techniques do not directly retain their $1 - \frac{1}{e}$ approximation guarantees [35].

To address these challenges, this paper introduces *Reverse Threshold Witnesses (RTWs)*, a novel reverse sampling object for HTC that generalizes the RR-set idea beyond simple node-set coverage. Unlike an RR-set, an RTW is not a simple set of nodes, but a structured witness that records ordinary reverse dependencies induced by live edges and threshold gates induced by hyperedges. Each threshold gate represents a condition requiring that at least the given threshold number of nodes in a hyperedge become active. The key property is that a seed set satisfies an RTW if and only if it activates the sampled root in the corresponding forward HTC process.

This equivalence yields an unbiased estimator for HTC-IE. Based on this estimator, we design the *RTW-Est* algorithm, which returns an (ϵ, δ) -relative approximation for any given seed set.

For HTC-IM, we propose the *RTW-Greedy* algorithm. It first constructs an upper-relaxed model, where each hyperedge threshold is relaxed into an easier activation condition, to obtain an upper bound on the optimum HTC influence. It then generates RTW samples and greedily maximizes the empirical RTW objective. Since

the HTC objective is non-submodular, *RTW-Greedy* does not directly inherit the classical $1 - \frac{1}{e}$ guarantee. Instead, it provides a solution-dependent guarantee that depends on the tightness of the upper-relaxed model. However, *RTW-Greedy* can be computationally expensive on large graphs because it needs to generate and maintain many RTW samples. To reduce this cost, we further present *RTW-Greedy-P*, a practical variant that uses a fixed sample budget for candidate generation and performs independent validation to select the final seed set.

To the best of our knowledge, this is the first work to directly address the reverse-sampling challenge for both IE and IM under HTC, providing scalable sampling algorithms with unbiased estimation for IE and solution-dependent guarantees for IM.

We conduct experiments on seven real-world datasets with up to millions of nodes. For HTC-IE, the results show that *RTW-Est* provides accurate influence estimates while running substantially faster than the baseline using simple Monte Carlo (MC) simulations. For HTC-IM, *RTW-Greedy* and *RTW-Greedy-P* achieve higher HTC influence than the baseline methods in most settings. Moreover, *RTW-Greedy-P* substantially improves scalability and can handle large-scale graphs on which *RTW-Greedy* becomes computationally expensive. These results demonstrate the effectiveness and scalability of the proposed algorithms for HTC-IE and HTC-IM.

The main contributions of this paper are summarized as follows.

- We formulate the HTC model, where influence propagates through both directed pairwise edges and threshold-based hyperedges. Based on HTC, we define HTC-IE and HTC-IM, and show that exact HTC-IE is #P-hard, HTC-IM is NP-hard, and the HTC influence function is non-submodular.
- We propose the RTW, a reverse sampling object for HTC. We prove the equivalence between forward activation and RTW satisfaction, which enables unbiased influence estimation under HTC.
- For HTC-IE, we design *RTW-Est*, which provides an (ϵ, δ) -relative approximation guarantee for any given seed set. For HTC-IM, we develop *RTW-Greedy* with a solution-dependent guarantee and further propose *RTW-Greedy-P* as a scalable practical variant.
- We conduct experiments on seven real-world datasets. The results show that *RTW-Est* provides accurate and efficient influence estimation, while *RTW-Greedy* and *RTW-Greedy-P* achieve effective and scalable seed selection under HTC.

Note. A full version of this paper, including omitted proofs and additional experimental results, is available at <https://anonymous.4open.science/r/codeHTC/appendix.pdf>.

2 RELATED WORK

This section reviews related studies on IE and IM.

IE on Ordinary Graphs. IE estimates the expected spread of a given seed set, whose exact computation is #P-hard under classical diffusion models [11]. Early methods rely on MC simulations [11, 12, 26], which are conceptually simple but often expensive on large graphs. RIS later became a fundamental tool for scalable IE via RR-set coverage [5, 47, 48]. Subsequent studies improve IE efficiency through importance and outward influence sampling [36, 37], faster RR-set generation [21, 58], sketching techniques [14], and tighter

sample complexity analysis [42]. However, they cannot be directly extended to the HTC-IE problem.

IM on Ordinary Graphs. IM was first formulated on ordinary graphs [26]. The classical influence function is monotone and submodular, which leads to the $1 - 1/e$ greedy approximation guarantee [35]. RIS was later introduced to support scalable IM by reducing seed selection to maximum coverage over RR-sets [5, 21, 22, 36, 38, 46–48]. However, these studies are designed for pairwise graph diffusion and rely on RR-set coverage and submodularity, which do not directly hold under HTC. Beyond the classical setting, a large body of work has studied IM under different scenarios, including time-aware [1, 10, 30], location-aware [6, 28, 50], topic-aware [3, 9, 23], and community-aware [13, 20, 49] settings. These studies are complementary to our work.

IE and IM on Hypergraphs. Recent studies have begun to investigate IE/IM on hypergraphs, with existing methods broadly grouped into heuristic and sampling-based approaches. Heuristic methods select seed nodes according to handcrafted importance scores, such as ranking scores [33], degree-based scores [52], multi-hop influence estimates [17], collective-influence scores [56], and evolutionary-search objectives [40]. Among them, Zhang et al. [56] are the closest to our setting, as they study threshold-based IM on pure hypergraphs using the *HCI-TM* heuristic. In contrast, our HTC captures a more general hybrid diffusion process in which pairwise propagation and threshold-based hyperedge activation coexist. Rather than relying on heuristic seed scoring, our algorithm directly tackles the resulting RIS barrier by replacing RR-set coverage with RTW satisfaction, yielding an unbiased IE estimator and certified IM algorithms at scale.

Sampling-based methods improve scalability by adapting RIS to hypergraph diffusion. Zhu et al. [57] study IM on directed hypergraphs, where a hyperedge models group-to-node influence. Zhang et al. [55] further improve the scalability of hypergraph IM by accelerating RR-set generation through stratified sampling. However, these two methods are developed for different hypergraph diffusion models and therefore address problems complementary to ours. As a result, they cannot be readily adapted to HTC-IE or HTC-IM.

3 PROBLEM FORMULATION

This section formalizes the HTC model and the two computational tasks studied in this paper. Let $\mathcal{G} = (V, E, H)$ be a hybrid hypergraph, where V is the set of nodes, $E \subseteq V \times V$ is the set of ordinary directed edges, and $H \subseteq 2^V$ is the set of hyperedges. We define $n = |V|$, $m = |E|$, and $q = |H|$.

Each ordinary edge $(u, v) \in E$ is associated with a propagation probability $p_{u,v} \in [0, 1]$. Each hyperedge $h \in H$ is associated with a threshold $\tau_h \in \{1, \dots, |h| - 1\}$. Ordinary edges represent pairwise propagation channels, while hyperedges represent higher-order group channels. For a hyperedge $h \in H$ and a target node $v \in h$, $h \setminus \{v\}$ denotes the set of nodes in h other than v .

3.1 Hypergraph Threshold Cascade

We define the HTC model as follows. Given a seed set S , all nodes in S are initially active. For ordinary edges, we follow the Independent Cascade (IC) model [26], where an active node u has one chance to activate each inactive out-neighbor v through an edge

$(u, v) \in E$ with probability $p_{u,v}$. In contrast, hyperedge propagation is threshold-based and deterministic. For a hyperedge $h \in H$ and a target node $v \in h$, v becomes active through h once at least τ_h nodes in $h \setminus \{v\}$ have been activated. The cascade is progressive, i.e., once a node becomes active, it remains active. The propagation process terminates when no inactive node can be further activated.

Let Ω be the sample space of possible worlds, where each $\omega \in \Omega$ independently samples every ordinary edge $(u, v) \in E$ as live with probability $p_{u,v}$, and let $E^\omega \subseteq E$ be the set of live edges. Given a seed set $S \subseteq V$, the final activated set $A^\omega(S)$ is the least set containing S that satisfies two closure rules. First, if $(u, v) \in E^\omega$ and $u \in A^\omega(S)$, then $v \in A^\omega(S)$. Second, if $h \in H$, $v \in h$, and $|A^\omega(S) \cap (h \setminus \{v\})| \geq \tau_h$, then $v \in A^\omega(S)$.

The expected influence spread of S under the HTC model is defined as $\sigma(S) = \mathbb{E}_{\omega \in \Omega} [|A^\omega(S)|]$, where the expectation is taken over the possible-world distribution induced by the ordinary edge probabilities. Exact computation of $\sigma(S)$ is #P-hard, since exact influence computation under the IC model is already #P-hard [11], and the IC model is a special case of HTC when $H = \emptyset$.

3.2 Influence Estimation and Maximization

We study two algorithmic tasks under the HTC model: influence estimation and influence maximization.

Definition 3.1 (HTC-IE). Given a hybrid hypergraph $\mathcal{G} = (V, E, H)$ and a seed set $S \subseteq V$, estimate the expected influence spread $\sigma(S)$.

HTC-IE is a fundamental primitive for evaluating a given seed set and comparing different seed selection strategies.

Definition 3.2 (HTC-IM). Given a hybrid hypergraph $\mathcal{G} = (V, E, H)$ and a budget k , find a seed set $S \subseteq V$ with $|S| \leq k$ that maximizes the expected influence spread $\sigma(S)$.

Equivalently, the objective is $S^* \in \arg \max_{S \subseteq V, |S| \leq k} \sigma(S)$. The following theorem summarizes the basic computational and structural properties of HTC-IM.

THEOREM 3.3. *HTC-IM is NP-hard. Moreover, the HTC influence function $\sigma(\cdot)$ is monotone but not submodular in general.*

The NP-hardness follows because IM under the IC model is a special case of HTC-IM when $H = \emptyset$. Monotonicity follows from the progressive nature of HTC diffusion, since adding more seeds cannot deactivate any node. Non-submodularity follows from hyperedge threshold complementarity. For example, consider $h = \{a, b, v\}$ with $\tau_h = 2$. Adding a alone cannot activate v , while adding a after b has been selected can activate v , violating the diminishing-return property. Due to the lack of submodularity, the standard $(1 - 1/e)$ -type guarantees [35] of classical RIS-based maximum coverage algorithms [48] cannot be directly applied to HTC-IM.

4 REVERSE THRESHOLD WITNESS

This section explains why RR-set coverage fails under HTC, constructs RTWs, and derives an unbiased RTW estimator. The key technical contribution is to design a reverse sampling object that preserves the root-activation equivalence of RR-sets while representing threshold dependencies that cannot be captured by ordinary RR-sets.

4.1 Why RR-set Coverage Fails under HTC

Classical RIS relies on the RR-set representation. Given a root node r and a possible world ω , an RR-set R_r^ω satisfies $S \cap R_r^\omega \neq \emptyset$ if and only if $r \in A^\omega(S)$. Thus, a seed set activates the root in ω exactly when it hits the corresponding RR-set. This set-hitting equivalence is the key reason why IE can be reduced to estimating RR-set coverage, and why IM can be reduced to a maximum coverage problem.

However, this representation breaks down under HTC. The reason is that a hyperedge may require several activated nodes to jointly trigger another node. Consider a hyperedge $h = \{a, b, c, v\}$ with threshold $\tau_h = 2$. To activate v through h , it is not sufficient that only one of a, b , and c can be activated by S . Instead, at least two of these nodes must be activated. Thus, the reverse condition is not a set-hitting condition over $\{a, b, c\}$, but an at-least-two condition over the activation events of a, b , and c . This condition cannot be represented as $S \cap R \neq \emptyset$ for any ordinary set R .

Therefore, a reverse sample for HTC must record more than a set of candidate seed nodes. It must encode how multiple prerequisite activations jointly certify the activation of the root. This motivates RTW, a structured reverse witness that generalizes RR-sets from set-hitting conditions to threshold-gate satisfaction conditions.

4.2 RTW Construction

The main challenge in constructing an RTW is to preserve the reverse activation equivalence without collapsing threshold dependencies into ordinary node sets. For $r \in V$ and $\omega \in \Omega$, the RTW rooted at r , denoted by $\mathcal{W}_r^\omega$, is a monotone Boolean dependency structure that represents the reverse conditions for activating r under ω . Conceptually, each node $v \in V$ is associated with a Boolean formula node F_v , while the sampled RTW materializes only the part of this dependency structure reached during the reverse construction from r .

For each live ordinary edge $(u, v) \in E^\omega$, the RTW contains a dependency $F_u \Rightarrow F_v$. This dependency means that if u can be activated, then v can also be activated through the live edge (u, v) .

For an integer τ , let Γ_τ denote an at-least- τ gate, which becomes true if at least τ of its inputs are true. For each hyperedge $h \in H$ and each target node $v \in h$, the RTW contains a threshold gate $G_{h \rightarrow v} = \Gamma_{\tau_h}(\{F_u : u \in h \setminus \{v\}\})$. When $G_{h \rightarrow v}$ becomes true, it sets F_v to true.

Given a seed set S , we initialize F_v as true for every $v \in S$ and false otherwise, and repeatedly apply all dependencies and threshold gates until no new formula node or gate can become true. This process gives the least truth assignment closed under all RTW dependencies. We say that S satisfies $\mathcal{W}_r^\omega$, denoted by $S \models \mathcal{W}_r^\omega$, if F_r is true in this truth assignment. The following lemma states the key equivalence between the forward HTC and the RTW.

LEMMA 4.1. *For any possible world $\omega \in \Omega$, root node $r \in V$, and seed set $S \subseteq V$, $S \models \mathcal{W}_r^\omega$ if and only if $r \in A^\omega(S)$.*

The intuition is that the forward HTC and the RTW are induced by the same closure rules. A live ordinary edge and a satisfied hyperedge threshold rule activate a node in the forward HTC, and they make the corresponding formula node true in the RTW. Therefore, $r \in A^\omega(S)$ holds exactly when F_r becomes true, i.e., when $S \models \mathcal{W}_r^\omega$.

4.3 An Unbiased Estimator

We use RTWs to estimate the HTC influence spread. To generate one RTW sample, we first sample a root node r uniformly from V , sample a possible world $\omega \in \Omega$, and construct the witness $\mathcal{W}_r^\omega$. Given θ independent RTW samples $\mathcal{W}_1, \dots, \mathcal{W}_\theta$, we define the RTW estimator as

$$\widehat{\sigma}(S) = \frac{n}{\theta} \sum_{i=1}^{\theta} 1[S \models \mathcal{W}_i]. \quad (1)$$

THEOREM 4.2. *For any seed set $S \subseteq V$, the RTW estimator satisfies $\mathbb{E}[\widehat{\sigma}(S)] = \sigma(S)$.*

The RTW estimator generalizes the classical RR-set estimator. In ordinary edge-based diffusion, each reverse sample is a set, and the corresponding event is $S \cap R_i \neq \emptyset$. In HTC, each reverse sample is a threshold witness, and the corresponding event becomes $S \models \mathcal{W}_i$. Thus, the set-hitting condition is replaced by witness satisfaction.

PROPOSITION 4.3. *Maximizing $\widehat{\sigma}(S)$ subject to $|S| \leq k$ is NP-hard. Moreover, $\widehat{\sigma}(\cdot)$ is monotone but not submodular in general.*

The NP-hardness follows because ordinary maximum coverage is a special case, where every witness is an ordinary RR-set. The monotonicity follows from the monotone structure of RTWs, since adding more seeds cannot make a satisfied witness unsatisfied. The non-submodularity follows from threshold gates, where several nodes may jointly satisfy a witness even though each of them has zero marginal contribution alone. Therefore, the empirical optimization problem is a generalized maximum coverage problem rather than an ordinary maximum coverage problem.

5 ALGORITHMS

This section presents the algorithmic framework based on RTW samples. The main algorithmic difficulty is that RTW samples are structured witnesses rather than ordinary node sets, so both sampling and optimization must maintain threshold-gate dependencies instead of simple set coverage. For HTC-IM, the additional challenge is that the empirical RTW objective is non-submodular, so classical maximum-coverage guarantees do not apply.

5.1 RTW Sampling

The key challenge in RTW sampling is to construct the reverse witness without explicitly materializing a full possible world or expanding threshold gates into exponentially many activating subsets. To address this challenge, Algorithm 1 adopts a lazy sampling strategy in which ordinary edges are sampled only when encountered during reverse expansion and each hyperedge condition is stored as a high-arity threshold gate.

When expanding a node v , the algorithm inspects each incoming ordinary edge $(u, v) \in E$ and samples whether it is live. If (u, v) is live, the dependency $F_u \Rightarrow F_v$ is added to the witness, and u is inserted into the queue. For each hyperedge $h \in H$ with $v \in h$, the algorithm adds the threshold gate $G_{h \rightarrow v}$ and inserts all nodes in $h \setminus \{v\}$ into the queue.

Algorithm 1 is equivalent to first sampling a complete possible world and then constructing the RTW. This equivalence holds because ordinary edges are sampled independently according to their

Algorithm 1: *GenerateRTW*

Input: Hybrid hypergraph $\mathcal{G} = (V, E, H)$
Output: An RTW sample $\mathcal{W}_r^\omega$

- 1 Sample a root node r uniformly from V ;
- 2 Initialize $\mathcal{W}_r^\omega$ with root formula node F_r , queue $Q \leftarrow \{r\}$, and visited set $P \leftarrow \emptyset$;
- 3 **while** $Q \neq \emptyset$ **do**
- 4 Pop a node v from Q ;
- 5 **if** $v \in P$ **then**
- 6 | continue;
- 7 Insert v into P ;
- 8 **foreach** incoming ordinary edge $(u, v) \in E$ **do**
- 9 | Sample whether (u, v) is live with probability $p_{u,v}$;
- 10 | **if** (u, v) is live **then**
- 11 | Add F_u if absent, add dependency $F_u \Rightarrow F_v$, and push u into Q ;
- 12 | **foreach** hyperedge $h \in H$ such that $v \in h$ **do**
- 13 | Add threshold gate $G_{h \rightarrow v}$ to $\mathcal{W}_r^\omega$;
- 14 | **foreach** $u \in h \setminus \{v\}$ **do**
- 15 | Add F_u if absent and push u into Q ;
- 16 **return** $\mathcal{W}_r^\omega$;

propagation probabilities, while hyperedges contribute deterministic threshold gates. Algorithm 1 generates one RTW sample. To obtain θ independent samples, we simply repeat it θ times.

Storing $\mathcal{W}_r^\omega$. In implementation, each RTW sample is materialized as a sparse dependency graph rather than an explicit Boolean formula. For an RTW sample $\mathcal{W}_r^\omega$, we store four types of objects: formula nodes, ordinary dependencies, threshold gates, and gate-input incidences. For each formula node F_v , we store a Boolean state indicating whether F_v is currently true, and the list of outgoing ordinary dependencies induced by live edges in E^ω . For each live ordinary edge $(u, v) \in E^\omega$, we store an implication dependency $F_u \Rightarrow F_v$. For each threshold gate $G_{h \rightarrow v}$, we store the list of input formula nodes $\{F_u : u \in h \setminus \{v\}\}$, and a counter recording the number of currently true inputs. We also maintain the reverse gate-input incidences from each formula node F_u to the threshold gates that use F_u as an input. These incidences allow truth-value propagation to update only the affected gates. Importantly, we do not expand $G_{h \rightarrow v}$ into a DNF formula over τ_h -subsets of $h \setminus \{v\}$. Instead, $G_{h \rightarrow v}$ is stored as a single high-arity threshold object. Thus, the storage cost of this gate is linear in $|h|$, rather than proportional to the combinatorial number $\binom{|h|-1}{\tau_h}$ of threshold-satisfying subsets.

5.2 Algorithm for HTC-IE

By the RTW equivalence, HTC-IE reduces to estimating the probability that a given seed set satisfies a random RTW sample. We use this observation to design an estimator with relative-error guarantees.

Algorithm 2 presents the RTW-based estimator for HTC-IE. Given a seed set S , the algorithm repeatedly generates RTW samples and counts the number of samples satisfied by S . Instead of fixing

Algorithm 2: *RTW-Est for HTC-IE*

Input: Hybrid hypergraph $\mathcal{G} = (V, E, H)$, nonempty seed set $S \subseteq V$, error parameter ε , failure probability δ
Output: Influence estimate $\widehat{\sigma}(S)$

- 1 Set $\Lambda \leftarrow \left\lceil 2(1 + \varepsilon) \left(1 + \frac{\varepsilon}{3}\right) \frac{\ln(2/\delta)}{\varepsilon^2} \right\rceil$;
- 2 Set $N_{\text{sat}} \leftarrow 0$ and $t \leftarrow 0$;
- 3 **while** $N_{\text{sat}} < \Lambda$ **do**
- 4 Set $t \leftarrow t + 1$;
- 5 Generate an RTW sample $\mathcal{W}_t$ using Algorithm 1;
- 6 **if** $S \models \mathcal{W}_t$ **then**
- 7 | Set $N_{\text{sat}} \leftarrow N_{\text{sat}} + 1$;
- 8 Set $\widehat{\sigma}(S) \leftarrow nN_{\text{sat}}/t$;
- 9 **return** $\widehat{\sigma}(S)$;

the sample size in advance, it uses an integer-threshold stopping rule adapted from the improved generalized stopping rule in [58]. The algorithm stops once the number of satisfied RTW samples reaches the threshold Λ defined in Line 1 of Algorithm 2.

THEOREM 5.1. *For any fixed nonempty seed set $S \subseteq V$, Algorithm 2 returns an estimate $\widehat{\sigma}(S)$ such that $\Pr[|\widehat{\sigma}(S) - \sigma(S)| \leq \varepsilon \sigma(S)] \geq 1 - \delta$.*

The theorem follows by applying the improved generalized stopping rule in [58] to the Bernoulli random variable $1[S \models \mathcal{W}]$. By Lemma 4.1, its expectation is $\sigma(S)/n$. Therefore, the relative error guarantee for the Bernoulli mean directly implies the same relative error guarantee for the influence estimate after scaling by n . The use of the integer threshold Λ is a conservative version of the stopping threshold, since it rounds the threshold upward.

In Lines 5–7, Algorithm 2 separately invokes Algorithm 1 and then checks whether S satisfies the generated witness $\mathcal{W}_t$. This separation is only for presentation. In implementation, these two operations can be fused. Specifically, while constructing $\mathcal{W}_t$, we can simultaneously maintain the truth values induced by the fixed seed set S . If the root formula node becomes true during the construction, then $S \models \mathcal{W}_t$ is certified, and the algorithm can stop processing the current sample. If the construction terminates without making the root formula node true, then the sample is not satisfied by S . This fused implementation avoids materializing the full RTW whenever early satisfaction is detected, while preserving exactly the same sampling distribution and estimator.

5.3 Algorithm for HTC-IM

This subsection introduces *RTW-Greedy*, which combines upper-bound estimation, RTW sampling, and greedy selection. We first give an overview, and then present each component in detail.

5.3.1 Overview. Designing an RTW-based algorithm for HTC-IM is more challenging than HTC-IE because the seed set is not fixed. Instead, the algorithm must optimize over seed sets using sampled RTW witnesses. This brings two main technical challenges.

First, HTC-IM requires sample-size control for optimization rather than estimation of a fixed seed set. In HTC-IE, the adaptive stopping rule only needs to estimate the influence of a given seed set. In HTC-IM, however, the output seed set is selected based

on the generated RTW samples. Thus, the sample size must be large enough to ensure uniform concentration over all feasible seed sets.

Second, the sampled RTW objective is not an ordinary coverage objective. Unlike RR-sets, RTW samples are structured witnesses with threshold gates. Due to threshold-gate complementarity, both the HTC influence function $\sigma(\cdot)$ and the empirical RTW objective $\widehat{\sigma}(\cdot)$ are monotone but non-submodular in general, as stated in Theorem 3.3 and Proposition 4.3. Therefore, the classical $(1 - 1/e)$ -type greedy guarantee for submodular maximum coverage no longer applies, and we instead derive a solution-dependent guarantee.

RTW-Greedy addresses these challenges through a sample-then-optimize framework. It first uses an upper-relaxed model to compute an upper bound B on the optimum HTC influence value. This upper bound is then used to determine the number of RTW samples needed for uniform concentration. After generating these RTW samples, the algorithm greedily maximizes the empirical RTW objective $\widehat{\sigma}(\cdot)$. The resulting greedy solution is denoted by S^* . The algorithm also keeps the relaxed solution S^+ obtained from the upper-relaxed model. Although S^+ is optimized for the relaxed objective $\sigma^+(\cdot)$, it may still have strong HTC influence when the relaxation is tight. Therefore, *RTW-Greedy* compares S^* and S^+ on the same RTW samples and returns the one with larger empirical influence. This final comparison is used in the solution-dependent guarantee developed later.

5.3.2 Upper-Bound Estimation. We describe how to compute the upper bound B . The key idea is to relax every hyperedge threshold into an OR condition. In the upper-relaxed model, a target node $v \in h$ can be activated through a hyperedge h as long as at least one node in $h \setminus \{v\}$ is active. Let $\sigma^+(S)$ denote the influence spread under this upper-relaxed model. Since every relaxed hyperedge is at least as easy to trigger as the original threshold hyperedge, we have $\sigma(S) \leq \sigma^+(S)$ for every seed set $S \subseteq V$.

The upper-relaxed model can be reduced to a standard IC model on an expanded directed graph. For each ordinary edge $(u, v) \in E$, we keep the edge (u, v) with probability $p_{u,v}$. For each hyperedge $h \in H$, we add a deterministic directed edge (u, v) with probability 1 for every ordered pair $u, v \in h$ with $u \neq v$. Let this expanded graph be denoted by $\mathcal{G}^+ = (V, E^+)$. Then the IC influence spread on $\mathcal{G}^+$ is exactly $\sigma^+(S)$.

This reduction is efficient when the second-order hyperedge incidence size is not too large. Let $\kappa = \sum_{h \in H} |h|^2$. For each hyperedge h , the expansion adds $|h|(|h| - 1)$ deterministic directed edges. Therefore, the number of deterministic edges added by the expansion is $O(\kappa)$, and the expanded graph $\mathcal{G}^+$ has $O(m + \kappa)$ directed edges. In the common case where the number of hyperedges is of the same order as the number of ordinary edges, i.e., $q = O(m)$, and each hyperedge has constant size, we have $\kappa = O(q) = O(m)$. Thus, under this common sparse-hyperedge setting, $\mathcal{G}^+$ has linear size with respect to the original input size.

Let $OPT = \sigma(S^*)$ and $OPT^+ = \max_{|S| \leq k} \sigma^+(S)$. Since $\sigma(S) \leq \sigma^+(S)$ for every $S \subseteq V$, we have $OPT \leq OPT^+$. We run *OPIM-C* [46] on the expanded IC graph $\mathcal{G}^+$. Although the original *OPIM-C* algorithm returns a seed set, its online verification procedure maintains an upper bound on OPT^+ , as shown in Line 6 of Algorithm 2 in [46]. We adapt *OPIM-C* by exposing this maintained upper bound, and denote it by B . Then, with the success probability guaranteed by

Algorithm 3: *RTW-Greedy* for HTC-IM

Input: Hybrid hypergraph $\mathcal{G} = (V, E, H)$, budget k , error parameter ϵ , failure probability δ , user-specified allocation parameter $\lambda \in (0, 1)$

Output: Seed set S

- 1 Construct the expanded IC graph $\mathcal{G}^+ = (V, E^+)$ for the upper-relaxed model;
 - 2 Run *OPIM-C* [46] on $\mathcal{G}^+$ with budget k , error parameter $(1 - \lambda)\epsilon$, and failure probability $\delta/2$ to obtain S^+ and an upper bound B ;
 - 3 Set $\theta \leftarrow \left\lceil \frac{2n^2}{\lambda^2 \epsilon^2 B^2} \left(\ln \binom{n}{k} + \ln \frac{4}{\delta} \right) \right\rceil$;
 - 4 Generate θ independent RTW samples $\mathcal{W}_1, \dots, \mathcal{W}_\theta$ using Algorithm 1;
 - 5 Set $S^* \leftarrow \emptyset$;
 - 6 **for** $j = 1$ **to** k **do**
 - 7 Select $v^* \in \arg \max_{v \in V \setminus S^*} \widehat{\sigma}(S^* \cup \{v\}) - \widehat{\sigma}(S^*)$;
 - 8 Set $S^* \leftarrow S^* \cup \{v^*\}$;
 - 9 Set $S \leftarrow \arg \max_{S^o \in \{S^*, S^+\}} \widehat{\sigma}(S^o)$;
 - 10 **return** S ;
-

OPIM-C, B is a valid upper bound on the optimum value of HTC-IM, since $OPT \leq OPT^+ \leq B$.

5.3.3 Main Algorithm. Algorithm 3 presents the main algorithm for HTC-IM. It takes a user-specified allocation parameter $\lambda \in (0, 1)$, which controls how the error budget ϵ is split between the upper-relaxed *OPIM-C* step and the RTW sampling step. We will discuss how to set λ in Section 5.4.3.

The algorithm first constructs the expanded IC graph $\mathcal{G}^+$ for the upper-relaxed model and runs *OPIM-C* to obtain the relaxed solution S^+ and the upper bound B (Lines 1–2). It then uses B to determine the number θ of RTW samples and generates θ independent RTW samples (Lines 3–4). In this algorithm, $\widehat{\sigma}(\cdot)$ denotes the empirical estimator evaluated on the fixed RTW sample set generated in Line 4. The algorithm greedily maximizes the empirical RTW objective $\widehat{\sigma}(\cdot)$ (Lines 5–8). At each iteration, it selects the node with the largest empirical marginal gain and adds it to the current solution. Finally, the algorithm compares the greedy solution S^* with the relaxed solution S^+ on the same RTW samples and returns the one with larger empirical HTC influence (Lines 9–10).

We discuss the tie-breaking rule in the greedy step (Line 7). When multiple nodes have the same empirical marginal gain, we prefer the node that makes more unsatisfied RTWs closer to satisfaction. Let $\mathcal{U}(S) = \{i \in [\theta] : S \not\models \mathcal{W}_i\}$ be the index set of RTW samples not yet satisfied by the current seed set S . For a candidate node v , we define its auxiliary progress score as $\psi(v | S) = \sum_{i \in \mathcal{U}(S)} (d_i(S) - d_i(S \cup \{v\}))$, where $d_i(S)$ is the total remaining deficit of unsatisfied threshold gates in $\mathcal{W}_i$ under S . Specifically, for a threshold gate $G_{h \rightarrow u}$, its remaining deficit under S is $\max\{0, \tau_h - c_{h \rightarrow u}(S)\}$, where $c_{h \rightarrow u}(S)$ is the number of true input formula nodes of $G_{h \rightarrow u}$ under S . Then $d_i(S)$ is the sum of this deficit over all currently unsatisfied threshold gates in $\mathcal{W}_i$. Among nodes with the same empirical marginal gain, we choose the node

with the largest auxiliary progress score $\psi(v \mid S)$. If there is still a tie, we break it by a fixed deterministic order of node IDs.

Reverse-indexed marginal update. The greedy step in Lines 7–8 maintains the empirical marginal gain of each candidate node over the sampled RTW collection. To avoid recomputing the contribution of every candidate node over all RTW samples after each selection, we build a reverse index from each formula node to the RTW samples in which it appears. After selecting v^* , only RTW samples containing F_{v^*} can be affected. For each affected sample, we set F_{v^*} to true and propagate this change by a queue-based procedure. Whenever a formula node becomes true, the procedure follows its outgoing ordinary dependencies and increments the counters of incident threshold gates. Once a gate counter reaches its threshold, the target formula node becomes true and is inserted into the queue. If the root formula node becomes true, the corresponding RTW sample is marked as satisfied. The marginal-gain array is updated only for these affected samples. For each affected sample, we first remove its old contribution to the marginal gains, then perform truth-value propagation, and finally add its updated contribution back to the marginal-gain array. All other RTW samples remain unchanged. Therefore, Line 7 can select the node with the largest maintained empirical marginal gain without global recomputation.

5.3.4 Theoretical Guarantees. Our analysis follows the sandwich approximation framework of Lu et al. [32], which uses tractable bounding objectives to derive instance-dependent guarantees for non-submodular maximization. Since we only use the upper-relaxed objective $\sigma^+(\cdot)$, our analysis can be viewed as a one-sided, sampling-based instantiation of this framework.

THEOREM 5.2. *With probability at least $1 - \delta$, the seed set S returned by Algorithm 3 satisfies $\sigma(S) \geq \left((1 - 1/e) \frac{\sigma(S^+)}{\sigma^+(S^+)} - \epsilon \right) \cdot OPT$, where S^+ is the seed set obtained in Line 2.*

The ratio $\sigma(S^+)/\sigma^+(S^+)$ measures the tightness of the upper relaxation at S^+ . Thus, the guarantee approaches $1 - 1/e - \epsilon$ when the relaxation is tight. Here, S^+ provides the certified fallback, while the greedy solution S^* serves as an HTC-aware candidate that may achieve higher influence. Algorithm 3 compares the two candidates using the empirical RTW objective and returns the better one.

The guarantee in Theorem 5.2 depends on unknown true influence values, but Algorithm 3 also provides a computable certificate. With probability at least $1 - \delta$, the returned seed set satisfies

$$\sigma(S) \geq (\widehat{\sigma}(S)/B - \lambda\epsilon/2) \cdot OPT. \quad (2)$$

Therefore, the algorithm can report $\alpha = \max\{0, \widehat{\sigma}(S)/B - \lambda\epsilon/2\}$ as a computable solution-dependent lower bound on its approximation ratio.

5.3.5 A Practical Variant of RTW-Greedy. The sample size in Algorithm 3 is conservative because it is chosen to ensure uniform concentration over all feasible seed sets. On large graphs, the value of θ computed in Line 3 can be too large to make the algorithm practical. For better scalability, we also consider a practical variant, called *RTW-Greedy-P*, which separates candidate generation from solution validation.

RTW-Greedy-P first runs *OPIM-C* on the upper-relaxed graph $\mathcal{G}^+$ and obtains S^+ and B . It then takes a user-specified sample

size θ_0 , generates θ_0 RTW samples, and runs the same empirical greedy procedure as Algorithm 3 to obtain a candidate solution S' . After that, it independently calls Algorithm 2 to estimate the HTC influence of S' and S^+ , respectively. Then, it returns the one with the larger validation estimate.

The key point is that the validation samples are independent of the samples used to generate S' . Therefore, the validation step only needs to estimate the influence of two fixed candidate seed sets and does not require uniform concentration over all $\binom{n}{k}$ seed sets of size k , as Algorithm 3 does. The sample size θ_0 affects the quality of S' , while the independent validation step provides the statistical certificate. The algorithmic flow and correctness proof of *RTW-Greedy-P* are provided in the full version, since they follow a similar structure to those of *RTW-Greedy*.

5.4 Complexity Analysis

This subsection analyzes the expected time and space complexities of RTW sampling, *RTW-Est*, and *RTW-Greedy*.

5.4.1 Complexities of RTW Sampling and RTW-Est. For an RTW sample $\mathcal{W}_r^\omega$, let $|\mathcal{W}_r^\omega|$ denote its construction size, including formula nodes, inspected incoming ordinary edges, threshold gates, and gate-input incidences. Let $w = \mathbb{E}_{r,\omega}[|\mathcal{W}_r^\omega|]$ be the expected construction size. Algorithm 1 expands each formula node and inspects each of its incoming ordinary edges at most once, while storing each threshold condition as one gate object. Therefore, generating one RTW sample takes expected time and space $O(w)$.

The quantity w is the RTW analogue of the average RR-set size in classical RIS. When a formula node F_v is expanded, the algorithm inspects the incoming ordinary edges of v and the hyperedges containing v , so the total local expansion cost over all nodes is $O(n + m + \kappa)$. Moreover, the probability that F_v appears in a random RTW sample is bounded by the probability that v can reach a uniformly sampled root in the upper-relaxed model, which is at most OPT^+/n . Thus, $w = O((OPT^+/n)(n + m + \kappa))$.

For HTC-IE, let $p_S = \Pr[S \models \mathcal{W}]$. By Lemma 4.1, $p_S = \sigma(S)/n$. Algorithm 2 stops when the number of satisfied RTW samples reaches Λ , so the expected number of generated samples is Λ/p_S . Using $O(w)$ as the expected construction cost per RTW sample, the expected running time is

$$O(\Lambda w/p_S) = O(OPT^+(n + m + \kappa) \ln(1/\delta)/(\sigma(S)\epsilon^2)).$$

The fused implementation used in our experiments evaluates $S \models \mathcal{W}$ during RTW construction and can only reduce the average cost. Since samples are processed one by one, the expected working space is $O(w)$.

5.4.2 Complexity of RTW-Greedy. Let m^+ be the number of directed edges in the expanded IC graph $\mathcal{G}^+$. By construction, $m^+ \leq m + \sum_{h \in H} |h|(|h| - 1) = O(m + \kappa)$. According to the complexity of *OPIM-C* [46], the call to *OPIM-C* on $\mathcal{G}^+$ takes expected time $O((k \ln n + \ln(1/\delta))(n + m + \kappa)/((1 - \lambda)\epsilon^2))$, where $(1 - \lambda)\epsilon$ is the error parameter assigned to *OPIM-C*.

When *OPIM-C* succeeds, the upper bound B is within a constant factor of OPT^+ . Therefore, the number of RTW samples is $\theta = O((k \ln n + \ln(1/\delta))/(\lambda^2 \epsilon^2 (OPT^+/n)^2))$. Generating and storing all RTW samples take expected time and space $O(\theta w) = O(n(n + m + \kappa)(k \ln n + \ln(1/\delta))/(\lambda^2 \epsilon^2 OPT^+))$.

For greedy selection, we maintain sparse reverse indices over sampled RTWs and update only the witnesses and threshold gates affected by newly selected nodes. Since each formula node and each threshold gate in a sampled RTW changes its state at most once, the state-maintenance cost is $O(\theta w)$ in expectation. We conservatively upper bound the marginal-gain maintenance cost over k greedy iterations by $O(k\theta w)$.

Combining the *OPIM-C* call and the greedy marginal-gain maintenance cost, the total expected running time of Algorithm 3 is

$$O\left(\frac{(k \ln n + \ln(1/\delta))(n + m + \kappa)}{\varepsilon^2} \left(\frac{1}{(1-\lambda)^2} + \frac{kn}{\lambda^2 OPT^+}\right)\right). \quad (3)$$

The expected space for storing the sampled RTWs is $O(n(n + m + \kappa)(k \ln n + \ln(1/\delta))/(\lambda^2 \varepsilon^2 OPT^+))$.

5.4.3 Choice of λ . The allocation parameter λ controls how the total error budget is split between the *OPIM-C* call and the RTW sampling step. A larger λ assigns more error budget to RTW sampling, which reduces the number of RTW samples. At the same time, it assigns a smaller error budget $(1-\lambda)\varepsilon$ to *OPIM-C*, which increases the cost of the upper-bound computation.

From the complexity bound above, the λ -dependent term in Eq. (3) is $g(\lambda) = \frac{1}{(1-\lambda)^2} + \frac{kn}{\lambda^2 OPT^+}$. Minimizing $g(\lambda)$ over $\lambda \in (0, 1)$ gives $\lambda^\circ = \frac{\sqrt[3]{kn/OPT^+}}{1 + \sqrt[3]{kn/OPT^+}}$. This value balances the cost of the *OPIM-C* call and the RTW-based greedy phase. The expression also gives a useful interpretation. When OPT^+ is large, fewer RTW samples are needed, and a smaller λ can be used. When OPT^+ is small, the RTW sampling term becomes more expensive, and λ° moves closer to one. Since OPT^+ is unknown, λ can be treated as a user-specified parameter. In practice, one can set $\lambda = 1/2$. In high-influence instances where $OPT^+ = \Theta(n)$, the theoretically optimized choice reduces to $\lambda^\circ = \Theta\left(\frac{\sqrt[3]{k}}{1 + \sqrt[3]{k}}\right)$.

5.5 Practical Regime and Limitations

The scalability of RTW-based algorithms is governed by the expected RTW construction size w . This dependence is inherent to preserving an unbiased RTW estimator, because when the reverse construction reaches a hyperedge, the corresponding threshold condition cannot be reduced to a single node-set hit. Instead, the RTW must materialize the threshold gate and its input incidences so that witness satisfaction remains exactly equivalent to root activation in the forward HTC process.

This requirement also reveals the main limitation of our framework. When hyperedges are highly overlapping and form a giant higher-order dependency component, a single RTW sample may materialize dependencies over a large fraction of the graph, causing high time and memory costs, especially for *RTW-Greedy*, which stores many RTW samples for empirical optimization. Thus, our algorithms are most suitable for sparse or moderately overlapping HTC instances where the expected RTW construction size remains manageable. Handling dense hypergraphs with large overlapping group structures may require compressed or approximate RTW representations, which we leave for future work.

Table 1: Statistics of datasets.

Dataset	n	m	q	Avg. $ h $	Type
<i>BlogCatalog</i>	5.20K	343K	41	6.78	Undirected
<i>Flickr</i>	7.58K	479K	107	7.01	Undirected
<i>CiaoDVD</i>	19.5K	40.1K	691	7.99	Directed
<i>GitHub</i>	37.7K	578K	657	7.43	Undirected
<i>Epinions</i>	114K	717K	396	9.23	Directed
<i>Pokec</i>	1.63M	30.6M	868	6.45	Directed
<i>Yelp</i>	1.99M	14.6M	855	6.00	Undirected

6 EXPERIMENTAL EVALUATION

This section presents the experimental setup and evaluation results. Our experiments are designed to answer the following questions.

- How accurate and efficient is *RTW-Est* for HTC-IE?
- How robust is *RTW-Est* across different HTC-IE settings?
- How effective and scalable are the proposed algorithms, *RTW-Greedy* and *RTW-Greedy-P*, for HTC-IM?
- How do model and algorithmic settings affect *RTW-Greedy* and *RTW-Greedy-P*?

6.1 Experimental Setup

We describe the experimental setup in this subsection.

6.1.1 Datasets. We evaluate the proposed algorithms on seven real-world datasets, including *BlogCatalog* [24, 29], *Flickr* [24, 29], *CiaoDVD* [19], *GitHub* [27, 41], *Epinions* [45], *Pokec* [27], and *Yelp* [53]. For datasets with undirected ordinary edges, we replace each undirected edge with two directed edges in opposite directions, following a common practice in the literature [46–48]. Table 1 reports their statistics. On the largest datasets, the hyperedge components are relatively sparse compared with the pairwise graphs, which matches the practical regime discussed in Section 5.5. This raises a natural question: do hyperedges still contribute to the HTC process under such sparse structures? The answer is affirmative, as shown in Section 6.3.1 by comparison with the *Edge-Only* baseline introduced in Section 6.1.2.

Recall that we need to obtain small and cohesive groups intended to resemble close-knit social circles in which within-group reinforcement is plausible, such as small groups of close friends. For datasets that provide node features, including all datasets except *CiaoDVD*, we construct candidate hyperedges from shared feature combinations. Notably, constructing hyperedges from node features is a common practice in prior studies [2, 8, 44, 57]. We first filter features by their support, i.e., the number of nodes containing each feature, and keep the top- K discriminative features for each node according to inverse document frequency (IDF) [43]. The value of K is set according to the dataset scale and feature density, with $K = 30$ for *BlogCatalog*, *Flickr*, and *Epinions*, $K = 20$ for *Pokec* and *Yelp*, and $K = 15$ for *GitHub*. We then enumerate all size-3 feature combinations for each node. For each feature combination, all nodes containing this combination form a candidate hyperedge. For *BlogCatalog*, *Flickr*, and *Epinions*, each candidate hyperedge is further split into multiple hyperedges according to the connected components induced by the ordinary graph, so as to preserve local coherence. For *CiaoDVD*, candidate hyperedges are generated from

users who give high ratings to the same movie within the same month. Finally, we keep hyperedges with sizes between 5 and 25 and remove duplicate hyperedges. This construction avoids overly broad hyperedges and produces semantically specific groups based on shared discriminative features. The preprocessing details for each dataset are provided in the full version.

6.1.2 Baselines. To the best of our knowledge, HTC-IE has not been directly studied in prior work. We therefore compare *RTW-Est* with the following baselines.

- *MC-HTC* [26]. This simple yet effective baseline estimates $\sigma(S)$ by directly running MC simulations of the forward HTC. For a fair comparison when estimating $\sigma(S)$ with relative-error guarantees, we set the number of MC simulations according to a standard two-sided multiplicative Chernoff bound [34], using $\sigma(S) \geq k$ as a conservative lower bound.
- *RTW-Fixed-10K* and *RTW-Fixed-100K*. These two baselines are heuristic fixed-sample variants of *RTW-Est*. They use the same RTW estimator as *RTW-Est*, but fix the number of RTW samples to 10,000 and 100,000, respectively, instead of using the adaptive stopping rule. For a fair comparison, both baselines use the same early-termination implementation as *RTW-Est*, which stops processing the current sample once the seed set has satisfied the witness. This implementation follows the early stopping idea of *HIST* [21] and does not affect the correctness of the RTW estimator.
- *Upper-relaxed IE (UR-IE)*. This baseline estimates the upper-relaxed influence $\sigma^+(S)$ instead of the original HTC influence $\sigma(S)$. It constructs the expanded IC graph $\mathcal{G}^+$, generates ordinary RR-sets, and estimates $\sigma^+(S)$ by the standard RR-set coverage estimator. To obtain a reliable estimate, we use the state-of-the-art adaptive stopping rule in [58].

For HTC-IM, we compare *RTW-Greedy* and *RTW-Greedy-P* with the following baselines.

- *MC-Greedy* [26]. This baseline directly applies the greedy algorithm to the HTC objective. At each iteration, it estimates each marginal gain by 10,000 MC simulations.
- *UR-IM*. This baseline optimizes the upper-relaxed objective $\sigma^+(\cdot)$ using the state-of-the-art IM algorithm *HIST* [21].
- *Edge-Only*. This baseline ignores all hyperedges and runs *HIST* [21] on the ordinary graph induced by the edge set E .
- *HTC Clique Expansion (HTC-CE)*. This baseline adapts the idea of converting hyperedges into ordinary edges from [55] to HTC-IM. For each hyperedge h , it adds a directed edge (u, v) for every ordered pair $u, v \in h$ with $u \neq v$. The propagation probability of each expanded edge is set to $(|h| - \tau_h) / (|h| - 1)$, which serves as a heuristic threshold-splitting weight over the $|h| - 1$ possible pairwise predecessors of each target node. It then runs *HIST* on the resulting graph.
- *HCI1-TM* and *HCI2-TM* [56]. These baselines rank nodes by the first-order and second-order hypergraph collective influence scores, respectively, and select the top k nodes.
- *HN-MOEA* [16]. This baseline formulates hypergraph IM as a multi-objective problem that maximizes influence spread

and minimizes seed-set size. For a direct comparison with other methods, we fix its seed set size to k .

Overall, *MC-Greedy* directly optimizes the HTC-IM objective and serves as a standard greedy baseline. *UR-IM*, *Edge-Only*, and *HTC-CE* optimize tractable surrogate objectives of HTC-IM by reducing them to ordinary graph IM problems and solving the resulting instances using *HIST*. *HCI1-TM*, *HCI2-TM*, and *HN-MOEA* are adapted from existing hypergraph IM methods under related settings.

6.1.3 Parameter Settings. For each ordinary edge (u, v) , the propagation probability is set to $1/\text{InDeg}(v)$, where $\text{InDeg}(v)$ denotes the in-degree of v in the ordinary graph. For each hyperedge h , the default threshold τ_h is sampled uniformly at random from the integers in $[2, |h| - 1]$. We use the same sampled thresholds for all algorithms on the same dataset. We set the error parameter to $\epsilon = 0.2$ and the failure probability to $\delta = 0.01$.

For HTC-IE, unless otherwise stated, we set $k = 100$ and consider two types of input seed sets. The first consists of the top- k nodes ranked by the number of outgoing ordinary edges plus the number of incident hyperedges. The second consists of k nodes selected uniformly at random.

For HTC-IM, we use $k = 100$ as the default seed budget. For *RTW-Greedy* and *RTW-Greedy-P*, we set the user-specified allocation parameter to $\lambda = \sqrt[3]{k} / (1 + \sqrt[3]{k})$. For *RTW-Greedy-P*, we set the user-specified sample size to $\theta_0 = 10,000$ on the five smaller datasets and $\theta_0 = 100,000$ on the two largest datasets, *Pokec* and *Yelp*, to account for the larger graph scale. This scale-dependent setting is justified by the parameter study in Section 6.3.6.

6.1.4 Implementation Details. All algorithms are implemented in C++ and compiled with the -O3 optimization flag. Experiments are conducted on a Linux server with 251 GB memory. Each run is subject to a time limit of two days and a memory limit of 100 GB. Each experiment is repeated 10 times, and we report the average results. For HTC-IM, after an algorithm returns a seed set, we evaluate its influence using 10,000 independent realizations.

6.2 Results for HTC-IE

We evaluate the proposed estimator *RTW-Est* for HTC-IE.

6.2.1 Effect of Seed-Set Size and Selection Strategy. We first evaluate the effect of input seed sets by varying the seed set size k . Figures 2 and 3 report the estimated influence value and runtime, respectively. As shown in Figure 2, all methods except *UR-IE* produce almost identical estimated influence values across different seed set sizes. This is consistent with our expectation, since these methods directly estimate the same influence objective under the HTC model, while *UR-IE* estimates an upper-bound surrogate and can therefore produce different values.

Figure 3 shows that the runtime generally decreases as k increases. This trend is also expected, because a larger seed set is more likely to satisfy a sampled witness, allowing witness-based estimators to terminate sample processing earlier. Compared with *MC-HTC*, *RTW-Est* improves efficiency by at least one order of magnitude on all datasets. The fixed sample baseline *RTW-Fixed-10K* can be faster than *RTW-Est* on *Pokec* and *Yelp* when k is small, but it is generally slower on the other datasets and becomes less competitive as k increases. In contrast, *RTW-Fixed-100K* is consistently

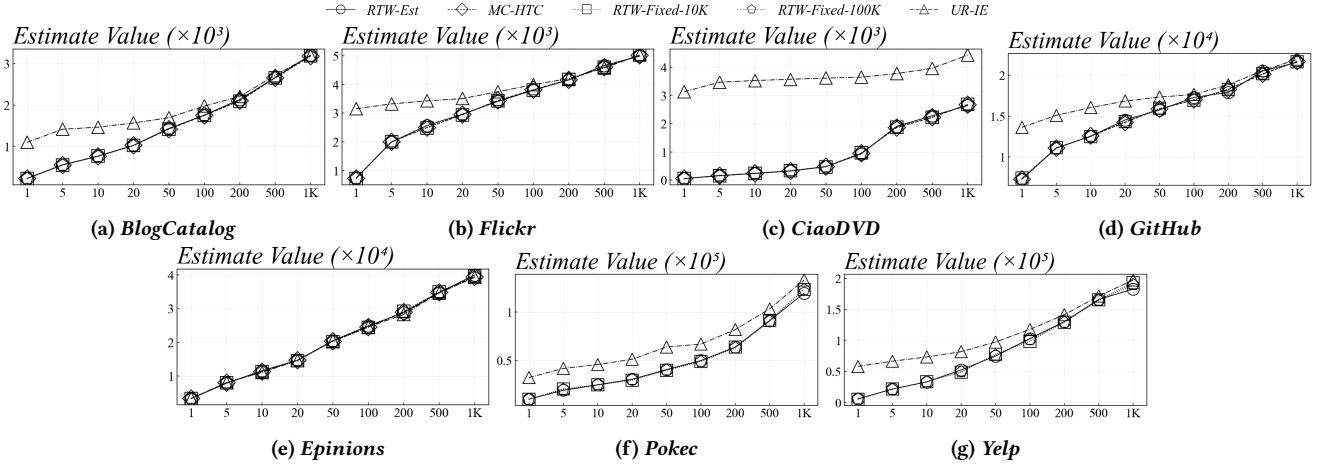


Figure 2: Estimated influence value vs. seed set size k for different methods on each dataset.

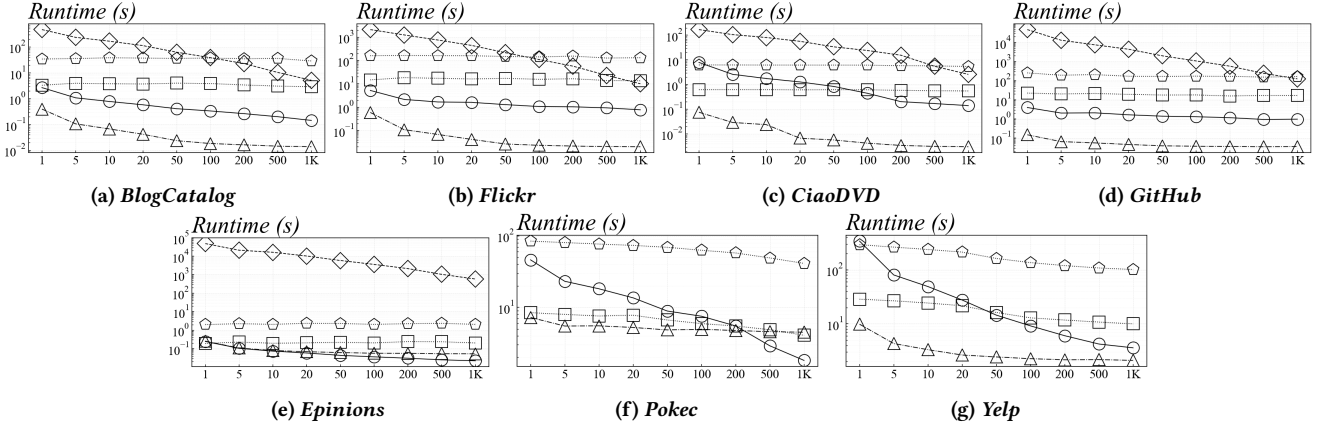


Figure 3: Runtime vs. seed set size k for different methods on each dataset.

slower than *RTW-Est* and is about one order of magnitude slower than *RTW-Fixed-10K*, which matches the difference in their fixed sample sizes. As reported in the full version, the sample-size trend is consistent with the runtime trend, and *RTW-Est* uses substantially fewer samples while remaining within the same order of magnitude as *UR-IE*, which estimates the upper-relaxed surrogate rather than the HTC influence.

We also repeat the experiment using randomly selected seed sets. The estimated influence values remain consistent for all methods except *UR-IE*, and the runtime still decreases as k increases. On *Pokec* and *Yelp*, *RTW-Est* can be slower than the fixed sample baselines, because randomly selected seeds tend to have lower influence and therefore satisfy sampled witnesses with lower probability. On the other datasets, *RTW-Est* remains the fastest method.

6.2.2 Effect of Estimation Parameters ϵ and δ . We study the effect of ϵ and δ in HTC-IE. The estimated influence values remain stable as either parameter varies. Larger ϵ substantially reduces the sample size and runtime of *RTW-Est*, while larger δ brings a milder reduction because the confidence term only affects the sample complexity logarithmically. Detailed trends are reported in the full version.

6.2.3 Effect of Hyperedge Threshold Distribution. To study the effect of hyperedge activation difficulty, we compare the default uniform threshold setting with two additional threshold sampling schemes. For each additional scheme and each hyperedge h , a sampled ratio $\rho \in [0, 1]$ is mapped to $\tau_h = 2 + \lceil \rho(|h| - 3) \rceil$, clipped to $[2, |h| - 1]$. In the exponential setting, we set $\rho = 1 - x$, where x is sampled from a truncated exponential distribution over $[0, 1]$, so larger thresholds are sampled with higher probability. In the mixed setting, we sample ρ uniformly from $\{0.3, 0.5, 0.7\}$, representing low, medium, and high activation requirements.

The results reported in the full version show that *RTW-Est* is robust to different threshold distributions. Changing the threshold distribution has little effect on the estimated influence values, runtime, or relative comparison among methods, and *RTW-Est* is either the fastest accurate estimator or comparably fast in most cases.

6.2.4 Effect of the Number of Hyperedges q . We further evaluate the effect of the number of hyperedges q on both IM and IE. For each dataset, we construct three variants, denoted by *Base*, *High*, and *XHigh*, where *Base* is the default hyperedge construction setting. The *High* and *XHigh* variants are obtained by progressively relaxing

the hyperedge construction settings of *Base*, with *XHigh* applying a stronger relaxation than *High*. To isolate the effect of q , the ordinary directed graph is kept fixed across the three variants, while only the hyperedges are regenerated. The detailed settings used to generate *High* and *XHigh* are provided in the full version.

The appendix results show that the estimated influence generally increases with q , because additional hyperedges introduce more threshold-based activation opportunities. The runtime and sample size of *RTW-Est* also increase in denser variants, especially on large datasets such as *Pokec*, since additional hyperedges enlarge the reverse witnesses and increase sample-processing cost.

6.3 Results for HTC-IM

We evaluate *RTW-Greedy* and *RTW-Greedy-P* for HTC-IM.

6.3.1 Effect of Seed Budget k . Figure 4 reports the objective value under different seed budgets. When k is small, most methods except *HCI1-TM* and *HCI2-TM* achieve similar objective values. As k increases, especially on larger datasets, *RTW-Greedy* and *RTW-Greedy-P* achieve higher objective values than the baselines. Moreover, *RTW-Greedy-P* achieves objective values close to *RTW-Greedy*, showing that the practical variant preserves most of the solution quality while improving scalability. One exception is *CiaoDVD*, where *HCI1-TM* and *HCI2-TM* achieve higher objective values when k is large. This indicates that hypergraph centrality heuristics can sometimes find strong seed sets, but their weaker performance on other datasets suggests limited robustness across HTC instances. The comparison with *Edge-Only* further clarifies the role of hyperedges. Since *Edge-Only* selects seeds by ignoring hyperedge activation, its gap from *RTW-Greedy* and *RTW-Greedy-P*, especially for larger k , indicates that hyperedges play an active role in the propagation process. This answers the question raised in Section 6.1.1.

Figure 5 reports the runtime under different seed budgets. *RTW-Greedy* is substantially slower than the other methods and becomes infeasible on large datasets such as *Pokec* and *Yelp*, because it stores many RTW samples and performs greedy selection over them. By contrast, *RTW-Greedy-P* runs on all datasets and remains within an acceptable runtime range. The runtime of *RTW-Greedy* and *RTW-Greedy-P* is relatively stable as k increases, suggesting that their cost is dominated by sample generation and validation rather than by the final seed-set size. Overall, *RTW-Greedy-P* provides a scalable alternative to *RTW-Greedy* while maintaining strong objective values.

Figures 6 and 7 report the memory usage and the solution-dependent lower-bound ratio α on two representative datasets, *CiaoDVD* and *Epinions*. Results on the remaining datasets are reported in the full version. *RTW-Greedy-P* uses much less memory than *RTW-Greedy*, since it avoids storing the large number of RTW samples required by the certified version. For α , *CiaoDVD* and *Epinions* show different but complementary behaviors. On *CiaoDVD*, α is zero when k is very small, but increases to a positive constant as k grows. On *Epinions*, α remains at a positive constant across different seed budgets. These results show that the solution-dependent certificate remains nontrivial and practically meaningful.

6.3.2 Effect of Estimation Parameters ε and δ . We study the effect of ε and δ in HTC-IM. The appendix results show that the objective

values of *RTW-Greedy* and *RTW-Greedy-P* remain largely stable as either parameter varies. A larger ε reduces the runtime and memory usage of *RTW-Greedy*, but has only a mild effect on *RTW-Greedy-P*. The lower-bound ratio α decreases with ε but remains nontrivial, and both efficiency and α are stable under different values of δ .

6.3.3 Effect of Hyperedge Threshold Distribution. We further evaluate HTC-IM under the same threshold distributions as in Section 6.2.3. As shown in the full version, *RTW-Greedy* achieves the highest objective values when feasible, while *RTW-Greedy-P* is consistently second-best or comparable. The heuristic baselines are less stable and can be sensitive to the threshold distribution, whereas runtime, memory usage, and α remain largely stable for the proposed methods.

6.3.4 Effect of the Number of Hyperedges q . We evaluate HTC-IM on the *Base*, *High*, and *XHigh* variants described in Section 6.2.4. These variants test whether RTW-based methods become more advantageous as threshold-based hyperedge effects become stronger. As shown in the full version, the cross-method comparison remains consistent with the main experiment. When q increases, additional hyperedges create more opportunities for complementary threshold-gate activations, making the HTC objective harder to capture by upper-relaxed surrogate methods. Accordingly, *RTW-Greedy* and *RTW-Greedy-P* generally achieve higher objective values, while runtime, memory usage, and the lower-bound ratio α remain stable in most cases.

6.3.5 Effect of Allocation Parameter λ . We study the effect of the allocation parameter λ . The experimental results, reported in the full version, show that the objective values remain relatively stable within the tested range, with *RTW-Greedy* slightly outperforming *RTW-Greedy-P*. Increasing λ generally reduces the runtime, memory usage, and solution-dependent lower-bound ratio α of *RTW-Greedy*, while *RTW-Greedy-P* remains largely stable. Overall, λ mainly affects the cost and guarantee of *RTW-Greedy*, rather than the solution quality.

6.3.6 Effect of Sample Budget θ_0 . We study the effect of the sample budget θ_0 used by *RTW-Greedy-P*. The experimental results show that increasing θ_0 generally improves the objective value and the solution-dependent lower-bound ratio α , but also increases runtime and memory usage. On the five smaller datasets, $\theta_0 = 10K$ already achieves objective values close to $\theta_0 = 100K$ with much lower cost. On *Pokec* and *Yelp*, $\theta_0 = 100K$ brings a more visible improvement. These results justify our default setting of $\theta_0 = 10K$ for the smaller datasets and $\theta_0 = 100K$ for *Pokec* and *Yelp*.

6.3.7 Ablation Study. *RTW-Greedy* uses the upper-relaxed model to obtain an upper bound B , which is then used to determine the RTW sample size. To evaluate the importance of this upper-bound step, we introduce *RTW-TrivialUB*, a variant that skips the upper-relaxed optimization and directly uses the trivial upper bound $B = n$.

Appendix results show that *RTW-TrivialUB* can achieve competitive objective values on several datasets, but its solution-dependent lower-bound ratio α is much smaller and even drops to zero in some cases. This confirms that the upper-bound computation based on the upper-relaxed model is important for controlling the RTW sample size and maintaining a nontrivial lower-bound ratio.

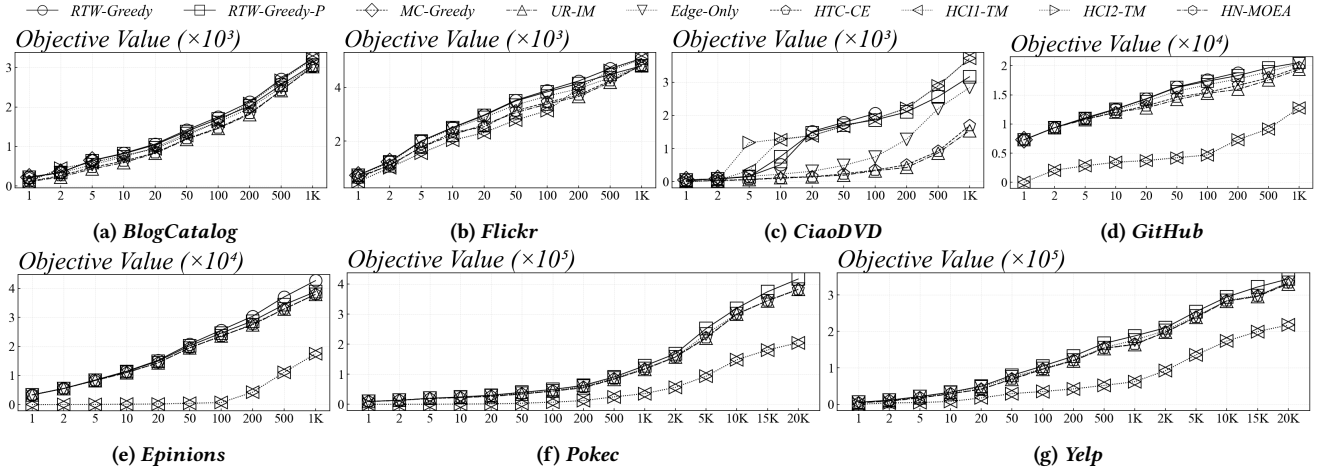


Figure 4: Objective value vs. seed budget k for different methods on each dataset.

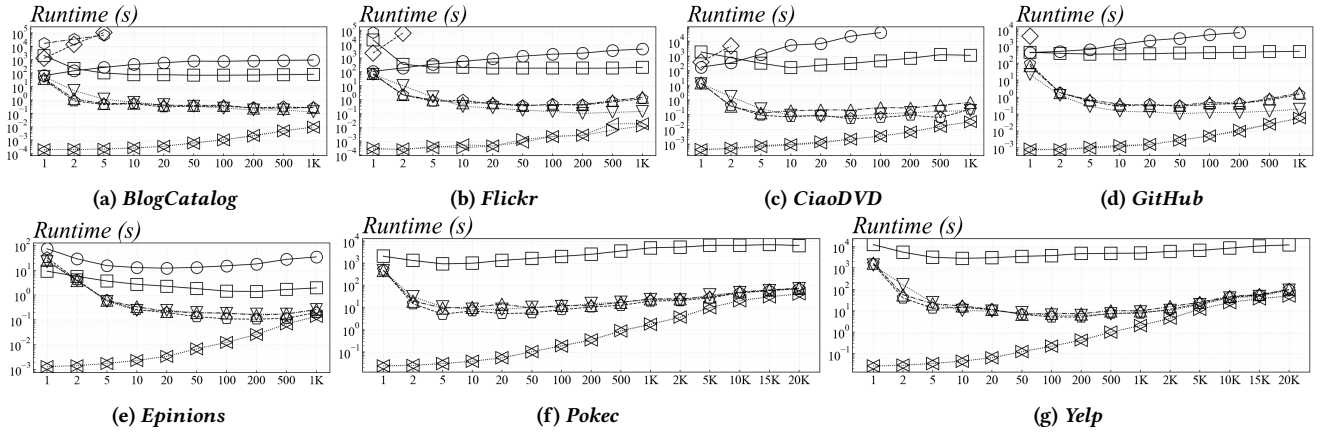


Figure 5: Runtime vs. seed budget k for different methods on each dataset.

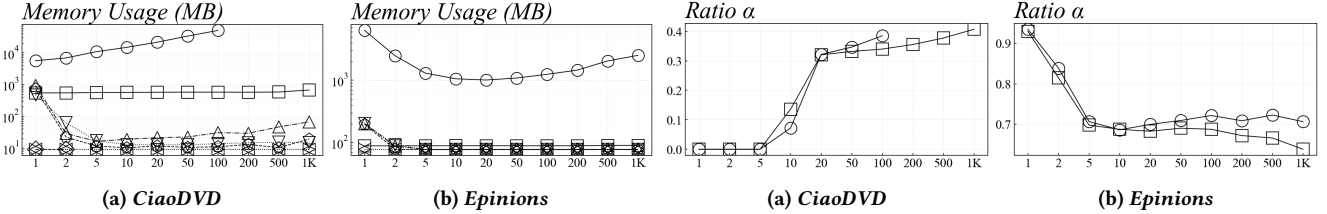


Figure 6: Memory usage (MB) vs. seed budget k for different methods on *CiaoDVD* and *Epinions*.

7 CONCLUSION

In this paper, we studied the HTC model, where pairwise probabilistic propagation and threshold-based hyperedge activation coexist, and formulated the corresponding HTC-IE and HTC-IM problems. For HTC-IE, we developed an efficient RTW-based estimator with relative-error guarantees. For HTC-IM, we proposed *RTW-Greedy* with a solution-dependent approximation guarantee and the scalable variant *RTW-Greedy-P*. Experiments on seven real-world datasets demonstrate accurate influence estimation, effective seed selection, and practical scalability.

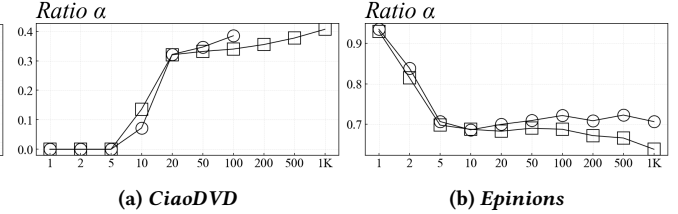


Figure 7: Solution-dependent lower-bound ratio α vs. seed budget k for different methods on *CiaoDVD* and *Epinions*.

Several directions remain for future work. First, it is useful to study HTC on dynamic or temporal hypergraphs, where pairwise edges and hyperedges may evolve over time. Second, propagation probabilities and hyperedge thresholds can be learned from cascade data rather than treated as given. Third, the scalability of theoretically guaranteed HTC-IM algorithms can be further improved for billion-scale networks.

ACKNOWLEDGMENTS

This work was supported by JSPS KAKENHI Grant Number JP22H00533.

REFERENCES

- [1] Junaid Ali, Mahmoudreza Babaei, Abhijnan Chakraborty, Baharan Mirza-soleiman, Krishna P. Gummadi, and Adish Singla. 2023. On the Fairness of Time-Critical Influence Maximization in Social Networks. *IEEE Transactions on Knowledge and Data Engineering* 35, 3 (2023), 2875–2886. <https://doi.org/10.1109/TKDE.2021.3120561>
- [2] Alessia Antelmi, Gennaro Cordasco, Carmine Spagnuolo, and Przemysław Szufel. 2021. Social Influence Maximization in Hypergraphs. *Entropy* 23, 7 (2021). <https://doi.org/10.3390/e23070796>
- [3] Cigdem Aslay, Nicola Barbieri, Francesco Bonchi, and Ricardo Baeza-Yates. 2014. Online Topic-aware Influence Maximization Queries. In *Proceedings of the 17th International Conference on Extending Database Technology*. 295–306.
- [4] Federico Battiston, Giulia Cencetti, Iacopo Iacopini, Vito Latora, Maxime Lucas, Alice Patania, Jean-Gabriel Young, and Giovanni Petri. 2020. Networks beyond pairwise interactions: Structure and dynamics. *Physics Reports* 874 (2020), 1–92. <https://doi.org/10.1016/j.physrep.2020.05.004>
- [5] Christian Borgs, Michael Brautbar, Jennifer Chayes, and Brendan Lucier. 2014. Maximizing social influence in nearly optimal time. In *Proceedings of the Twenty-Fifth Annual ACM-SIAM Symposium on Discrete Algorithms* (Portland, Oregon) (SODA '14). Society for Industrial and Applied Mathematics, USA, 946–957.
- [6] Taotao Cai, Jianxin Li, Ajmal Mian, Rong-Hua Li, Timos Sellis, and Jeffrey Xu Yu. 2022. Target-Aware Holistic Influence Maximization in Spatial Social Networks. *IEEE Transactions on Knowledge and Data Engineering* 34, 4 (2022), 1993–2007. <https://doi.org/10.1109/TKDE.2020.3003047>
- [7] Damon Centola and Michael Macy. 2007. Complex Contagions and the Weakness of Long Ties. *Amer. J. Sociology* 113, 3 (2007), 702–734. <https://doi.org/10.1086/521848>
- [8] Amartya Chakraborty and Nandini Mukherjee. 2025. Topic and opinion infused hypergraphs for influence maximization. *Physics Letters A* 545 (2025), 130507. <https://doi.org/10.1016/j.physleta.2025.130507>
- [9] Shuo Chen, Ju Fan, Guoliang Li, Jianhua Feng, Kian-lee Tan, and Jinhui Tang. 2015. Online topic-aware influence maximization. *Proc. VLDB Endow.* 8, 6 (Feb. 2015), 666–677. <https://doi.org/10.14778/2735703.2735706>
- [10] Wei Chen, Wei Lu, and Ning Zhang. 2012. Time-critical influence maximization in social networks with time-delayed diffusion process. In *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence* (Toronto, Ontario, Canada) (AAAI'12). AAAI Press, 592–598. <https://doi.org/10.1609/aaai.v26i1.8204>
- [11] Wei Chen, Chi Wang, and Yajun Wang. 2010. Scalable influence maximization for prevalent viral marketing in large-scale social networks. In *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Washington, DC, USA) (KDD '10). Association for Computing Machinery, New York, NY, USA, 1029–1038. <https://doi.org/10.1145/1835804.1835934>
- [12] Wei Chen, Yajun Wang, and Siyu Yang. 2009. Efficient influence maximization in social networks. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Paris, France) (KDD '09). Association for Computing Machinery, New York, NY, USA, 199–208. <https://doi.org/10.1145/1557019.1557047>
- [13] Yi-Cheng Chen, Wen-Yuan Zhu, Wen-Chih Peng, Wang-Chien Lee, and Suh-Yin Lee. 2014. CIM: Community-Based Influence Maximization in Social Networks. *ACM Trans. Intell. Syst. Technol.* 5, 2, Article 25 (April 2014), 31 pages. <https://doi.org/10.1145/2532549>
- [14] Edith Cohen, Daniel Delling, Thomas Pajor, and Renato F. Werneck. 2014. Sketch-based Influence Maximization and Computation: Scaling up with Guarantees. In *Proceedings of the 23rd ACM International Conference on Conference on Information and Knowledge Management* (Shanghai, China) (CIKM '14). Association for Computing Machinery, New York, NY, USA, 629–638. <https://doi.org/10.1145/2661829.2662077>
- [15] Guilherme Ferraz de Arruda, Giovanni Petri, and Yamir Moreno. 2020. Social contagion models on hypergraphs. *Phys. Rev. Res.* 2 (Apr 2020), 023032. Issue 2. <https://doi.org/10.1103/PhysRevResearch.2.023032>
- [16] Stefano Genetti, Eros Ribaga, Elia Cunegatti, Quintino F. Lotito, and Giovanni Iacca. 2024. Influence Maximization in Hypergraphs Using Multi-Objective Evolutionary Algorithms. In *Parallel Problem Solving from Nature – PPSN XVIII*, Michael Affenzeller, Stephan M. Winkler, Anna V. Kononova, Heike Trautmann, Tea Tušar, Penousal Machado, and Thomas Bäck (Eds.). Springer Nature Switzerland, Cham, 217–235.
- [17] Xulu Gong, Hanchen Wang, Xiaoyang Wang, Chen Chen, Wenjie Zhang, and Ying Zhang. 2024. Influence maximization on hypergraphs via multi-hop influence estimation. *Information Processing & Management* 61, 3 (2024), 103683. <https://doi.org/10.1016/j.ipm.2024.103683>
- [18] Mark Granovetter. 1978. Threshold Models of Collective Behavior. *Amer. J. Sociology* 83, 6 (1978), 1420–1443. <https://doi.org/10.1086/226707>
- [19] Guibing Guo, Jie Zhang, Daniel Thalmann, and Neil Yorke-Smith. 2014. ETAF: An extended trust antecedents framework for trust prediction. In *2014 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM 2014)*. 540–547. <https://doi.org/10.1109/ASONAM.2014.6921639>
- [20] Jianxiong Guo, Qiufen Ni, Weili Wu, and Ding-Zhu Du. 2024. Composite Community-Aware Diversified Influence Maximization With Efficient Approximation. *IEEE/ACM Transactions on Networking* 32, 2 (2024), 1584–1599. <https://doi.org/10.1109/TNET.2023.3321870>
- [21] Qintian Guo, Sibao Wang, Zhewei Wei, and Ming Chen. 2020. Influence Maximization Revisited: Efficient Reverse Reachable Set Generation with Bound Tightened. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (Portland, OR, USA) (SIGMOD '20). Association for Computing Machinery, New York, NY, USA, 2167–2181. <https://doi.org/10.1145/3318464.3389740>
- [22] Qintian Guo, Sibao Wang, Zhewei Wei, Wenqing Lin, and Jing Tang. 2022. Influence Maximization Revisited: Efficient Sampling with Bound Tightened. *ACM Trans. Database Syst.* 47, 3, Article 12 (Aug. 2022), 45 pages. <https://doi.org/10.1145/3533817>
- [23] Taha Halal, Bogdan Cautis, Benoît Groz, and Ruizhe Gao. 2025. Topic-aware influence maximization with deep reinforcement learning and graph attention networks. *Data Min. Knowl. Discov.* 39, 6 (Aug. 2025), 35. <https://doi.org/10.1007/s10618-025-01133-3>
- [24] Xiao Huang, Jundong Li, and Xia Hu. 2017. Label Informed Attributed Network Embedding. In *Proceedings of the Tenth ACM International Conference on Web Search and Data Mining* (Cambridge, United Kingdom) (WSDM '17). Association for Computing Machinery, New York, NY, USA, 731–739. <https://doi.org/10.1145/3018661.3018667>
- [25] Iacopo Iacopini, Giovanni Petri, Alain Barrat, and Vito Latora. 2019. Simplicial models of social contagion. *Nature Communications* 10, 1 (2019), 2485.
- [26] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Washington, D.C.) (KDD '03). Association for Computing Machinery, New York, NY, USA, 137–146. <https://doi.org/10.1145/956750.956769>
- [27] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>.
- [28] Guoliang Li, Shuo Chen, Jianhua Feng, Kian-lee Tan, and Wen-syan Li. 2014. Efficient location-aware influence maximization. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data* (Snowbird, Utah, USA) (SIGMOD '14). Association for Computing Machinery, New York, NY, USA, 87–98. <https://doi.org/10.1145/2588555.2588561>
- [29] Jundong Li, Xia Hu, Jiliang Tang, and Huan Liu. 2015. Unsupervised Streaming Feature Selection in Social Media. In *Proceedings of the 24th ACM International Conference on Information and Knowledge Management* (Melbourne, Australia) (CIKM '15). Association for Computing Machinery, New York, NY, USA, 1041–1050. <https://doi.org/10.1145/2806416.2806501>
- [30] Bo Liu, Gao Cong, Yifeng Zeng, Dong Xu, and Yeow Meng Chee. 2014. Influence Spreading Path and Its Application to the Time Constrained Social Influence Maximization Problem and Beyond. *IEEE Transactions on Knowledge and Data Engineering* 26, 8 (2014), 1904–1917. <https://doi.org/10.1109/TKDE.2013.106>
- [31] Xingjie Liu, Qi He, Yuan Yuan Tian, Wang-Chien Lee, John McPherson, and Jiawei Han. 2012. Event-based social networks: linking the online and offline social worlds. In *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Beijing, China) (KDD '12). Association for Computing Machinery, New York, NY, USA, 1032–1040. <https://doi.org/10.1145/2339530.2339693>
- [32] Wei Lu, Wei Chen, and Laks V. S. Lakshmanan. 2015. From competition to complementarity: comparative influence diffusion and maximization. *Proc. VLDB Endow.* 9, 2 (Oct. 2015), 60–71. <https://doi.org/10.14778/2850578.2850581>
- [33] Athul M A and Arun Rajkumar. 2022. Hyper-IMRANK: Ranking-based Influence Maximization for Hypergraphs. In *Proceedings of the 5th Joint International Conference on Data Science & Management of Data* (9th ACM IKDD CODS and 27th COMAD) (Bangalore, India) (CODS-COMAD '22). Association for Computing Machinery, New York, NY, USA, 100–104. <https://doi.org/10.1145/3493700.3493706>
- [34] Rajeev Motwani and Prabhakar Raghavan. 1995. *Randomized Algorithms*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511814075>
- [35] George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. 1978. An analysis of approximations for maximizing submodular set functions—I. *Mathematical programming* 14 (1978), 265–294. <https://doi.org/10.1007/BF01588971>
- [36] Hung T. Nguyen, Tri P. Nguyen, NhatHai Phan, and Thang N. Dinh. 2017. Importance Sketching of Influence Dynamics in Billion-Scale Networks. In *2017 IEEE International Conference on Data Mining (ICDM)*. 337–346. <https://doi.org/10.1109/ICDM.2017.43>
- [37] Hung T. Nguyen, Tri P. Nguyen, Tam N. Vu, and Thang N. Dinh. 2017. Outward Influence and Cascade Size Estimation in Billion-scale Networks. *Proc. ACM Meas. Anal. Comput. Syst.* 1, 1, Article 20 (June 2017), 30 pages. <https://doi.org/10.1145/3084457>
- [38] Hung T. Nguyen, My T. Thai, and Thang N. Dinh. 2016. Stop-and-Stare: Optimal Sampling Algorithms for Viral Marketing in Billion-scale Networks. In *Proceedings of the 2016 International Conference on Management of Data* (San Francisco, California, USA) (SIGMOD '16). Association for Computing Machinery, New York, NY, USA, 695–710. <https://doi.org/10.1145/2882903.2915207>

- [39] Jiezhong Qiu, Yixuan Li, Jie Tang, Zheng Lu, Hao Ye, Bo Chen, Qiang Yang, and John E. Hopcroft. 2016. The Lifecycle and Cascade of WeChat Social Messaging Groups. In *Proceedings of the 25th International Conference on World Wide Web* (Montréal, Québec, Canada) (WWW '16). International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 311–320. <https://doi.org/10.1145/2872427.2882979>
- [40] Xilong Qu, Wenbin Pei, Yingchao Yang, Xirong Xu, Renquan Zhang, and Qiang Zhang. 2024. Influence maximization in hypergraphs using a genetic algorithm with new initialization and evaluation methods. *arXiv preprint arXiv:2405.09185* (2024).
- [41] Benedek Rozemberczki, Carl Allen, and Rik Sarkar. 2021. Multi-Scale attributed node embedding. *Journal of Complex Networks* 9, 2 (04 2021), cnab014. <https://doi.org/10.1093/comnet/cnab014>
- [42] Gal Sadeh, Edith Cohen, and Haim Kaplan. 2020. Sample Complexity Bounds for Influence Maximization. In *11th Innovations in Theoretical Computer Science Conference (ITCS 2020) (Leibniz International Proceedings in Informatics (LIPIcs))*, Thomas Vidick (Ed.), Vol. 151. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 29:1–29:36. <https://doi.org/10.4230/LIPIcs.ITCS.2020.29>
- [43] Gerard Salton and Christopher Buckley. 1988. Term-weighting approaches in automatic text retrieval. *Information Processing & Management* 24, 5 (1988), 513–523. [https://doi.org/10.1016/0306-4573\(88\)90021-0](https://doi.org/10.1016/0306-4573(88)90021-0)
- [44] Yanhao Sun, Jie Wu, Nuan Song, Tianwei Lin, Longxiang Li, and Dong Li. 2025. Deep reinforcement learning-based influence maximization for heterogeneous hypergraphs. *Physica A: Statistical Mechanics and its Applications* 660 (2025), 130361. <https://doi.org/10.1016/j.physa.2025.130361>
- [45] Jiliang Tang, Huiji Gao, Huan Liu, and Atish Das Sarma. 2012. eTrust: understanding trust evolution in an online world. In *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Beijing, China) (KDD '12). Association for Computing Machinery, New York, NY, USA, 253–261. <https://doi.org/10.1145/2339530.2339574>
- [46] Jing Tang, Xueyan Tang, Xiaokui Xiao, and Junsong Yuan. 2018. Online Processing Algorithms for Influence Maximization. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) (SIGMOD '18). Association for Computing Machinery, New York, NY, USA, 991–1005. <https://doi.org/10.1145/3183713.3183749>
- [47] Youze Tang, Yanchen Shi, and Xiaokui Xiao. 2015. Influence Maximization in Near-Linear Time: A Martingale Approach. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data* (Melbourne, Victoria, Australia) (SIGMOD '15). Association for Computing Machinery, New York, NY, USA, 1539–1554. <https://doi.org/10.1145/2723372.2723734>
- [48] Youze Tang, Xiaokui Xiao, and Yanchen Shi. 2014. Influence maximization: near-optimal time complexity meets practical efficiency. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data* (Snowbird, Utah, USA) (SIGMOD '14). Association for Computing Machinery, New York, NY, USA, 75–86. <https://doi.org/10.1145/2588555.2593670>
- [49] Yu Wang, Gao Cong, Guojie Song, and Kunqing Xie. 2010. Community-based greedy algorithm for mining top-K influential nodes in mobile social networks. In *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Washington, DC, USA) (KDD '10). Association for Computing Machinery, New York, NY, USA, 1039–1048. <https://doi.org/10.1145/1835804.1835935>
- [50] Yanhao Wang, Yuchen Li, Ju Fan, and Kian-Lee Tan. 2018. Location-aware Influence Maximization over Dynamic Social Streams. *ACM Trans. Inf. Syst.* 36, 4, Article 43 (July 2018), 35 pages. <https://doi.org/10.1145/3230871>
- [51] Duncan J. Watts. 2002. A simple model of global cascades on random networks. *Proceedings of the National Academy of Sciences* 99, 9 (2002), 5766–5771. <https://doi.org/10.1073/pnas.082090499>
- [52] Ming Xie, Xiu-Xiu Zhan, Chuang Liu, and Zi-Ke Zhang. 2023. An efficient adaptive degree-based heuristic algorithm for influence maximization in hypergraphs. *Information Processing & Management* 60, 2 (2023), 103161. <https://doi.org/10.1016/j.ipm.2022.103161>
- [53] Yelp Inc. 2022. Yelp Dataset. <https://www.kaggle.com/datasets/yelp-dataset/yelp-dataset>
- [54] Yulin Yu, Julie Jiang, and Paramveer S. Dhillon. 2024. Characterizing the Structure of Online Conversations Across Reddit. *Proc. ACM Hum.-Comput. Interact.* 8, CSCW2, Article 374 (Nov. 2024), 23 pages. <https://doi.org/10.1145/3686913>
- [55] Lingling Zhang, Tiancheng Lu, Zhiping Shi, Zhiwei Zhang, Ye Yuan, and Guoren Wang. 2025. Efficient Algorithms for Influence Maximization in Hypergraphs by Stratified Sampling. *IEEE Transactions on Knowledge and Data Engineering* 37, 11 (2025), 6392–6405. <https://doi.org/10.1109/TKDE.2025.3599790>
- [56] Renquan Zhang, Xilong Qu, Qiang Zhang, Xirong Xu, and Sen Pei. 2024. Influence maximization based on threshold models in hypergraphs. *Chaos: An Interdisciplinary Journal of Nonlinear Science* 34, 2 (02 2024), 023111. <https://doi.org/10.1063/5.0178329>
- [57] Jianming Zhu, Junlei Zhu, Smita Ghosh, Weili Wu, and Jing Yuan. 2019. Social Influence Maximization in Hypergraph in Social Networks. *IEEE Transactions on Network Science and Engineering* 6, 4 (2019), 801–811. <https://doi.org/10.1109/TNSE.2018.2873759>

Algorithm 4: RTW-Greedy-P

Input: Hybrid hypergraph $\mathcal{G} = (V, E, H)$, budget k , error parameter ϵ , failure probability δ , allocation parameter $\lambda \in (0, 1)$, training sample size θ_0

Output: Seed set S

- 1 Construct the expanded IC graph $\mathcal{G}^+ = (V, E^+)$;
 - 2 Run *OPIM-C* on $\mathcal{G}^+$ with budget k , error parameter $(1 - \lambda)\epsilon$, and failure probability $\delta/2$ to obtain S^+ and B ;
 - 3 Generate θ_0 independent RTW samples $\mathcal{W}_1, \dots, \mathcal{W}_{\theta_0}$;
 - 4 Run the empirical greedy procedure on $\mathcal{W}_1, \dots, \mathcal{W}_{\theta_0}$ to obtain a candidate solution S' ;
 - 5 Set $\eta \leftarrow \lambda\epsilon/2$;
 - 6 Call Algorithm 2 with seed set S' , error parameter η , and failure probability $\delta/4$, and denote the returned estimate by $\tilde{\sigma}(S')$;
 - 7 Call Algorithm 2 with seed set S^+ , error parameter η , and failure probability $\delta/4$, and denote the returned estimate by $\tilde{\sigma}(S^+)$;
 - 8 Set $S \leftarrow \arg \max_{T \in \{S', S^+\}} \tilde{\sigma}(T)$;
 - 9 **return** S ;
-

- [58] Yuqing Zhu, Jing Tang, Xueyan Tang, Sibao Wang, and Andrew Lim. 2023. 2-hop+ Sampling: Efficient and Effective Influence Estimation. *IEEE Transactions on Knowledge and Data Engineering* 35, 2 (Feb 2023), 1088–1103. <https://doi.org/10.1109/TKDE.2021.3093934>

A ALGORITHMIC FLOW OF RTW-GREEDY-P

This section summarizes the algorithmic flow of *RTW-Greedy-P*. The algorithm uses a user-specified sample size θ_0 to generate a candidate solution, and then independently validates the candidate solution together with the relaxed solution S^+ . The validation step calls Algorithm 2, and the validation samples are independent of the samples used for candidate generation.

Algorithm 4 differs from Algorithm 3 in the way RTW samples are used. In Algorithm 3, the sample size is chosen to support uniform concentration over all feasible seed sets. In *RTW-Greedy-P*, the samples used for greedy selection are treated only as training samples. The statistical validation is performed separately by independent calls to Algorithm 2. Therefore, the practical variant avoids the large sample size in Line 3 while still allowing the returned solution to be certified through independent validation.

THEOREM A.1. *Under the parameter settings in Algorithm 4, with probability at least $1 - \delta$, the seed set S returned by Algorithm 4 satisfies*

$$\sigma(S) \geq \left((1 - 1/e) \frac{\sigma(S^+)}{\sigma^+(S^+)} - \epsilon \right) \cdot \text{OPT},$$

where S^+ is the seed set obtained by the *OPIM-C* call on the upper-relaxed graph $\mathcal{G}^+$.

The proof is provided in Section B.5.

In addition to the guarantee in Theorem A.1, *RTW-Greedy-P* provides a computable solution-dependent certificate. Let $\tilde{\sigma}(S)$ denote the independent validation estimate associated with the returned seed set S , and recall that $\eta = \lambda\epsilon/2$. With probability at least $1 - \delta$,

we have

$$\sigma(S) \geq \left(\frac{\bar{\sigma}(S)}{B} - \eta \right) \cdot \text{OPT}.$$

Therefore, *RTW-Greedy-P* can report

$$\alpha = \max \left\{ 0, \frac{\bar{\sigma}(S)}{B} - \frac{\lambda \varepsilon}{2} \right\}$$

as a computable solution-dependent lower bound on its approximation ratio.

Complexity. Let w denote the expected size of one RTW sample, and let $p_T = \sigma(T)/n$ for a seed set T . The *OPIM-C* call on the expanded graph $\mathcal{G}^+$ has the same expected time complexity as in Algorithm 3, namely $O((k \ln n + \ln(1/\delta))(n + m + \kappa)/((1 - \lambda)^2 \varepsilon^2))$. The training phase generates θ_0 RTW samples and runs the empirical greedy procedure on them. Using the same sparse update implementation as Algorithm 3, this phase takes expected time $O(k\theta_0 w)$ and space $O(\theta_0 w)$. The validation phase calls Algorithm 2 twice, once for S' and once for S^+ . With $\eta = \lambda \varepsilon/2$, the stopping threshold in the validation calls is $\Lambda_\eta = O(\ln(1/\delta)/(\lambda^2 \varepsilon^2))$. Thus, the expected validation time is $O(\Lambda_\eta w/p_{S'} + \Lambda_\eta w/p_{S^+})$, and the expected working space for validation is $O(w)$. Therefore, the expected time of *RTW-Greedy-P* is the sum of the *OPIM-C* cost, the training cost $O(k\theta_0 w)$, and the validation cost, while its dominant additional space beyond *OPIM-C* is $O(\theta_0 w)$.

B MISSING PROOFS

This section provides the proofs omitted from the main text. We first prove the equivalence between HTC diffusion and the RTW witness formulation. We then present the proofs of the estimator guarantees, the upper-bound construction, and the approximation results for HTC-IM.

B.1 Proof of Lemma 4.1

Fix a possible world $\omega \in \Omega$, a root node $r \in V$, and a seed set $S \subseteq V$. Let P_r^ω denote the set of nodes whose formula nodes are materialized in the RTW $\mathcal{W}_r^\omega$. That is, $v \in P_r^\omega$ if and only if the formula node F_v appears in $\mathcal{W}_r^\omega$. Let $T_r^\omega(S)$ denote the set of materialized nodes whose formula nodes become true after evaluating $\mathcal{W}_r^\omega$ from the initial assignment induced by S . That is,

$$T_r^\omega(S) = \{v \in P_r^\omega : F_v \text{ becomes true}\}.$$

We prove the stronger local statement

$$T_r^\omega(S) = A^\omega(S) \cap P_r^\omega.$$

Since $r \in P_r^\omega$, this statement directly implies that F_r becomes true in $\mathcal{W}_r^\omega$ if and only if $r \in A^\omega(S)$.

We first prove $T_r^\omega(S) \subseteq A^\omega(S) \cap P_r^\omega$. The inclusion $T_r^\omega(S) \subseteq P_r^\omega$ follows from the definition of $T_r^\omega(S)$. It remains to show $T_r^\omega(S) \subseteq A^\omega(S)$. Consider the least-fixed-point evaluation of the RTW and order formula nodes by the first time they become true. We prove by induction on this order that every node whose formula node becomes true belongs to $A^\omega(S)$.

For the base case, if F_v is initially true, then $v \in S$, and hence $v \in A^\omega(S)$. For the induction step, suppose F_v becomes true during RTW evaluation. There are two possible cases. First, F_v becomes true through an ordinary dependency $F_u \Rightarrow F_v$. By construction of the RTW, this dependency corresponds to a live edge $(u, v) \in E^\omega$.

By the induction hypothesis, $u \in A^\omega(S)$. Since $A^\omega(S)$ is closed under live ordinary edges, we have $v \in A^\omega(S)$. Second, F_v becomes true through a satisfied threshold gate $G_{h \rightarrow v}$. Then at least τ_h input formula nodes F_u , where $u \in h \setminus \{v\}$, have already become true. By the induction hypothesis, these input nodes all belong to $A^\omega(S)$. Since $A^\omega(S)$ is closed under the hyperedge-threshold rule, we have $v \in A^\omega(S)$. Therefore, every node in $T_r^\omega(S)$ belongs to $A^\omega(S)$, and hence

$$T_r^\omega(S) \subseteq A^\omega(S) \cap P_r^\omega.$$

We next prove $A^\omega(S) \cap P_r^\omega \subseteq T_r^\omega(S)$. Let $A_0 = S$, and let $A_1, A_2, \dots$ be the standard iterative construction of the least forward HTC closure under the possible world ω . That is, A_{t+1} is obtained from A_t by adding every node that can be activated either through a live ordinary edge from a node in A_t , or through a hyperedge whose threshold is satisfied by nodes in A_t . Then $A^\omega(S) = \bigcup_{t \geq 0} A_t$.

We prove by induction on t that for every node $v \in A_t \cap P_r^\omega$, the formula node F_v becomes true in $\mathcal{W}_r^\omega$. For $t = 0$, every $v \in A_0 \cap P_r^\omega$ belongs to S , so F_v is initially true in the RTW evaluation.

Assume the claim holds for A_t , and consider any node $v \in A_{t+1} \cap P_r^\omega$. If $v \in A_t$, then the claim follows directly from the induction hypothesis. Otherwise, v is newly added at step $t + 1$. There are two possible activation rules.

First, v is activated through a live ordinary edge $(u, v) \in E^\omega$ with $u \in A_t$. Since $v \in P_r^\omega$, the reverse construction of $\mathcal{W}_r^\omega$ expands v and inspects its incoming ordinary edges. Thus, the RTW contains the dependency $F_u \Rightarrow F_v$, and F_u is materialized, which implies $u \in P_r^\omega$. Because $u \in A_t \cap P_r^\omega$, the induction hypothesis gives that F_u becomes true. Therefore, the dependency $F_u \Rightarrow F_v$ makes F_v true.

Second, v is activated through a hyperedge h such that

$$|A_t \cap (h \setminus \{v\})| \geq \tau_h.$$

Choose any τ_h nodes $u_1, \dots, u_{\tau_h}$ from $A_t \cap (h \setminus \{v\})$. Since $v \in P_r^\omega$, the reverse construction of $\mathcal{W}_r^\omega$ expands v , adds the threshold gate $G_{h \rightarrow v}$, and materializes every input formula node F_u for $u \in h \setminus \{v\}$. In particular, $u_1, \dots, u_{\tau_h} \in P_r^\omega$. By the induction hypothesis, $F_{u_1}, \dots, F_{u_{\tau_h}}$ all become true. Hence, the gate $G_{h \rightarrow v}$ is satisfied and makes F_v true.

Thus, for every $v \in A_{t+1} \cap P_r^\omega$, F_v becomes true in $\mathcal{W}_r^\omega$. By induction over t , every node in $A^\omega(S) \cap P_r^\omega$ belongs to $T_r^\omega(S)$. Therefore,

$$A^\omega(S) \cap P_r^\omega \subseteq T_r^\omega(S).$$

Combining the two inclusions, we obtain

$$T_r^\omega(S) = A^\omega(S) \cap P_r^\omega.$$

Since $r \in P_r^\omega$, the equality $T_r^\omega(S) = A^\omega(S) \cap P_r^\omega$ implies that F_r becomes true in $\mathcal{W}_r^\omega$ if and only if $r \in A^\omega(S)$. By the definition of witness satisfaction, this means that $S \models \mathcal{W}_r^\omega$ exactly when $r \in A^\omega(S)$. This proves the lemma. $\square$

B.2 Proof of Theorem 4.2

Fix a seed set $S \subseteq V$. For one RTW sample, let r be the root node sampled uniformly from V , and let $\omega \in \Omega$ be the sampled possible world. Define $X(S; r, \omega) = 1[S \models \mathcal{W}_r^\omega]$. By Lemma 4.1, we have $X(S; r, \omega) = 1[r \in A^\omega(S)]$. Conditioned on a fixed possible world

ω , the randomness only comes from the uniformly sampled root r . Therefore,

$$\mathbb{E}_r[X(S; r, \omega) \mid \omega] = \Pr[r \in A^\omega(S) \mid \omega] = \frac{|A^\omega(S)|}{n}.$$

Taking expectation over ω , we obtain

$$\mathbb{E}_{r, \omega}[X(S; r, \omega)] = \mathbb{E}_\omega \left[\frac{|A^\omega(S)|}{n} \right] = \frac{\sigma(S)}{n}.$$

Recall that $\mathcal{W}_1, \dots, \mathcal{W}_\theta$ are θ independent RTW samples. For each sample $\mathcal{W}_i$, define $X_i(S) = \mathbf{1}[S \models \mathcal{W}_i]$. Then $\mathbb{E}[X_i(S)] = \sigma(S)/n$. By Eq. (1) and linearity of expectation,

$$\mathbb{E}[\widehat{\sigma}(S)] = \frac{n}{\theta} \sum_{i=1}^{\theta} \mathbb{E}[X_i(S)] = \frac{n}{\theta} \cdot \theta \cdot \frac{\sigma(S)}{n} = \sigma(S).$$

This proves the theorem. $\square$

B.3 Proof of Theorem 5.1

We first state an integer-threshold version of the improved generalized stopping rule.

LEMMA B.1. *Let $X_1, X_2, \dots$ be random variables satisfying $X_i \in \{0, 1\}$ and $\mathbb{E}[X_i \mid X_1, \dots, X_{i-1}] = \mu$ for every $i \geq 1$, where $\mu \in (0, 1]$. Let $0 < \varepsilon, \delta < 1$, and define $\Lambda = \left\lceil 2(1 + \varepsilon) \left(1 + \frac{\varepsilon}{3}\right) \frac{\ln(2/\delta)}{\varepsilon^2} \right\rceil$. Let T be the first time such that $\sum_{i=1}^T X_i \geq \Lambda$, and let $\widehat{\mu} = \frac{1}{T} \sum_{i=1}^T X_i$. Then $\Pr[|\widehat{\mu} - \mu| \leq \varepsilon\mu] \geq 1 - \delta$.*

Lemma B.1 follows from Theorem 3 of [58]. The original improved stopping rule is stated for bounded random variables and uses a real-valued threshold Υ . In our case, $X_i \in \{0, 1\}$, so the bounded range is $[0, 1]$, and the corresponding threshold becomes $2(1 + \varepsilon)(1 + \varepsilon/3) \ln(2/\delta)/\varepsilon^2$. Algorithm 2 uses its rounded-up integer threshold $\Lambda = \lceil \Upsilon \rceil$. Since $\Lambda \geq \Upsilon$, the integer threshold is conservative. Moreover, because the variables are Bernoulli, the stopping sample must satisfy $X_T = 1$, and the accumulated sum at stopping is exactly Λ . Thus, the fractional adjustment used in the original improved stopping rule is unnecessary in our integer-threshold variant.

PROOF OF THEOREM 5.1. Fix a nonempty seed set $S \subseteq V$. For the i -th RTW sample, define $X_i(S) = \mathbf{1}[S \models \mathcal{W}_i]$. By Theorem 4.2, each $X_i(S)$ has expectation $\sigma(S)/n$. Since the RTW samples are generated independently, we have

$$\mathbb{E}[X_i(S) \mid X_1(S), \dots, X_{i-1}(S)] = \frac{\sigma(S)}{n}.$$

Therefore, the sequence $X_1(S), X_2(S), \dots$ satisfies the condition of Lemma B.1 with $\mu = \sigma(S)/n$.

Let T be the stopping time of Algorithm 2, and let $N_{\text{sat}} = \sum_{i=1}^T X_i(S)$. The algorithm returns $\widehat{\sigma}(S) = nN_{\text{sat}}/T$. Equivalently, if $\widehat{\mu} = N_{\text{sat}}/T$, then $\widehat{\sigma}(S) = n\widehat{\mu}$.

By Lemma B.1, with probability at least $1 - \delta$, we have $|\widehat{\mu} - \mu| \leq \varepsilon\mu$. Multiplying both sides by n , we obtain

$$|\widehat{\sigma}(S) - \sigma(S)| = n|\widehat{\mu} - \mu| \leq n\varepsilon\mu = \varepsilon\sigma(S).$$

Hence, $\Pr[|\widehat{\sigma}(S) - \sigma(S)| \leq \varepsilon\sigma(S)] \geq 1 - \delta$. This proves the theorem. $\square$

B.4 Proofs of Theorem 5.2 and Eq. (2)

Before proving the theorem, we first introduce a useful concentration inequality.

LEMMA B.2 (Hoeffding's Inequality). *Let $Y_1, \dots, Y_\theta$ be independent random variables with $Y_i \in [0, 1]$. Let $\bar{Y} = \frac{1}{\theta} \sum_{i=1}^{\theta} Y_i$ and $\mu = \mathbb{E}[\bar{Y}]$. For any $a > 0$, we have*

$$\Pr[|\bar{Y} - \mu| \geq a] \leq 2 \exp(-2\theta a^2). \quad (4)$$

PROOF OF THEOREM 5.2 AND EQ. (2). Let $\widehat{\sigma}(\cdot)$ denote the empirical RTW estimator induced by the samples generated in Line 4. Let $\mathcal{S}_k = \{T \subseteq V : |T| = k\}$. Let $c = 1 - 1/e - (1 - \lambda)\varepsilon$. We assume $c > 0$, which is required by the *OPIM-C* call.

By the guarantee of *OPIM-C* in Line 2 of Algorithm 3, with probability at least $1 - \delta/2$, the exposed upper bound B satisfies $OPT \leq OPT^+ \leq B$, and the verification condition implies $B \leq \sigma^+(S^+)/c$ (see Algorithm 2 in [46]).

Next, we prove the uniform concentration of the empirical RTW estimator. Conditioned on B , the RTW samples are independent. For any fixed $T \in \mathcal{S}_k$, define $Y_i(T) = \mathbf{1}[T \models \mathcal{W}_i]$. Then $Y_i(T) \in [0, 1]$, $\widehat{\sigma}(T)/n = \frac{1}{\theta} \sum_{i=1}^{\theta} Y_i(T)$, and $\mathbb{E}[Y_i(T)] = \sigma(T)/n$. By Eq. (4), with $a = \lambda\varepsilon B/(2n)$, we have

$$\Pr \left[|\widehat{\sigma}(T) - \sigma(T)| > \frac{\lambda\varepsilon B}{2} \right] \leq 2 \exp \left(-\frac{\theta \lambda^2 \varepsilon^2 B^2}{2n^2} \right). \quad (5)$$

By the choice of θ in Line 3, the right-hand side of Eq. (5) is at most $\delta/(2\binom{n}{k})$. Taking a union bound over all seed sets in $\mathcal{S}_k$, with probability at least $1 - \delta/2$, the following inequality holds for every $T \in \mathcal{S}_k$:

$$|\widehat{\sigma}(T) - \sigma(T)| \leq \frac{\lambda\varepsilon B}{2}. \quad (6)$$

By another union bound, the *OPIM-C* event and the uniform concentration event hold simultaneously with probability at least $1 - \delta$. We condition on these two events in the rest of the proof.

We next prove the computable certificate in Eq. (2). By Eq. (6), we have $\sigma(S) \geq \widehat{\sigma}(S) - \lambda\varepsilon B/2$. If $\widehat{\sigma}(S)/B - \lambda\varepsilon/2 < 0$, Eq. (2) is trivial because $\sigma(S) \geq 0$. Otherwise, since $OPT \leq B$, we have

$$\sigma(S) \geq \left(\frac{\widehat{\sigma}(S)}{B} - \frac{\lambda\varepsilon}{2} \right) B \geq \left(\frac{\widehat{\sigma}(S)}{B} - \frac{\lambda\varepsilon}{2} \right) OPT.$$

This proves Eq. (2).

Then, we prove Theorem 5.2. Since Algorithm 3 returns the better one between S^* and S^+ according to $\widehat{\sigma}(\cdot)$ (Line 9), we have $\widehat{\sigma}(S) \geq \widehat{\sigma}(S^+)$. Using Eq. (6) twice, we get

$$\sigma(S) \geq \widehat{\sigma}(S) - \frac{\lambda\varepsilon B}{2} \geq \widehat{\sigma}(S^+) - \frac{\lambda\varepsilon B}{2} \geq \sigma(S^+) - \lambda\varepsilon B. \quad (7)$$

Since $B \leq \sigma^+(S^+)/c$, Eq. (7) implies

$$\sigma(S) \geq \sigma(S^+) - \frac{\lambda\varepsilon \sigma^+(S^+)}{c}. \quad (8)$$

If $(1 - 1/e)\sigma(S^+)/\sigma^+(S^+) - \varepsilon < 0$, the theorem is trivial because $\sigma(S) \geq 0$. Otherwise, since $OPT \leq OPT^+ \leq \sigma^+(S^+)/c$, we have

$$\left(\left(1 - \frac{1}{e} \right) \frac{\sigma(S^+)}{\sigma^+(S^+)} - \varepsilon \right) OPT \leq \frac{\left(1 - \frac{1}{e} \right) \sigma(S^+) - \varepsilon \sigma^+(S^+)}{c}. \quad (9)$$

Moreover,

$$\begin{aligned} c\sigma(S^+) - \lambda\varepsilon\sigma^+(S^+) - \left(\left(1 - \frac{1}{e}\right)\sigma(S^+) - \varepsilon\sigma^+(S^+)\right) \\ = (1 - \lambda)\varepsilon(\sigma^+(S^+) - \sigma(S^+)) \geq 0, \end{aligned} \quad (10)$$

where the last inequality follows from $\sigma(S^+) \leq \sigma^+(S^+)$. Combining Eq. (8), Eq. (9), and Eq. (10), we obtain

$$\sigma(S) \geq \left(\left(1 - \frac{1}{e}\right)\frac{\sigma(S^+)}{\sigma^+(S^+)} - \varepsilon\right) OPT. \quad (11)$$

This proves Theorem 5.2. $\square$

B.5 Proof of Theorem A.1

PROOF. Let S' denote the candidate solution generated from the training RTW samples in Algorithm 4. For $T \in \{S', S^+\}$, let $\tilde{\sigma}(T)$ denote the validation estimate returned by Algorithm 2. Recall that Algorithm 4 sets $\eta = \lambda\varepsilon/2$. Set $c = 1 - 1/e - (1 - \lambda)\varepsilon$, and assume $c > 0$, as required by the *OPIM-C* call.

By the guarantee of *OPIM-C*, with probability at least $1 - \delta/2$, the exposed upper bound B satisfies $OPT \leq OPT^+ \leq B$, and the verification condition implies $B \leq \sigma^+(S^+)/c$. We condition on this event in the following.

Conditioned on the *OPIM-C* call and the training RTW samples, both S' and S^+ are fixed before the validation step. Moreover, the validation samples used by Algorithm 2 are independent of the training samples. Therefore, by Theorem 5.1, each validation call succeeds with probability at least $1 - \delta/4$. Taking a union bound over the two validation calls, with probability at least $1 - \delta/2$, the following inequalities hold simultaneously:

$$|\tilde{\sigma}(S') - \sigma(S')| \leq \eta\sigma(S'). \quad (12)$$

$$|\tilde{\sigma}(S^+) - \sigma(S^+)| \leq \eta\sigma(S^+). \quad (13)$$

Since both S' and S^+ are feasible seed sets, we have $\sigma(S') \leq OPT \leq B$ and $\sigma(S^+) \leq OPT \leq B$. Thus, Eq. (12) and Eq. (13) imply that for each $T \in \{S', S^+\}$, we have

$$|\tilde{\sigma}(T) - \sigma(T)| \leq \eta B. \quad (14)$$

By another union bound, the *OPIM-C* event and the two validation events hold simultaneously with probability at least $1 - \delta$. We condition on these events in the rest of the proof. Since Algorithm 4 returns the solution with the larger validation estimate, we have $\tilde{\sigma}(S) \geq \tilde{\sigma}(S^+)$. Using Eq. (14), we get

$$\sigma(S) \geq \tilde{\sigma}(S) - \eta B \geq \tilde{\sigma}(S^+) - \eta B \geq \sigma(S^+) - 2\eta B = \sigma(S^+) - \lambda\varepsilon B. \quad (15)$$

Since $B \leq \sigma^+(S^+)/c$, Eq. (15) gives

$$\sigma(S) \geq \sigma(S^+) - \frac{\lambda\varepsilon\sigma^+(S^+)}{c}. \quad (16)$$

If $(1 - 1/e)\sigma(S^+)/\sigma^+(S^+) - \varepsilon < 0$, the theorem is trivial because $\sigma(S) \geq 0$. Otherwise, since $OPT \leq OPT^+ \leq \sigma^+(S^+)/c$, we have

$$\left(\left(1 - \frac{1}{e}\right)\frac{\sigma(S^+)}{\sigma^+(S^+)} - \varepsilon\right) OPT \leq \frac{\left(1 - \frac{1}{e}\right)\sigma(S^+) - \varepsilon\sigma^+(S^+)}{c}. \quad (17)$$

Moreover,

$$\begin{aligned} c\sigma(S^+) - \lambda\varepsilon\sigma^+(S^+) - \left(\left(1 - \frac{1}{e}\right)\sigma(S^+) - \varepsilon\sigma^+(S^+)\right) \\ = (1 - \lambda)\varepsilon(\sigma^+(S^+) - \sigma(S^+)) \geq 0, \end{aligned} \quad (18)$$

where the last inequality follows from $\sigma(S^+) \leq \sigma^+(S^+)$. Combining Eq. (16), Eq. (17), and Eq. (18), we obtain

$$\sigma(S) \geq \left(\left(1 - \frac{1}{e}\right)\frac{\sigma(S^+)}{\sigma^+(S^+)} - \varepsilon\right) OPT.$$

This proves Theorem A.1. $\square$

C ADDITIONAL EXPERIMENTAL DETAILS

This section provides additional experimental details omitted from the main text due to space limitations. We first describe the dataset preprocessing procedures and the settings used to construct additional hyperedge variants. We then present supplementary experimental results for HTC-IE and HTC-IM under different parameter settings and model configurations.

C.1 Dataset Preprocessing Details

This subsection provides additional details on how hyperedges are constructed for each dataset. The preprocessing aims to generate hyperedges from shared and discriminative attributes, profiles, or behavioral patterns. We avoid constructing one hyperedge from a single feature, since such a rule may produce overly broad hyperedges with weak group semantics. Instead, most datasets use feature combinations to construct candidate hyperedges.

BlogCatalog and Flickr [24, 29]. *BlogCatalog* and *Flickr* are attributed social network datasets, where nodes represent users, ordinary edges represent social relationships, and node attributes describe user interests or profile information. We use these node attributes as feature tokens for hyperedge construction. We first compute the support of each feature, i.e., the number of nodes containing the feature. Features whose support is outside the range [3, 100] are removed to avoid overly rare or overly common features. For each node, the remaining features are ranked by inverse document frequency (IDF) [43], and the top 30 features are retained. We then enumerate all size-3 feature combinations for each node, which makes the construction more specific than single-feature grouping while keeping enough shared support to form candidate hyperedges. For each feature combination, all nodes containing this combination form a candidate hyperedge. Each candidate hyperedge is further split into multiple hyperedges according to the connected components induced by the ordinary graph. Finally, we keep hyperedges with sizes between 5 and 25 and remove duplicate hyperedges to avoid tiny groups with weak group-level semantics and large groups that may be overly broad or computationally expensive.

CiaoDVD [19]. *CiaoDVD* is a review and rating dataset in which users provide ratings to movies. We construct hyperedges from rating events to capture users with similar positive feedback in the same time period. Specifically, users who give ratings of at least 4 to the same movie within the same month form a candidate hyperedge. We then keep hyperedges with sizes between 5 and 25 and remove duplicate hyperedges.

GitHub [27, 41]. *GitHub* is a developer social network, where nodes represent users and ordinary edges represent mutual follower relationships. Each node is associated with anonymous features.

We use these anonymous node features as feature tokens for hyperedge construction. Features whose support is outside the range [3, 500] are removed to avoid overly rare or overly common features. For each node, the remaining features are ranked by IDF, and the top 15 features are retained. We then enumerate all size-3 feature combinations for each node. For each feature combination, all nodes containing this combination form a candidate hyperedge. We keep hyperedges with sizes between 5 and 25 and remove duplicate hyperedges.

Epinions [45]. *Epinions* is a trust and rating dataset, where users provide ratings to objects and trust links describe user relationships. We use highly rated objects as behavioral features for hyperedge construction. Only ratings with a score of at least 5 are used. Before constructing feature combinations, we remove users who have more than 30 highly rated objects. This step avoids users with overly broad rating histories from inducing generic feature combinations. Features whose support is outside the range [3, 100] are removed to avoid overly rare or overly common features. For each node, the remaining features are ranked by IDF, and the top 30 features are retained. We then enumerate all size-3 feature combinations for each node. For each feature combination, all nodes containing this combination form a candidate hyperedge. Each candidate hyperedge is further split into multiple hyperedges according to the connected components induced by the ordinary trust graph. Finally, we keep hyperedges with sizes between 5 and 25 and remove duplicate hyperedges.

Pokec [27]. *Pokec* is a large-scale social network dataset, where nodes represent users and ordinary edges represent friendship relations. Each user is associated with structured profile information. We extract feature tokens from profile fields, such as region, age group, gender, education, and interests. Features whose support is outside the range [5, 2000] are removed to avoid overly rare or overly common features. For each node, the remaining features are ranked by IDF, and the top 20 features are retained. We then enumerate all size-3 feature combinations for each node. For each feature combination, all nodes containing this combination form a candidate hyperedge. Finally, we keep hyperedges with sizes between 5 and 25, remove duplicate hyperedges, and limit each node to at most three incident hyperedges.

Yelp [53]. *Yelp* is a review dataset, where users provide ratings and reviews to businesses and friendship links describe user relationships. We use highly rated businesses as behavioral features for hyperedge construction. Only reviews with at least 4.0 stars are used. Before constructing feature combinations, we remove users who have more than 100 highly rated businesses. This step avoids users with overly broad review histories from inducing generic feature combinations. Features whose support is outside the range [5, 2000] are removed to avoid overly rare or overly common features. For each node, the remaining features are ranked by IDF, and the top 20 features are retained. We then enumerate all size-3 feature combinations for each node. For each feature combination, all nodes containing this combination form a candidate hyperedge. We do not split candidate hyperedges according to connected components of the friendship graph for *Yelp*. Finally, we keep hyperedges with sizes between 5 and 25 and remove duplicate hyperedges.

Table 2: Statistics of hyperedge variants.

Dataset	q			Avg. $ h $		
	Base	High	XHigh	Base	High	XHigh
<i>BlogCatalog</i>	41	73	119	6.78	7.97	7.69
<i>Flickr</i>	107	138	154	7.01	7.05	7.24
<i>CiaoDVD</i>	691	892	984	7.99	8.03	8.13
<i>GitHub</i>	657	1,188	2,225	7.43	7.19	7.36
<i>Epinions</i>	396	1,014	2,394	9.23	8.59	7.88
<i>Pokec</i>	868	1,230	1,760	6.45	6.74	7.06
<i>Yelp</i>	855	2,339	5,657	6.00	6.15	6.31

C.2 Settings for Varying the Number of Hyperedges q

This subsection provides the detailed settings used to generate the *High* and *XHigh* variants in the experiments on the effect of the number of hyperedges q . The *Base* variant is the default hyperedge construction described in the previous subsection. For each dataset, the ordinary directed graph is kept fixed. Only the hyperedges are regenerated, and the corresponding thresholds are regenerated from the resulting hyperedges. The *High* and *XHigh* variants are obtained by progressively relaxing the construction settings of *Base*, with *XHigh* applying a stronger relaxation than *High*. The settings listed below describe the changes from *Base* to *High* and *XHigh*. All settings not mentioned below are kept the same as in *Base*. Table 2 reports the resulting statistics of the three variants, including the number of hyperedges q and the average hyperedge size.

BlogCatalog and *Flickr*. For *High*, we retain the top 40 features for each node and expand the feature support range from [3, 100] to [3, 200]. For *XHigh*, we retain the top 50 features for each node and further expand the feature support range to [3, 300].

CiaoDVD. For *High*, the rating threshold for forming movie month rating groups is relaxed from 4 to 3. For *XHigh*, the rating threshold is further relaxed to 2.

GitHub. For *High*, we retain the top 18 features for each node and expand the feature support range from [3, 500] to [3, 700]. For *XHigh*, we retain the top 20 features for each node and further expand the feature support range to [3, 1000].

Epinions. For *High*, the maximum number of highly rated objects per user is increased from 30 to 50, the number of retained objects per user is increased from 30 to 40, and the object support range is expanded from [3, 100] to [3, 200]. For *XHigh*, the maximum number of highly rated objects per user is increased to 80, the number of retained objects per user is increased to 50, and the object support range is further expanded to [3, 500].

Pokec. For *High*, we retain the top 30 features for each node, expand the feature support range from [5, 2000] to [5, 3000], and increase the maximum number of hyperedges per node from 3 to 5. For *XHigh*, we retain the top 40 features for each node, further expand the feature support range to [5, 5000], and increase the maximum number of hyperedges per node to 8.

Yelp. For *High*, the number of retained businesses per user is increased from 20 to 25, and the business support range is expanded

from [5, 2000] to [5, 3000]. For *XHigh*, the number of retained businesses per user is increased to 30, the business support range is further expanded to [5, 4000], and the maximum number of highly rated businesses per user is increased from 100 to 120.

C.3 Additional Results for HTC-IE

This subsection provides supplementary results for HTC-IE that are omitted from the main text due to space limitations. We report additional results on the effect of input seed sets, threshold distributions, estimation parameters, and the number of hyperedges. The results include estimated influence values, runtime, and the number of generated samples, which together provide a more complete evaluation of the accuracy and efficiency of *RTW-Est*.

C.3.1 Additional Results on the Effect of Input Seed Sets. This subsection provides additional results for the experiment in Section 6.2.1. Figure 8 reports the number of generated samples for the structurally ranked seed sets used in the main experiment. Figures 9, 10, and 11 report the estimated influence value, runtime, and number of generated samples when the input seed sets are randomly selected.

As shown in Figure 8, the number of generated samples follows a trend similar to runtime as the seed set size k varies. For *RTW-Est*, the number of samples generally decreases as k increases, because a larger seed set is more likely to satisfy a sampled witness. Compared with the fixed sample baselines, *RTW-Est* adaptively reduces the number of generated samples. Its sample size is also within the same order of magnitude as *UR-IE*, which uses fewer samples but estimates an upper-bound surrogate rather than the original HTC influence objective.

We also evaluate the methods under random seed selection. As shown in Figure 9, the estimated influence values are still consistent across all methods except *UR-IE*. This is consistent with the results in the main experiment, since *UR-IE* estimates an upper-bound surrogate and can therefore give larger estimates than the other methods. Figure 10 shows that the runtime still generally decreases as k increases. On *Pokec* and *Yelp*, *RTW-Est* can be slower than the fixed sample baselines under random seed selection. This is because randomly selected seeds tend to have lower influence, making a sampled witness less likely to be satisfied by the input seed set. On the other datasets, *RTW-Est* remains faster than the baselines.

Figure 11 further shows the sample-count trend under random seed selection, which is consistent with the runtime trend in Figure 10. The number of generated samples decreases as k increases, and the sample trend is largely consistent with the runtime results. These results show that the efficiency of *RTW-Est* mainly comes from adaptive sample generation and early termination when sampled witnesses are satisfied.

C.3.2 Additional Results on the Effect of Estimation Parameters ϵ and δ . This subsection provides additional results for the experiment in Section 6.2.2. Figures 12, 13, and 14 report the estimated influence value, runtime, and number of generated samples under different values of ϵ , respectively. Figures 15, 16, and 17 report the corresponding results under different values of δ .

We first discuss the effect of ϵ . As shown in Figure 12, the estimated influence values do not change significantly as ϵ varies.

This is expected, since changing ϵ mainly controls the required estimation accuracy rather than the underlying influence objective. Across methods, *UR-IE* generally gives larger estimates than the other methods, because it estimates an upper-bound surrogate of the influence value. The other methods produce similar estimates, which confirms that they are estimating the same HTC influence objective.

Figure 13 shows that the runtime of *RTW-Est* decreases as ϵ increases. This trend is consistent with the concentration bound, since a larger error tolerance requires fewer samples. The reduction is more pronounced on large datasets. For example, on *Pokec*, increasing ϵ from 0.1 to 0.4 reduces the runtime of *RTW-Est* by about one order of magnitude.

Figure 14 further confirms this trend. The number of generated samples decreases as ϵ increases, and its trend is largely consistent with the runtime results. This indicates that the runtime improvement mainly comes from the reduction in sample complexity.

We next discuss the effect of δ . As shown in Figure 15, the estimated influence values remain largely stable as δ varies. This is expected, since changing δ affects the confidence requirement of the estimator rather than the underlying influence objective. The comparison across methods is consistent with the main results. In particular, *UR-IE* generally gives larger estimates than the other methods because it estimates an upper-bound surrogate of the influence value.

Figure 16 shows that the runtime of *RTW-Est* slightly decreases as δ increases. This is because a larger δ relaxes the confidence requirement and reduces the required number of samples. However, the reduction is relatively mild compared with the effect of ϵ , since the dependence on δ is logarithmic in the concentration bound. The relative comparison across methods remains similar to the main experiment.

Figure 17 further confirms this observation. The number of generated samples decreases mildly as δ increases, and its trend is consistent with the runtime results. These results show that *RTW-Est* behaves stably under different estimation requirements.

C.3.3 Additional Results on the Effect of Hyperedge Threshold Distribution. This subsection provides additional results for the experiment in Section 6.2.3. Figures 18, 19, and 20 report the estimated influence value, runtime, and number of generated samples under different threshold distributions, respectively.

As shown in Figure 18, changing the threshold distribution does not significantly affect the estimated influence values. The comparison across methods is consistent with the main experiment. In particular, *UR-IE* usually gives larger estimates than the other methods, because it estimates an upper-bound surrogate of the influence value. The other methods produce similar estimates, which confirms that they are estimating the same HTC influence objective.

Figure 19 shows that *RTW-Est* remains the fastest method among the methods that accurately estimate the objective, or achieves runtime comparable to *RTW-Fixed-10K*. The main exception is *Pokec*, where *RTW-Fixed-10K* is faster than *RTW-Est*. Across the three threshold distributions, the runtime does not show a clear difference, indicating that the proposed estimator is not sensitive to the tested threshold sampling schemes.

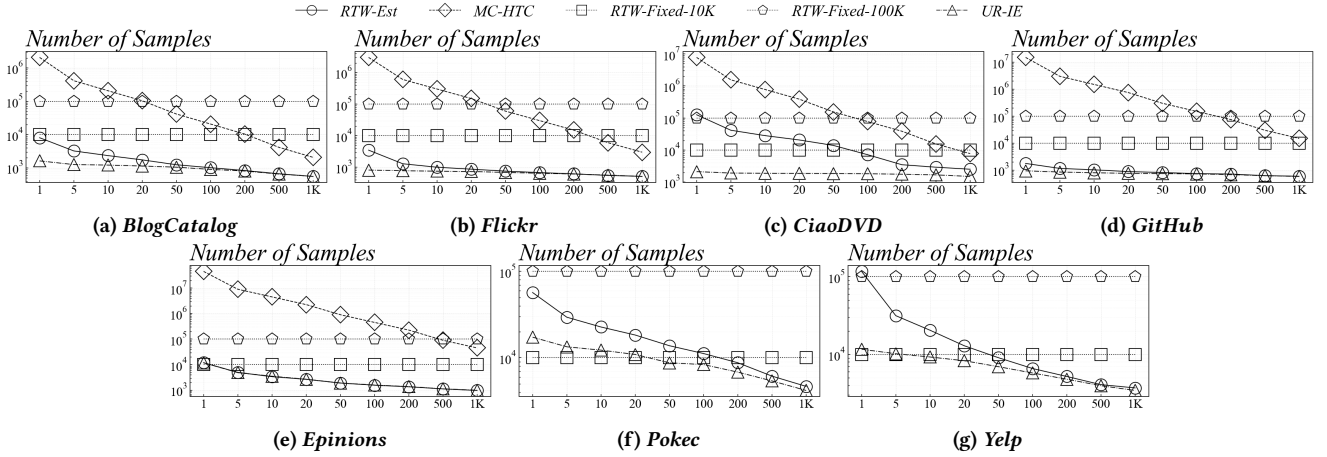


Figure 8: Number of generated samples vs. seed set size k for different methods on each dataset.

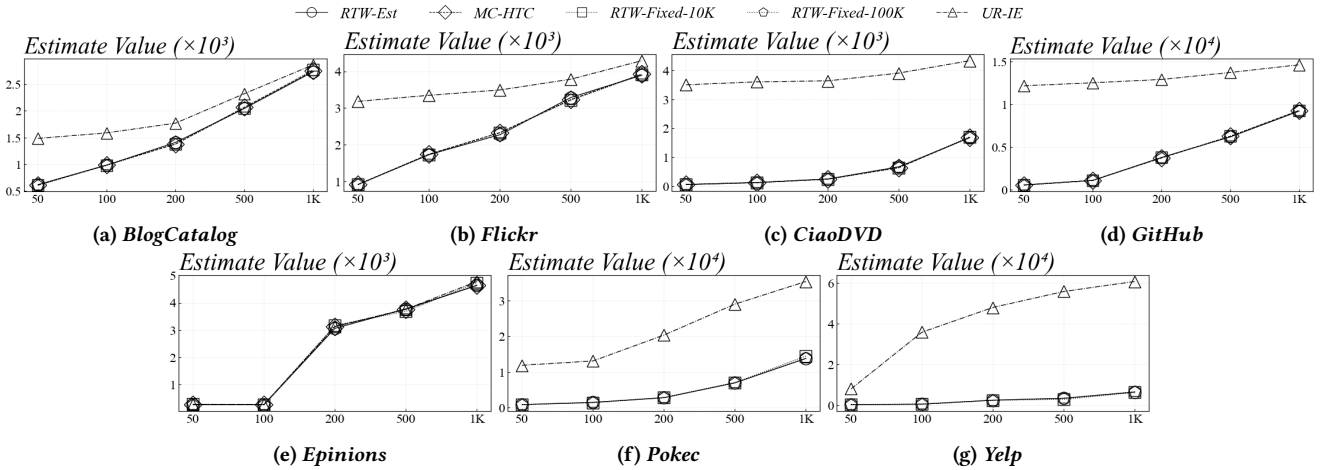


Figure 9: Estimated influence value under random seed selection vs. seed set size k for different methods on each dataset.

Figure 20 further shows that the number of generated samples follows a trend similar to runtime. The sample comparison across methods is consistent with the runtime comparison, and changing the threshold distribution does not lead to a significant change in the number of generated samples. These results are consistent with the main experiment and show that *RTW-Est* remains stable under different hyperedge activation requirements.

C.3.4 Additional Results on the Effect of the Number of Hyperedges q . This subsection provides additional results for the experiment in Section 6.2.4. Figures 21, 22, and 23 report the estimated influence value, runtime, and number of generated samples under different numbers of hyperedges q , respectively.

As shown in Figure 21, the estimated influence value generally increases as q increases. This is expected, since adding more hyperedges provides more opportunities for threshold-based activation and can increase the final influence spread. Across methods, the comparison is mostly consistent with the main experiment, where *UR-IE* usually gives larger estimates than the other methods. On

Epinions, the estimated values of all methods are close to each other. In the *High* setting, *UR-IE* gives a slightly smaller estimate than the other methods, but this difference is within the allowed estimation error. This is because the influence value on *Epinions* is already close to the upper-bound estimate produced by *UR-IE*, which is also observed in the main experiment.

Figure 22 shows that the runtime does not change significantly with q on most datasets. The main exception is *Pokec*, where increasing the number of hyperedges leads to higher runtime. This indicates that denser hyperedge structures can increase the cost of sample processing on large datasets. Nevertheless, within the tested range of q , the runtime of *RTW-Est* remains relatively stable on most datasets.

Figure 23 shows a similar pattern for the number of generated samples. For most datasets, the number of samples does not change significantly as q increases. For *Yelp*, the number of samples generated by *RTW-Est* decreases as q increases. A similar decrease also appears on *CiaoDVD* under the *XHigh* setting. Except for these

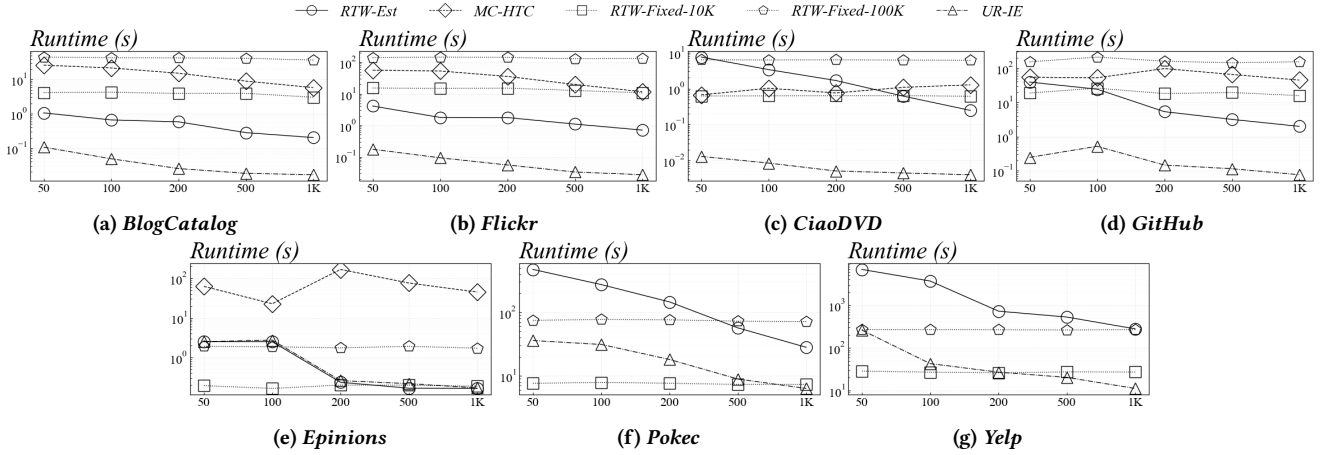


Figure 10: Runtime under random seed selection vs. seed set size k for different methods on each dataset.

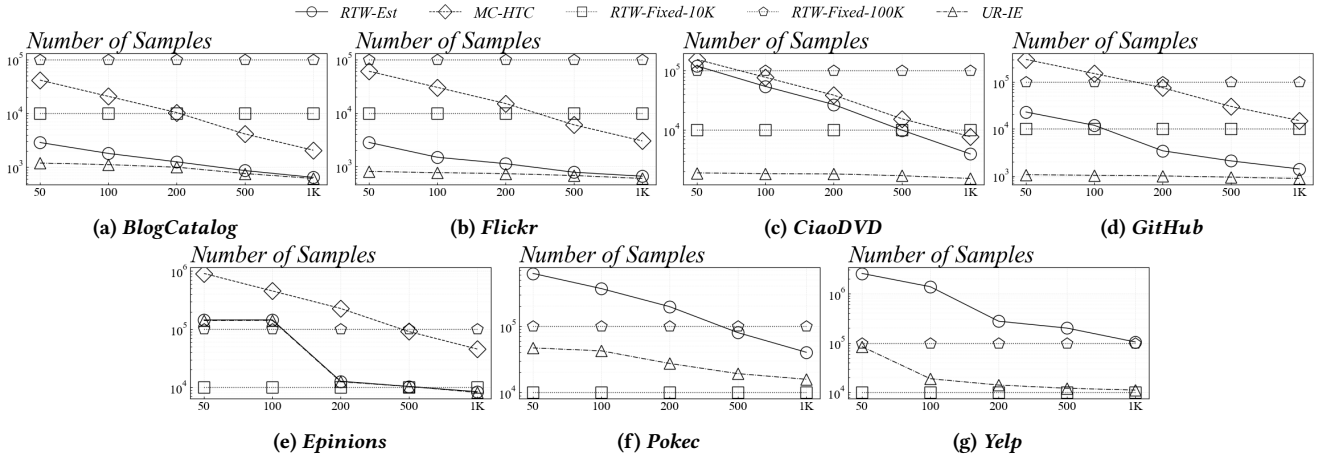


Figure 11: Number of generated samples under random seed selection vs. seed set size k for different methods on each dataset.

cases, the number of generated samples remains stable across the three variants. Overall, these results show that, within the tested range, increasing the number of hyperedges does not substantially increase the runtime or sample size of *RTW-Est*.

C.4 Additional Results for HTC-IM

This subsection provides supplementary HTC-IM results omitted from the main text, including the effects of seed budget, threshold distributions, estimation parameters, allocation parameter, sample budget, and the number of hyperedges, as well as the ablation study. We report objective values, runtime, memory usage, and the solution-dependent lower-bound ratio α to further evaluate the effectiveness, scalability, and practical guarantee of *RTW-Greedy* and *RTW-Greedy-P*.

C.4.1 Additional Results on the Effect of Seed Budget k . This subsection provides additional results for the experiments in Section 6.3.1. Figures 24 and 25 report the memory usage and the solution-dependent lower-bound ratio α on the remaining datasets, respectively.

As shown in Figure 24, memory usage generally increases with the seed budget k . This trend is expected because larger seed budgets require the algorithms to maintain more selected nodes and related sample information. Among the proposed methods, *RTW-Greedy* uses more memory because it stores more RTW samples for the theoretically guaranteed selection procedure. In contrast, *RTW-Greedy-P* uses much less memory and remains more scalable on large datasets.

Figure 25 reports the corresponding results for α . Similar to the trend observed on *CiaoDVD*, α generally increases as k increases. When k is small, the selected seed set may satisfy only a limited number of RTW samples, resulting in a small empirical influence relative to the upper bound B . As k grows, the seed set generally satisfies more RTW samples, yielding a larger and more stable lower-bound ratio. These results show that the solution-dependent lower-bound ratio used in our theoretical guarantee is practically meaningful across datasets.

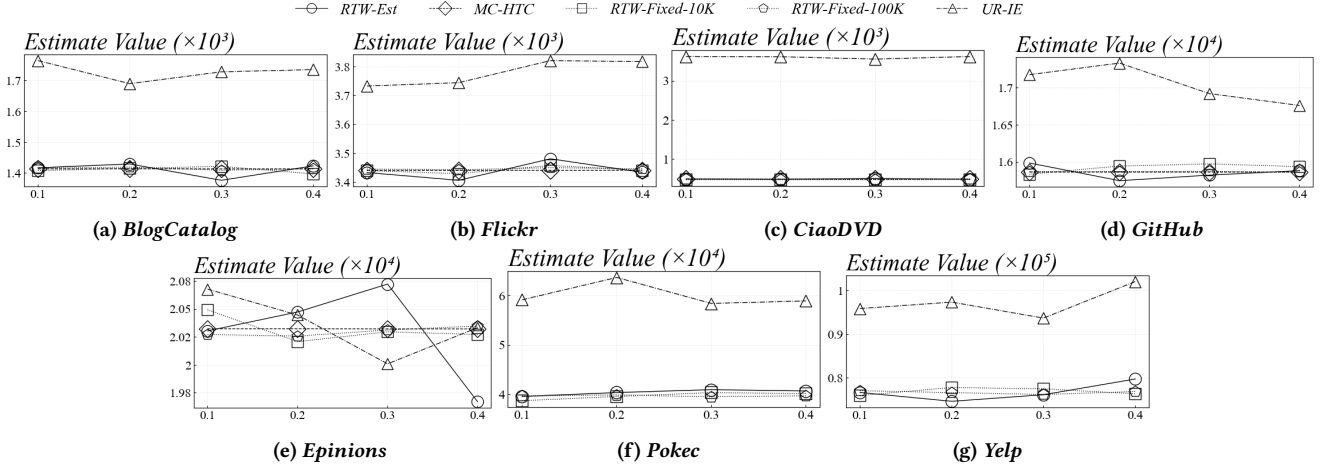


Figure 12: Estimated influence value vs. error parameter ϵ for different methods on each dataset.

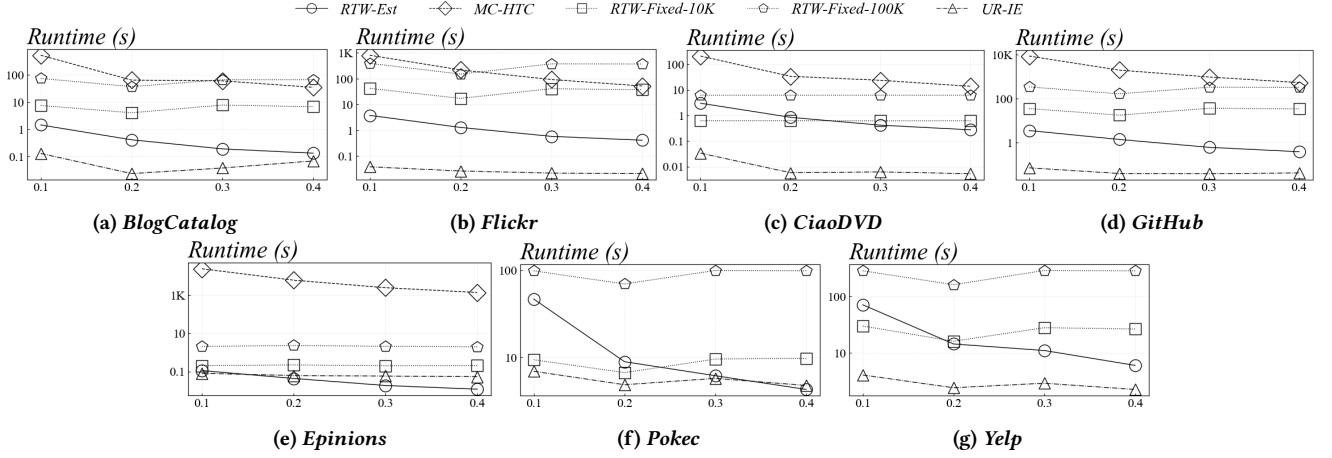


Figure 13: Runtime vs. error parameter ϵ for different methods on each dataset.

C.4.2 Additional Results on the Effect of Estimation Parameters ϵ and δ . This subsection provides additional results for the experiment in Section 6.3.2. Figures 26, 27, 28, and 29 report the objective value, runtime, memory usage, and solution-dependent lower-bound ratio α under different values of ϵ , respectively. Figures 30, 31, 32, and 33 report the corresponding results under different values of δ .

As shown in Figure 26, the objective values of *RTW-Greedy* and *RTW-Greedy-P* are largely stable as ϵ varies. This shows that relaxing the error parameter does not noticeably hurt the solution quality of the proposed methods in these experiments. In contrast, some baselines, such as *Edge-Only*, show decreasing objective values when ϵ becomes larger. Overall, *RTW-Greedy* and *RTW-Greedy-P* maintain strong objective values across different error requirements.

Figure 27 shows that increasing ϵ reduces the runtime of *RTW-Greedy*. This is expected because a larger ϵ relaxes the approximation requirement and reduces the required number of RTW samples.

For *RTW-Greedy-P*, the runtime changes only mildly as ϵ varies, indicating that the practical variant is relatively stable under different error parameters.

Figure 28 shows a trend similar to runtime. The memory usage of *RTW-Greedy* decreases as ϵ increases, since fewer RTW samples need to be stored. The memory usage of *RTW-Greedy-P* is much lower and remains relatively stable across different values of ϵ .

Figure 29 reports the solution-dependent lower-bound ratio α . As ϵ increases, α generally decreases because the theoretical requirement is relaxed. Nevertheless, α remains at a nontrivial constant level in our experiments, showing that the solution-dependent guarantee remains meaningful in practice.

As shown in Figure 30, the objective values of all methods are not sensitive to δ . Across different confidence requirements, *RTW-Greedy* remains the most stable method and generally achieves the best objective values. This confirms that changing δ mainly affects the confidence level in the sampling procedure, rather than the quality of the selected seed sets.

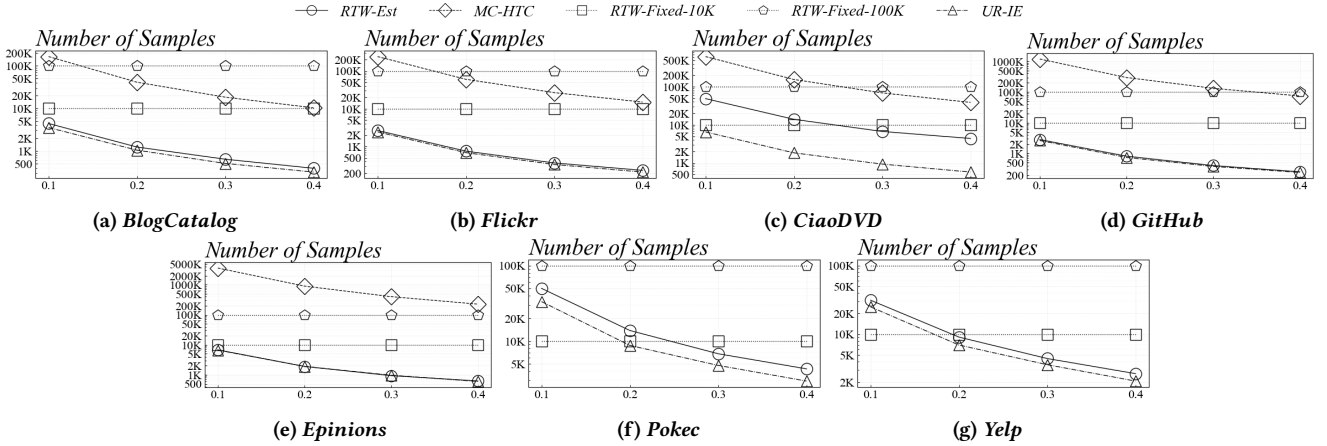


Figure 14: Number of generated samples vs. error parameter ε for different methods on each dataset.

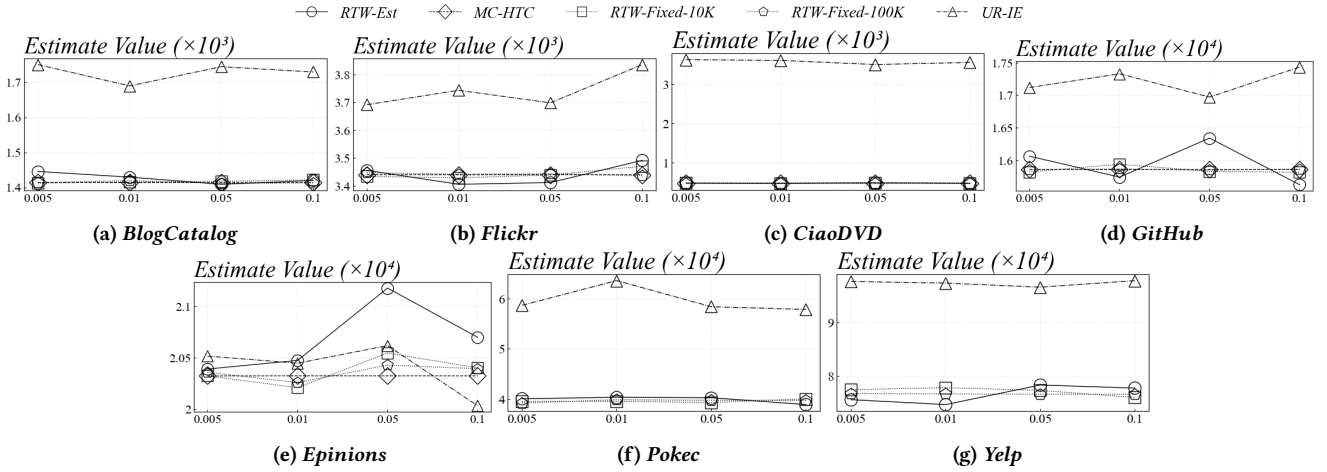


Figure 15: Estimated influence value vs. failure probability δ for different methods on each dataset.

Figure 31 shows that the runtime comparison is consistent with the main experiment. On *BlogCatalog*, *Flickr*, *GitHub*, and *Epinions*, *RTW-Greedy* is about one order of magnitude slower than *RTW-Greedy-P*. On *CiaoDVD*, the gap becomes about two orders of magnitude. The heuristic baselines, such as *Edge-Only*, *HTC-CE*, and *UR-IM*, are faster than *RTW-Greedy-P*, which is expected because they rely on simpler RR-set-based structures rather than RTW samples. However, their faster runtime comes at the cost of less stable objective values across datasets.

Figure 32 reports the memory usage. *RTW-Greedy* requires substantially more memory and may run out of memory on large datasets. In contrast, the other methods, including *RTW-Greedy-P*, finish within a reasonable memory range, typically below 10^4 MB. This again shows that *RTW-Greedy-P* significantly improves the scalability of the theoretically motivated RTW-based approach.

Figure 33 reports the solution-dependent lower-bound ratio α . Across different values of δ , α remains at a nontrivial constant level. This suggests that the solution-dependent approximation guarantee

remains meaningful in practice under different confidence requirements.

C.4.3 Additional Results on the Effect of Hyperedge Threshold Distribution. This subsection provides additional results for the experiment in Section 6.3.3. Figures 34, 35, 36, and 37 report the objective value, runtime, memory usage, and solution-dependent lower-bound ratio α under different threshold distributions, respectively. Table 3 reports the α values of *RTW-Greedy-P* on *Pokec* and *Yelp*.

As shown in Figure 34, *RTW-Greedy* achieves the highest objective values when it is feasible, and *RTW-Greedy-P* usually achieves the second-best or comparable objective values. The heuristic baselines are less stable across datasets. For example, *UR-IM*, *Edge-Only*, and *HTC-CE* obtain objective values close to *RTW-Greedy-P* on *Epinions*, but their objective values are much lower on *CiaoDVD*. These results show that *RTW-Greedy* and *RTW-Greedy-P* are more robust for *HTC-IM* across different threshold distributions.

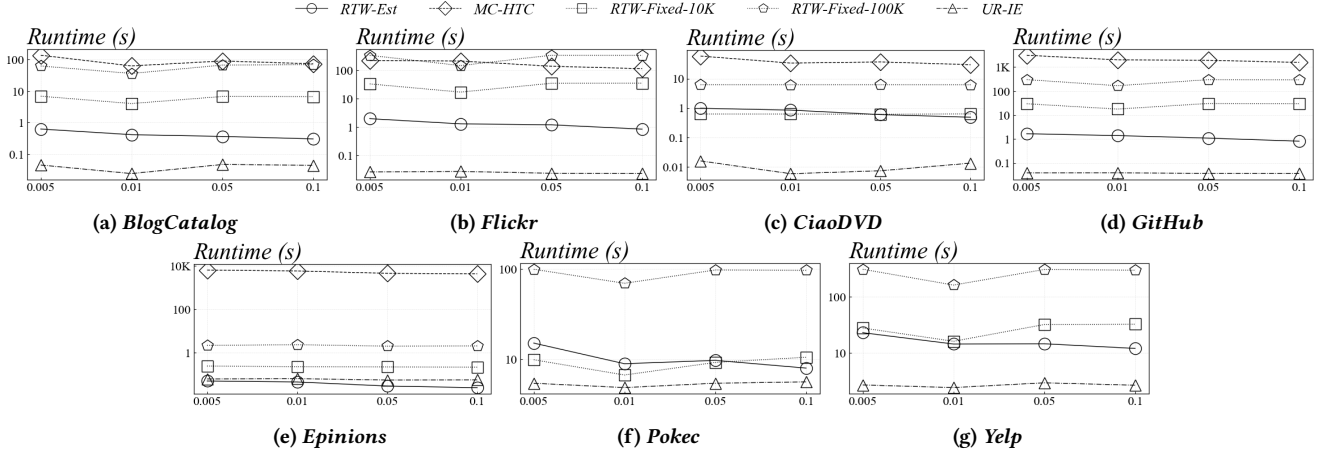


Figure 16: Runtime vs. failure probability δ for different methods on each dataset.

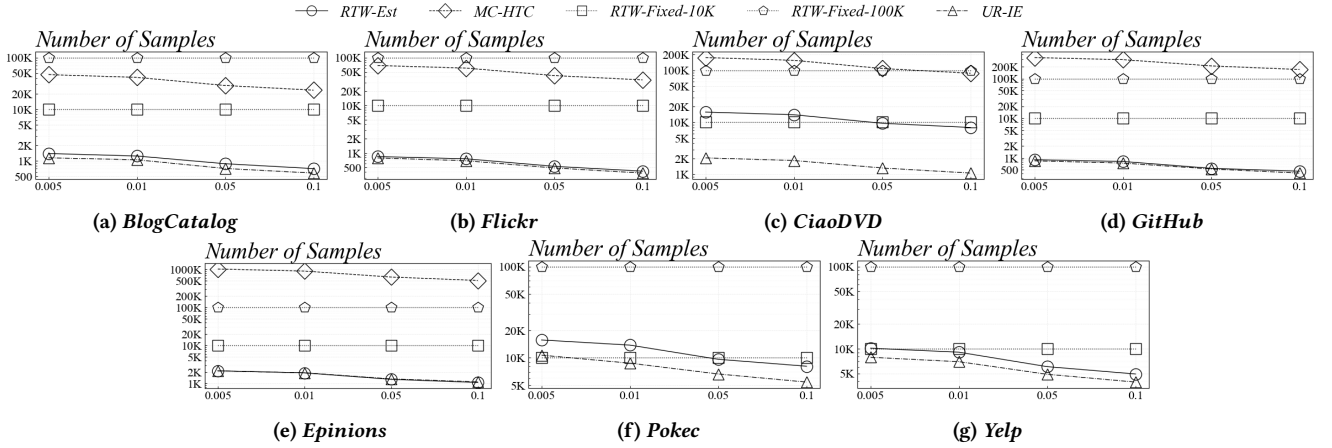


Figure 17: Number of generated samples vs. failure probability δ for different methods on each dataset.

The threshold distribution can also have a clear effect on the objective value. On *CiaoDVD*, the objective value under the *Uniform* setting is roughly twice as large as those under the *Exponential* and *Mixed* settings. This indicates that the activation difficulty of hyperedges can substantially affect the achievable influence spread.

Figure 35 shows that the runtime comparison is consistent with the main experiment. *RTW-Greedy* is the slowest method, while *RTW-Greedy-P* is much more scalable and remains practical even on large datasets such as *Pokec* and *Yelp*. The heuristic baselines are faster, but they do not consistently achieve high objective values. For the same method, changing the threshold distribution does not lead to a clear runtime difference.

Figure 36 shows a similar pattern for memory usage. The comparison across methods is consistent with the main experiment, and the memory usage of each method is largely stable under different threshold distributions. This is expected, since changing the threshold distribution affects activation conditions but does not substantially change the sampled data structures maintained by each method.

Table 3: Solution-dependent lower-bound ratio α of *RTW-Greedy-P* under different threshold distributions on *Pokec* and *Yelp*.

Dataset	Uniform	Exponential	Mixed
<i>Pokec</i>	0.505	0.494	0.490
<i>Yelp</i>	0.579	0.512	0.527

Finally, Figure 37 and Table 3 report the solution-dependent lower-bound ratio α . Overall, α remains within a reasonable constant range. The main exception is *CiaoDVD*, where α is below 0.1 under the *Exponential* and *Mixed* settings. This value is small, but it is still a positive constant, and α is a certified lower bound on the approximation ratio, rather than the realized approximation ratio itself. On the large datasets *Pokec* and *Yelp*, α is stable across the three threshold distributions and stays around 0.5, which is practically meaningful. These results further support the effectiveness of the solution-dependent guarantee in practice.

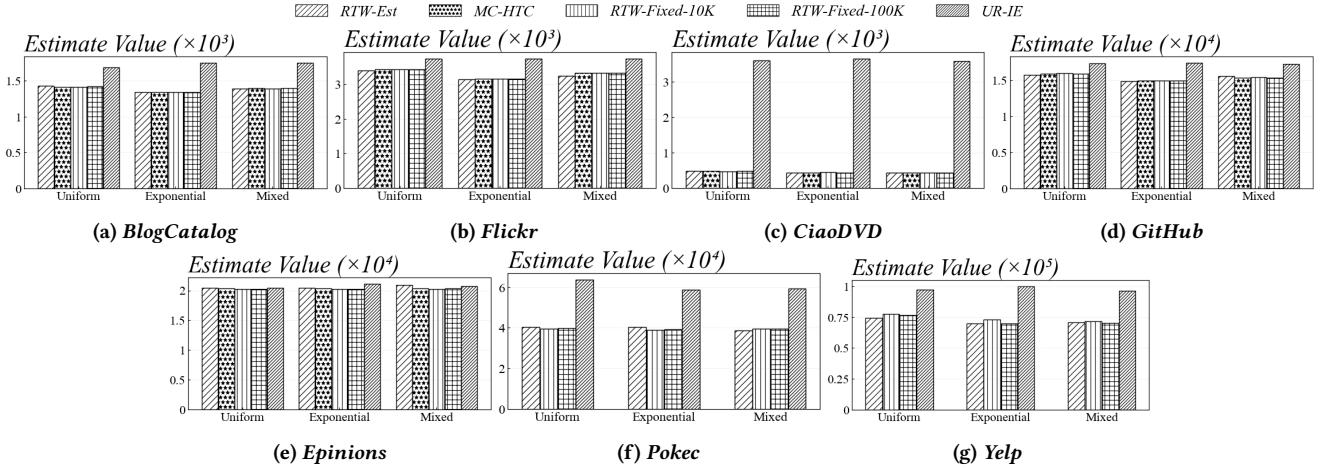


Figure 18: Estimated influence value vs. threshold distribution for different methods on each dataset.

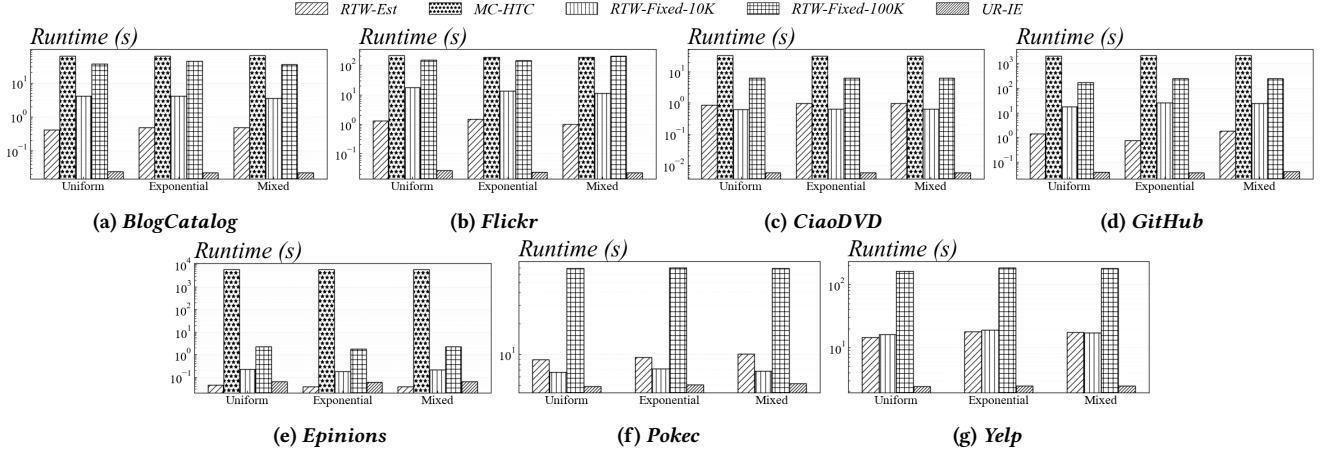


Figure 19: Runtime vs. threshold distribution for different methods on each dataset.

C.4.4 Additional Results on the Effect of the Number of Hyperedges q . This subsection provides additional results for the experiment in Section 6.3.4. Figures 38, 39, 40, and 41 report the objective value, runtime, memory usage, and solution-dependent lower-bound ratio α under different numbers of hyperedges q , respectively.

As shown in Figure 38, the comparison across methods is consistent with the main experiment. *RTW-Greedy* and *RTW-Greedy-P* remain the most effective methods for HTC-IM. Moreover, their objective values generally increase as q increases. This is expected, since adding more hyperedges introduces more opportunities for threshold activation and can increase the achievable influence spread.

Figure 39 reports the runtime results. The relative comparison across methods remains similar under different values of q . For the same method, increasing q does not lead to a large runtime increase on most datasets. On larger datasets, including *GitHub*, *Epinions*, *Pokec*, and *Yelp*, the runtime of *RTW-Greedy* and *RTW-Greedy-P* increases mildly as q increases. However, the runtime does not

increase by an order of magnitude, indicating that the proposed methods remain stable within the tested range of q .

Figure 40 shows the memory usage. For *RTW-Greedy-P*, increasing q leads to higher memory usage on *GitHub*, *Pokec*, and *Yelp*. This is reasonable because denser hyperedge structures can increase the amount of information maintained during RTW-based optimization. On the other datasets, the increase is less pronounced. Overall, the memory usage of *RTW-Greedy-P* remains within an acceptable range.

Finally, Figure 41 reports the solution-dependent lower-bound ratio α . Both *RTW-Greedy* and *RTW-Greedy-P* maintain stable α values across the three variants. The values remain reasonable constants, showing that the solution-dependent approximation guarantee is still meaningful as the number of hyperedges increases.

C.4.5 Additional Results on the Effect of Allocation Parameter λ . This subsection provides additional results for the experiment in Section 6.3.5. Figures 42, 43, 44, and 45 report the objective value,

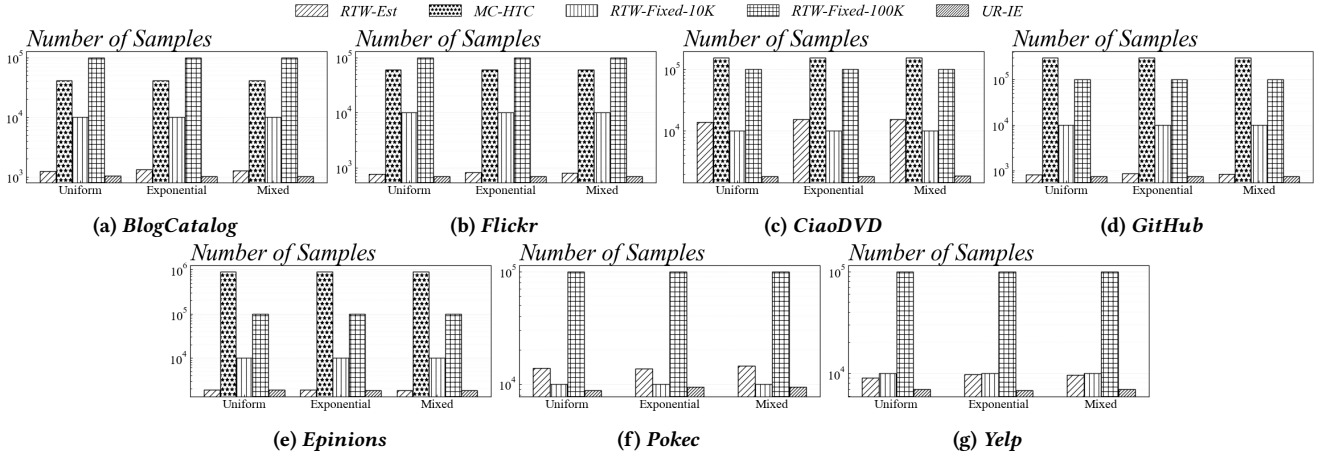


Figure 20: Number of generated samples vs. threshold distribution for different methods on each dataset.

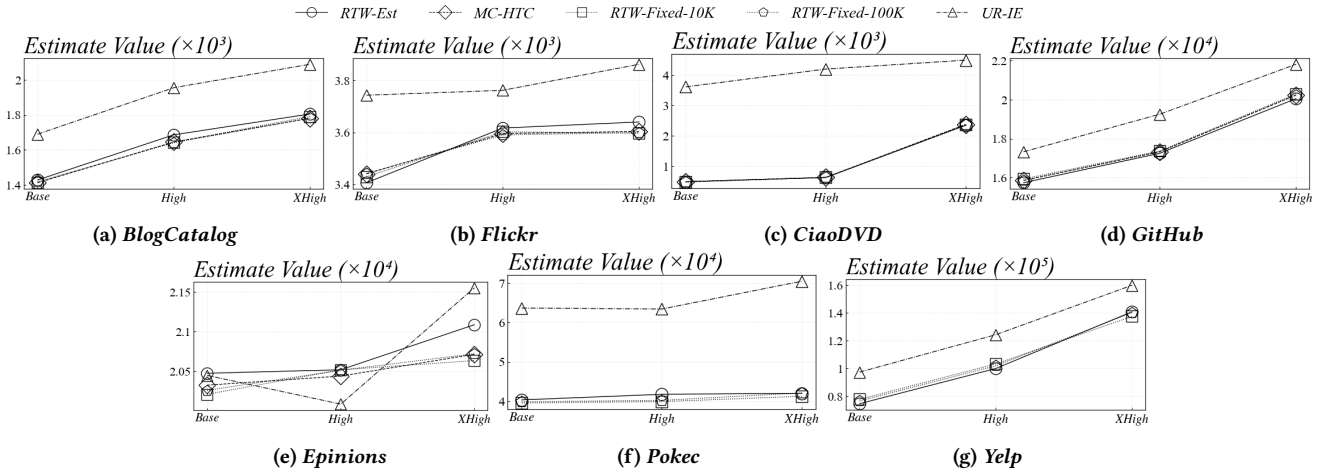


Figure 21: Estimated influence value vs. number of hyperedges q for different methods on each dataset.

runtime, memory usage, and solution-dependent lower-bound ratio α under different values of λ , respectively. This experiment is conducted on the five smaller datasets, since *RTW-Greedy* has high runtime and memory costs on large datasets such as *Pokec* and *Yelp*, as discussed in the main text.

As shown in Figure 42, the objective values are relatively stable within the tested range of λ . Across all five datasets, *RTW-Greedy* consistently achieves slightly higher objective values than *RTW-Greedy-P*. This is expected, since *RTW-Greedy* uses a more expensive sample-based procedure, while *RTW-Greedy-P* is designed as a more scalable practical variant. The stability of the objective values indicates that the solution quality of both methods is not very sensitive to λ .

Figure 43 reports the runtime results. Overall, the runtime of *RTW-Greedy* decreases as λ increases. In contrast, the runtime of *RTW-Greedy-P* remains largely unchanged. This suggests that λ mainly affects the computational cost of *RTW-Greedy*, while the practical variant is much less sensitive to this parameter.

Figure 44 shows a similar trend for memory usage. The memory usage of *RTW-Greedy* clearly decreases as λ increases, whereas the memory usage of *RTW-Greedy-P* remains stable. This is consistent with the runtime results and further shows that increasing λ reduces the resource cost of *RTW-Greedy*.

Finally, Figure 45 reports the solution-dependent lower-bound ratio α . In general, *RTW-Greedy* achieves larger α values than *RTW-Greedy-P*. As λ increases, α tends to decrease, although the trend is not strictly monotonic. This is reasonable because α is a solution-dependent quantity and can vary with the selected seed set. Overall, α remains within a meaningful range, showing that the solution-dependent guarantee is still practically useful under different allocation settings.

C.4.6 Additional Results on the Effect of Sample Budget θ_0 . This subsection provides additional results for the experiment in Section 6.3.6. Table 4 reports the objective value, runtime, memory usage, and solution-dependent lower-bound ratio α of *RTW-Greedy-P* under different sample budgets θ_0 .

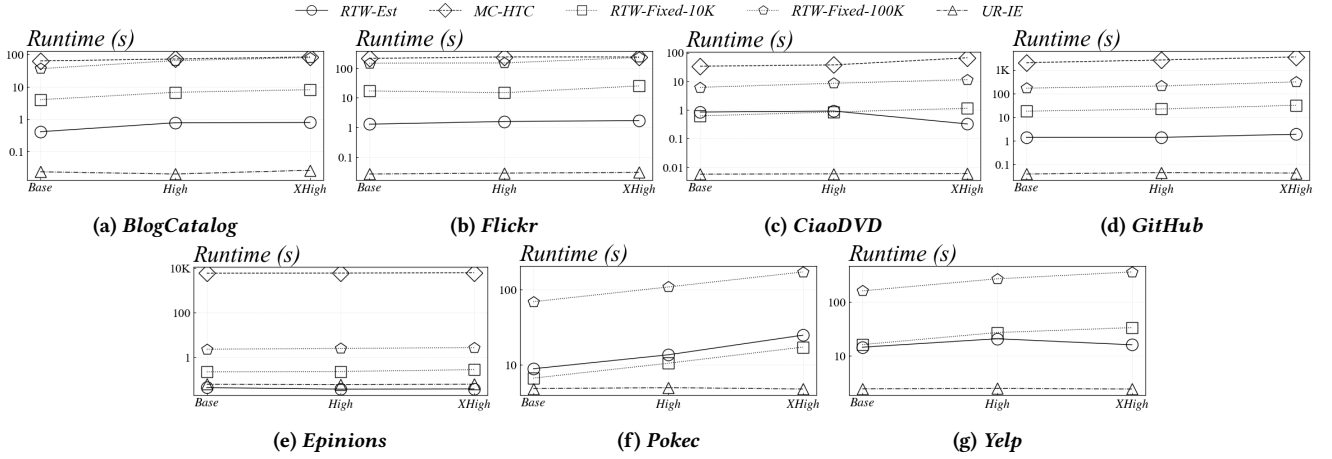


Figure 22: Runtime vs. number of hyperedges q for different methods on each dataset.

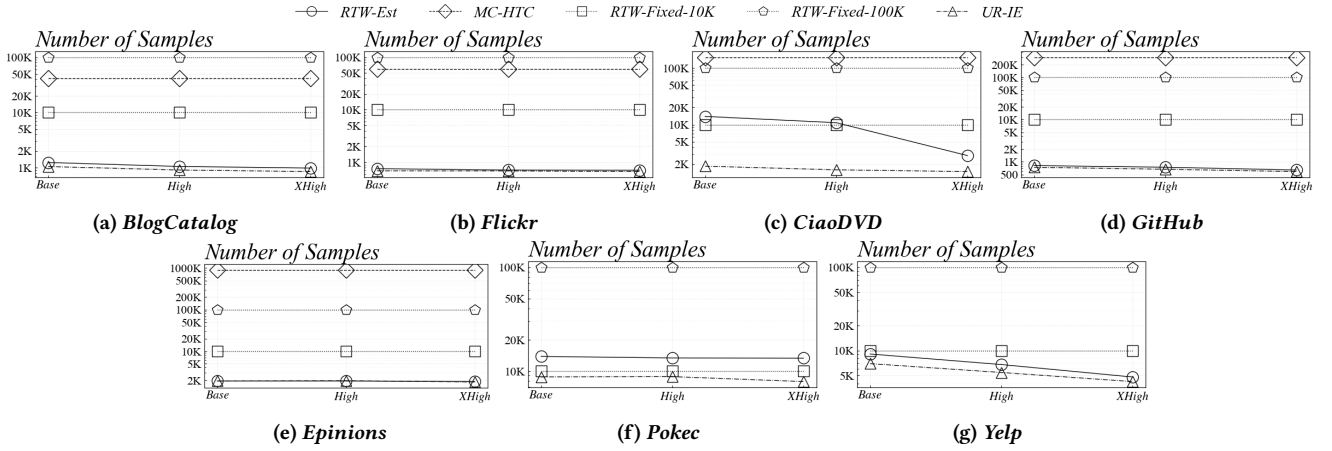


Figure 23: Number of generated samples vs. number of hyperedges q for different methods on each dataset.

Overall, increasing θ_0 tends to improve the objective value. This is expected, since a larger sample budget provides more RTW samples and allows *RTW-Greedy-P* to make more accurate greedy decisions. The solution-dependent lower-bound ratio α also generally increases with θ_0 , indicating that a larger sample budget strengthens the empirical lower-bound ratio.

On the five smaller datasets, $\theta_0 = 10K$ already provides a good trade-off between solution quality and efficiency. Compared with $\theta_0 = 1K$, it improves the objective value and α on most datasets. Compared with $\theta_0 = 100K$, it achieves similar objective values while using much less time and memory. For example, on *Flickr*, the objective value increases from 3,491 under 10K to only 3,516 under 100K, while the runtime increases from 182.64 seconds to 1,230.77 seconds. A similar trade-off can also be observed on *GitHub* and *Epinions*.

On the two largest datasets, *Pokec* and *Yelp*, using $\theta_0 = 100K$ brings a more visible improvement. On *Pokec*, the objective value increases from 36,111 under 10K to 39,940 under 100K. On *Yelp*, it

increases from 74,620 to 79,517. This suggests that larger graphs benefit more from a larger sample budget.

The runtime and memory usage generally increase with θ_0 , because more RTW samples need to be generated and stored. This cost is especially clear when increasing θ_0 from 10K to 100K. For instance, on *GitHub*, the runtime increases from 393.35 seconds to 2,810.82 seconds, and the memory usage increases from 1,473.75 MB to 13,689.09 MB.

These results justify our scale-dependent default setting for θ_0 . For the five smaller datasets, $\theta_0 = 10K$ provides a good balance between objective value, runtime, memory usage, and α . For million-scale datasets such as *Pokec* and *Yelp*, $\theta_0 = 100K$ is a more suitable choice when higher solution quality is desired. In practice, these results suggest that 10K is sufficient for the smaller graphs tested here, while 100K is more suitable for the two million-scale graphs when higher solution quality is desired.

C.4.7 Additional Results of the Ablation Study. This subsection provides additional results for the ablation study in Section 6.3.7.

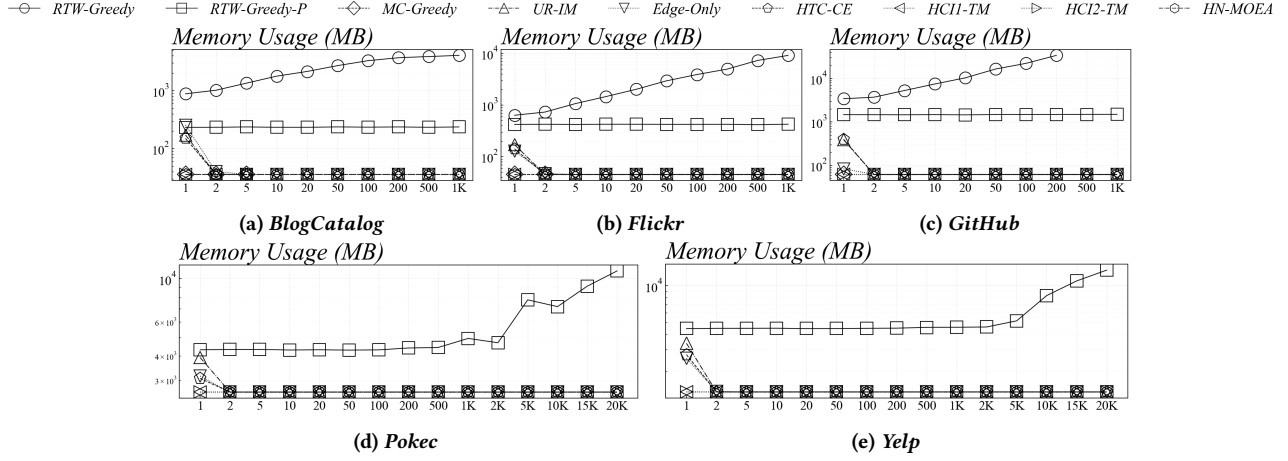


Figure 24: Memory usage (MB) vs. seed budget k for different methods on each dataset.

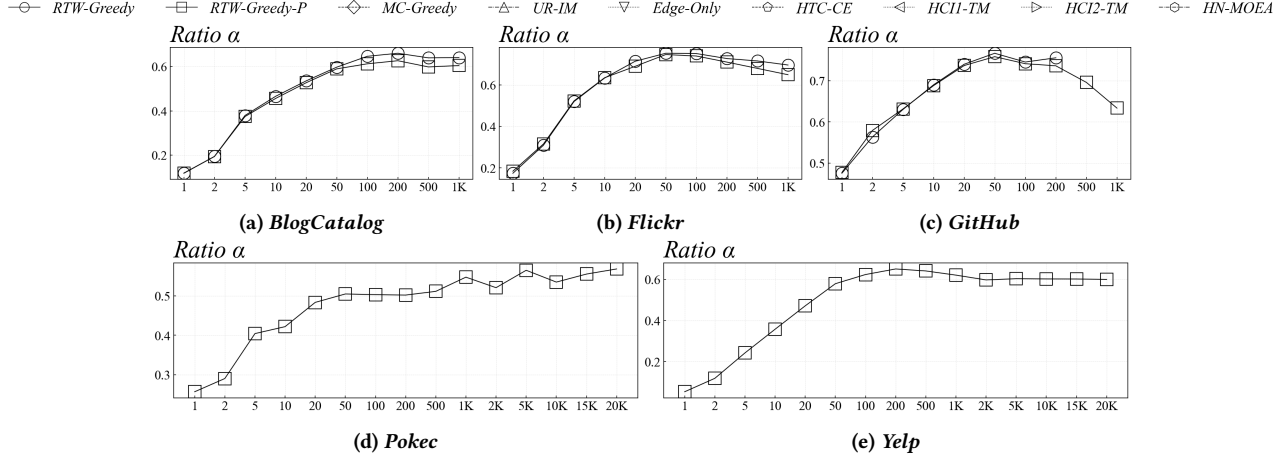


Figure 25: Solution-dependent lower-bound ratio α vs. seed budget k for different methods on each dataset.

Table 4: Performance of *RTW-Greedy-P* under different sample budgets θ_0 .

Dataset	Objective			Runtime (s)			Memory (MB)			Ratio α		
	1K	10K	100K	1K	10K	100K	1K	10K	100K	1K	10K	100K
<i>BlogCatalog</i>	1,244	1,381	1,412	15.54	68.44	430.79	60.97	236.22	1,917.40	0.521	0.590	0.597
<i>Flickr</i>	3,311	3,491	3,516	35.48	182.64	1,230.77	87.40	420.37	3,690.22	0.706	0.746	0.748
<i>CiaoDVD</i>	1,460	1,708	1,779	33.91	308.87	2,079.09	105.70	576.10	5,526.23	0.274	0.332	0.345
<i>GitHub</i>	15,407	16,256	16,325	60.07	393.35	2,810.82	226.68	1,473.75	13,689.09	0.714	0.758	0.772
<i>Epinions</i>	20,175	20,205	20,641	1.65	1.83	4.20	77.80	89.12	228.69	0.673	0.690	0.703
<i>Pokec</i>	35,724	36,111	39,940	286.96	275.35	1,622.77	2,611.64	2,611.59	4,291.19	0.439	0.440	0.505
<i>Yelp</i>	74,792	74,620	79,517	483.82	642.87	3,458.04	1,486.85	1,641.57	4,463.35	0.543	0.547	0.579

Tables 5 and 6 report the objective value, solution-dependent lower-bound ratio α , runtime, and memory usage of *RTW-Greedy*, *RTW-Greedy-P*, and *RTW-TrivialUB*. Recall that *RTW-TrivialUB* skips the upper-relaxed optimization step and directly uses the trivial upper bound $B = n$.

As shown in Table 5, *RTW-TrivialUB* can still achieve competitive objective values on several datasets. For example, on *BlogCatalog*, *Flickr*, *CiaoDVD*, *GitHub*, and *Epinions*, its objective values are close to those of *RTW-Greedy* and *RTW-Greedy-P*. However, on large datasets such as *Pokec* and *Yelp*, *RTW-TrivialUB* achieves lower objective values than *RTW-Greedy-P*. This suggests that simply

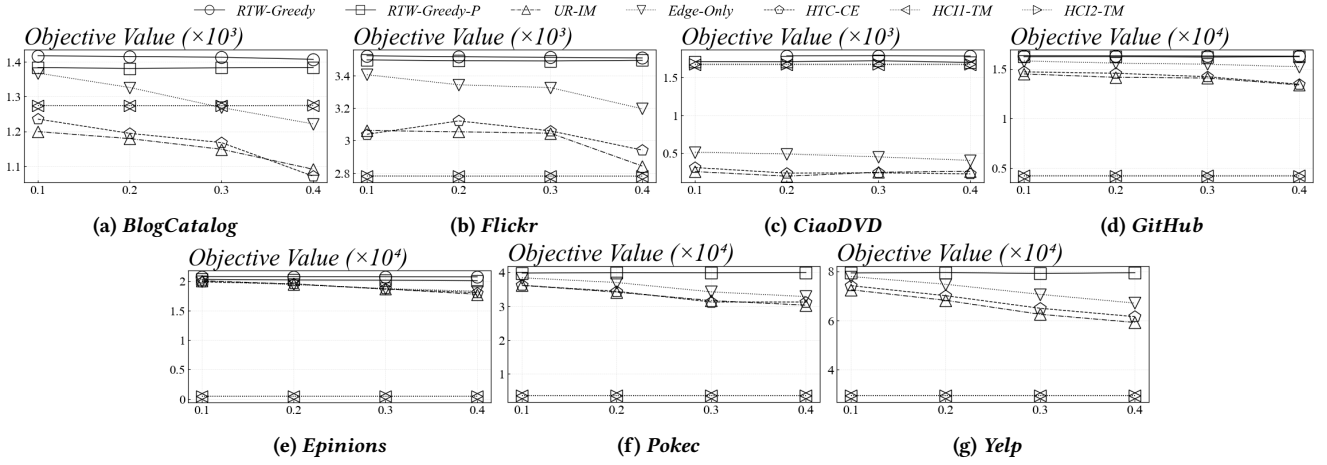


Figure 26: Objective value vs. error parameter ε for different methods on each dataset.

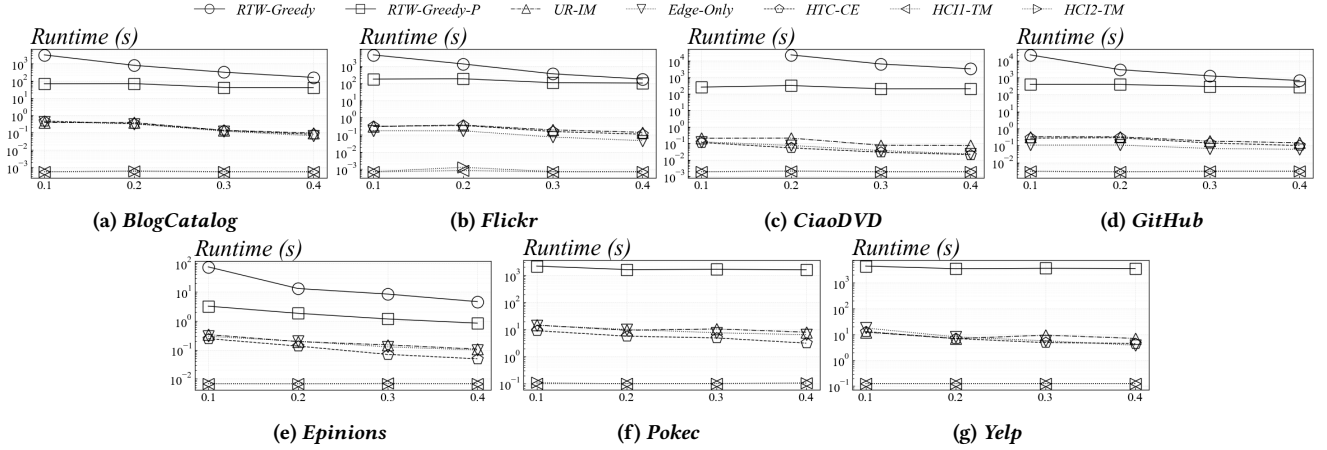


Figure 27: Runtime vs. error parameter ε for different methods on each dataset.

using the trivial upper bound is less robust when the graph scale becomes large.

The difference is more significant for the solution-dependent lower-bound ratio α . Compared with *RTW-Greedy* and *RTW-Greedy-P*, *RTW-TrivialUB* gives much smaller α values on all datasets. On *CiaoDVD*, *Pokec*, and *Yelp*, its α even becomes zero. This indicates that the trivial upper bound $B = n$ can make the solution-dependent guarantee too loose to be useful, even when the objective value is still competitive. In contrast, *RTW-Greedy* and *RTW-Greedy-P* achieve substantially larger α values, demonstrating that the upper-bound computation based on the upper-relaxed model is important for obtaining a nontrivial computable certificate.

Table 6 reports the runtime and memory usage. *RTW-TrivialUB* is sometimes faster than *RTW-Greedy-P*, especially on large datasets, because it avoids the upper-relaxed optimization step. However, this speedup comes with weaker objective values on large datasets and a substantially worse lower-bound ratio α . Moreover, *RTW-TrivialUB* does not consistently reduce memory usage, and it uses more memory than *RTW-Greedy-P* on several smaller datasets.

Overall, the ablation results show that replacing the upper-relaxed upper bound with the trivial upper bound $B = n$ is not a reliable alternative. Although *RTW-TrivialUB* can sometimes obtain similar objective values, it significantly weakens the solution-dependent lower-bound ratio and can reduce robustness on large datasets. These results validate the importance of the upper-bound computation based on the upper-relaxed model in the proposed RTW-based framework.

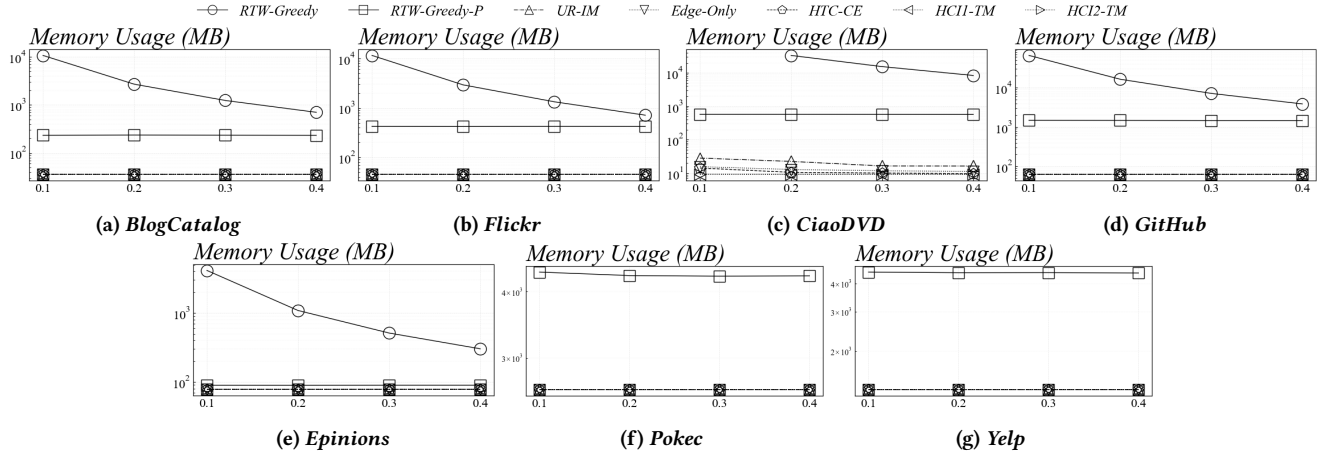


Figure 28: Memory usage (MB) vs. error parameter ε for different methods on each dataset.

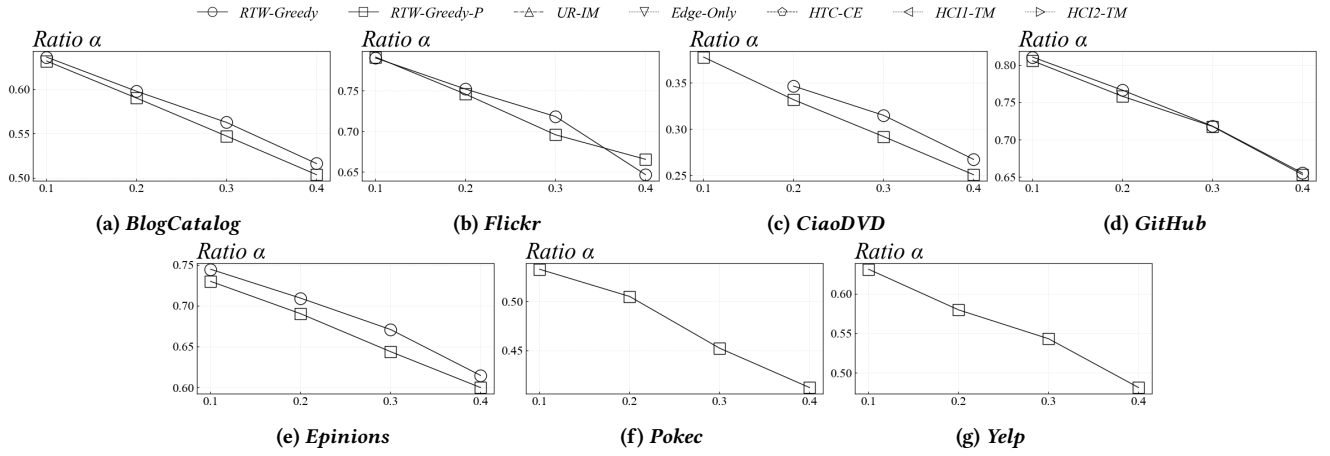


Figure 29: Solution-dependent lower-bound ratio α vs. error parameter ε for different methods on each dataset.

Table 5: Ablation study on objective value and ratio α .

Dataset	Objective			Ratio α		
	RTW-Greedy	RTW-Greedy-P	RTW-TrivialUB	RTW-Greedy	RTW-Greedy-P	RTW-TrivialUB
BlogCatalog	1,416	1,381	1,396	0.598	0.590	0.169
Flickr	3,515	3,491	3,504	0.752	0.746	0.363
CiaoDVD	1,785	1,708	1,745	0.347	0.332	0.000
GitHub	16,332	16,256	16,284	0.766	0.758	0.332
Epinions	20,767	20,205	20,448	0.709	0.690	0.079
Pokec	—	39,940	36,880	—	0.505	0.000
Yelp	—	79,517	76,712	—	0.579	0.000

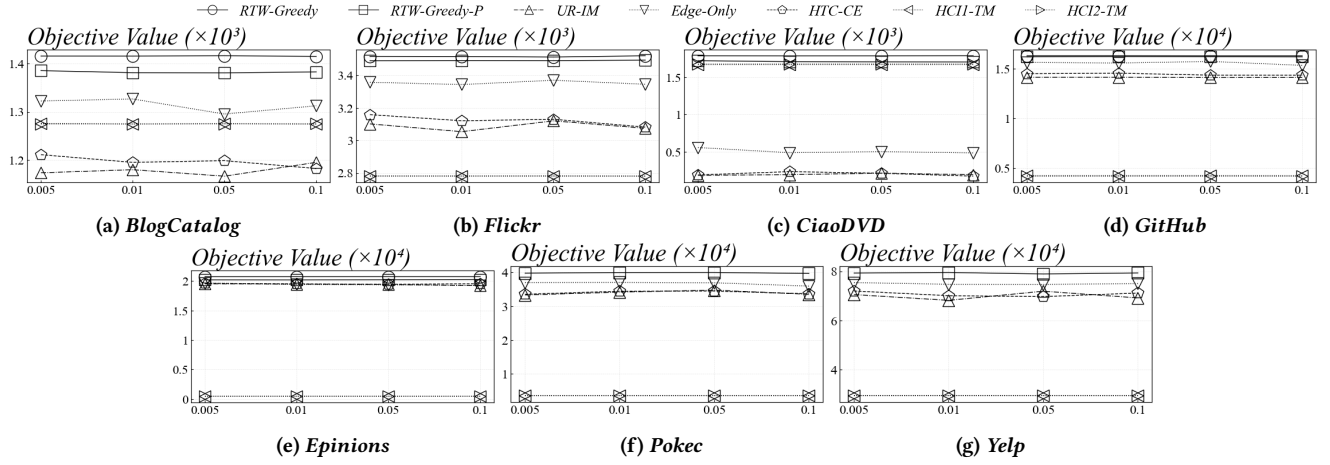


Figure 30: Objective value vs. failure probability δ for different methods on each dataset.

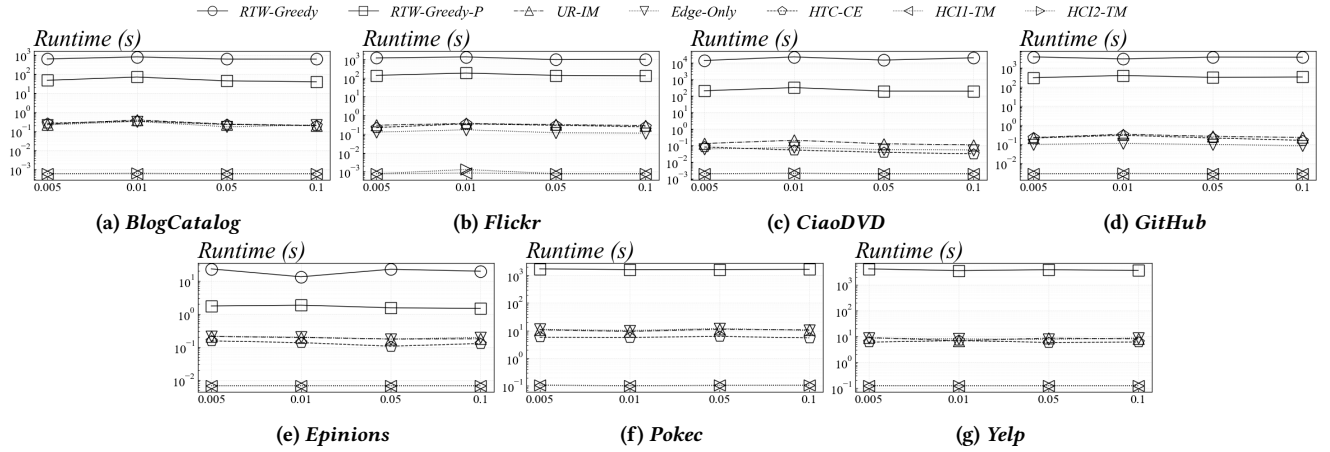


Figure 31: Runtime vs. failure probability δ for different methods on each dataset.

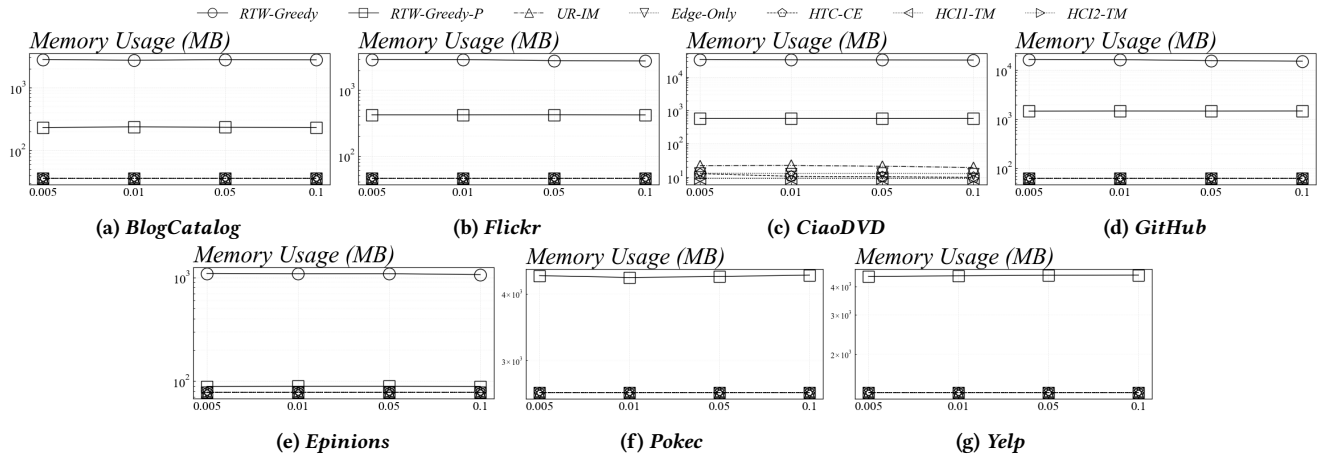


Figure 32: Memory usage (MB) vs. failure probability δ for different methods on each dataset.

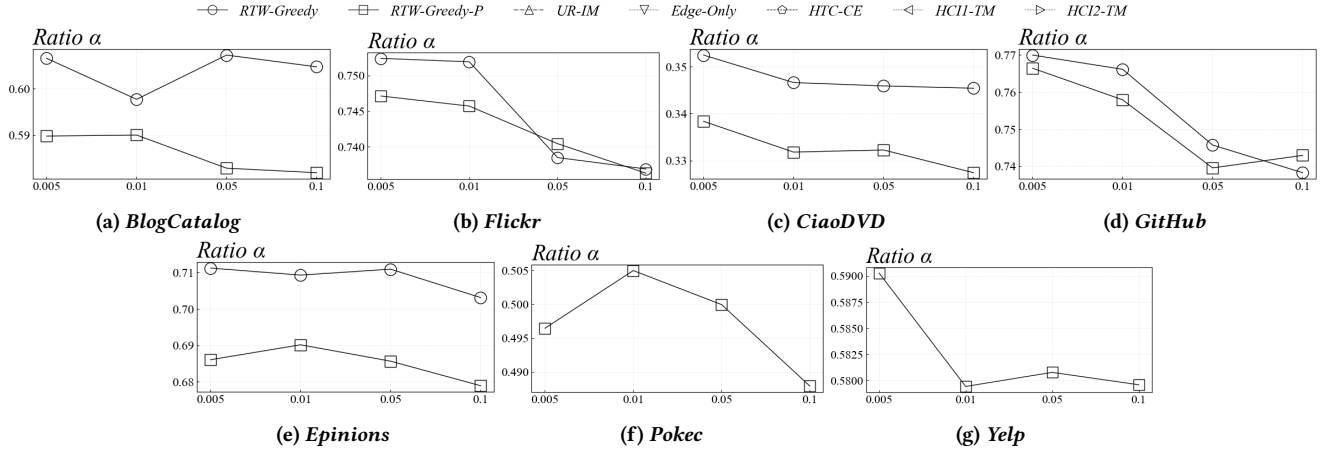


Figure 33: Solution-dependent lower-bound ratio α vs. failure probability δ for different methods on each dataset.

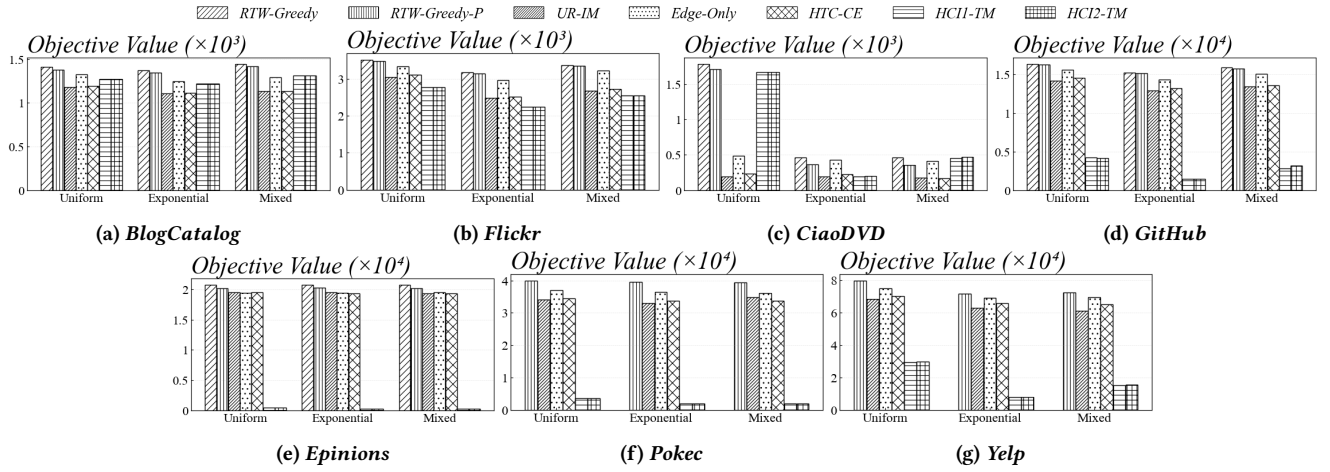


Figure 34: Objective value vs. threshold distribution for different methods on each dataset.

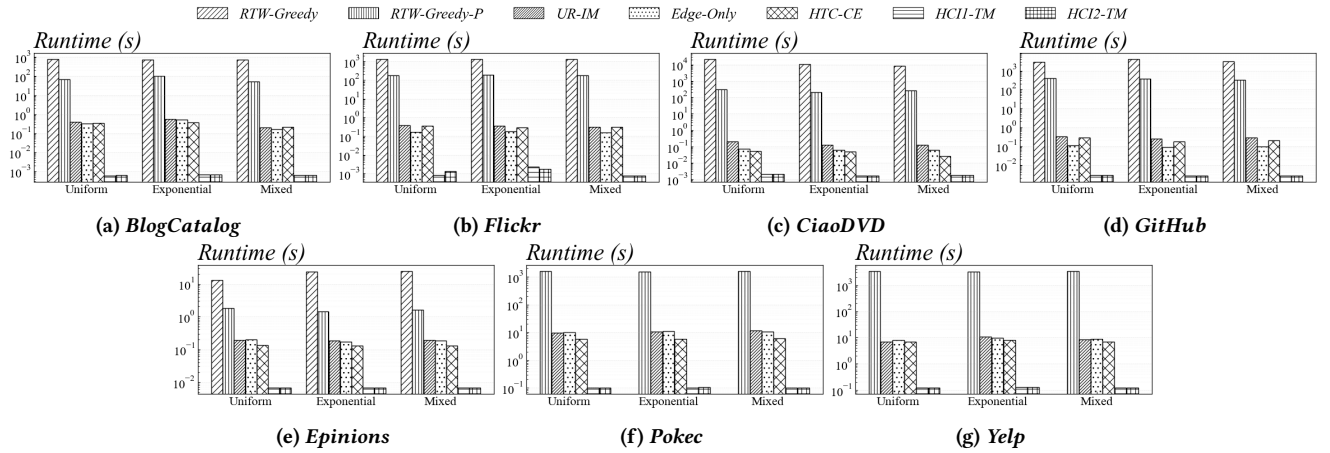


Figure 35: Runtime vs. threshold distribution for different methods on each dataset.

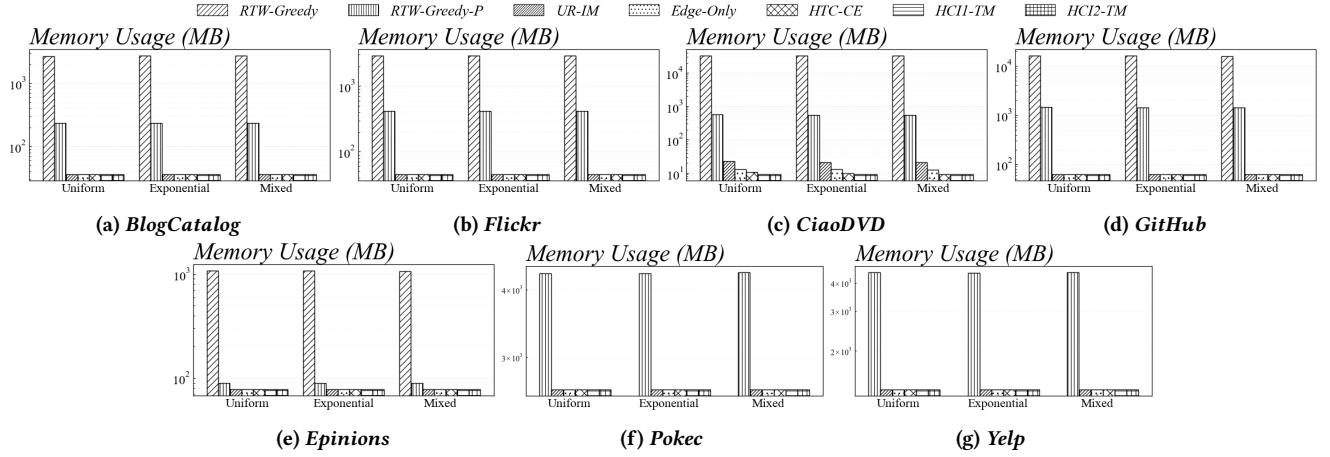


Figure 36: Memory usage (MB) vs. threshold distribution for different methods on each dataset.

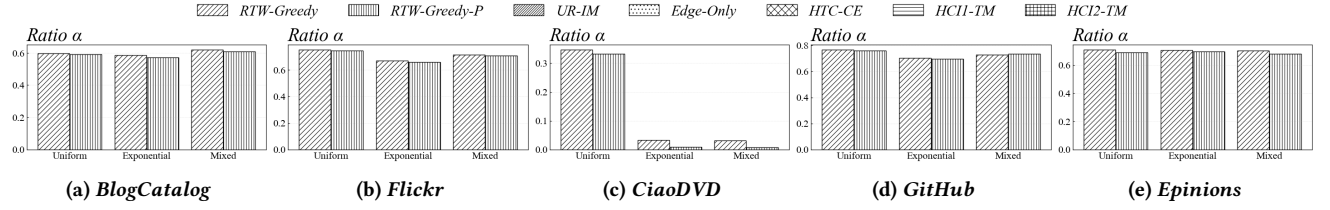


Figure 37: Solution-dependent lower-bound ratio α vs. threshold distribution for different methods on *BlogCatalog*, *Flickr*, *CiaoDVD*, *GitHub* and *Epinions*.

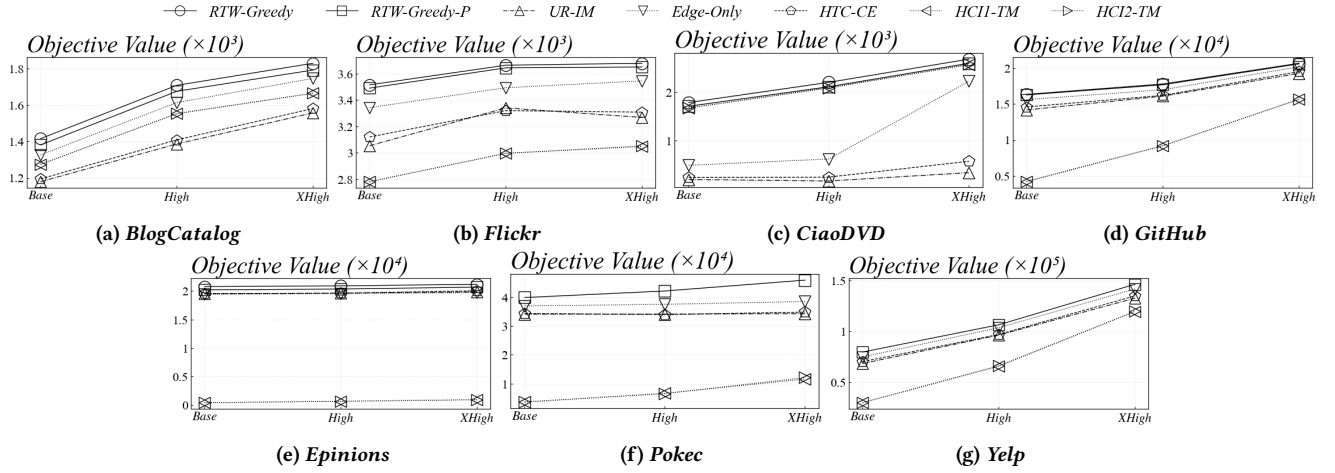


Figure 38: Objective value vs. number of hyperedges q for different methods on each dataset.

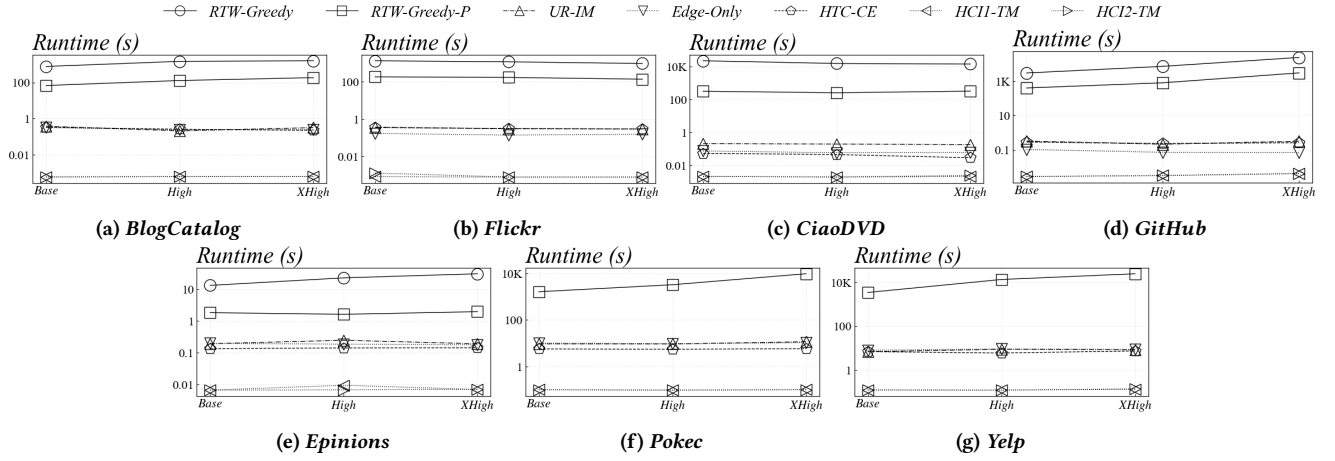


Figure 39: Runtime vs. number of hyperedges q for different methods on each dataset.

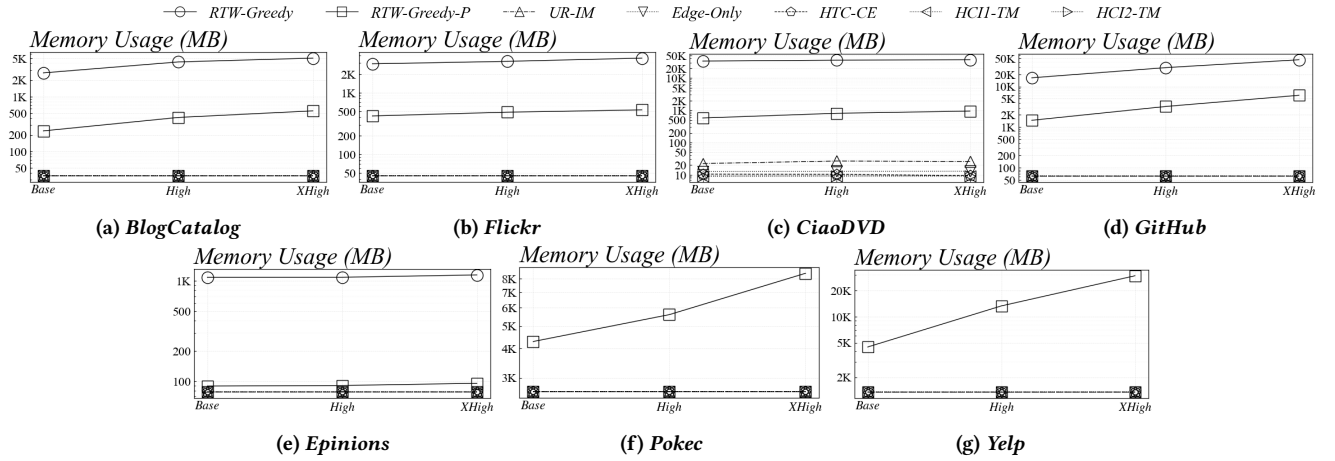


Figure 40: Memory usage (MB) vs. number of hyperedges q for different methods on each dataset.

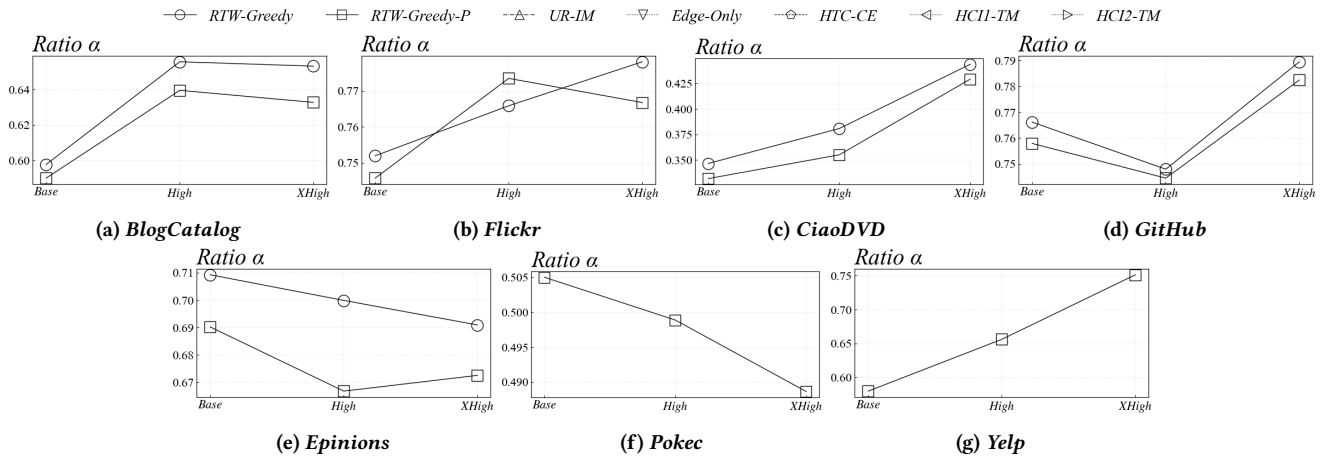


Figure 41: Solution-dependent lower-bound ratio α vs. number of hyperedges q for different methods on each dataset.

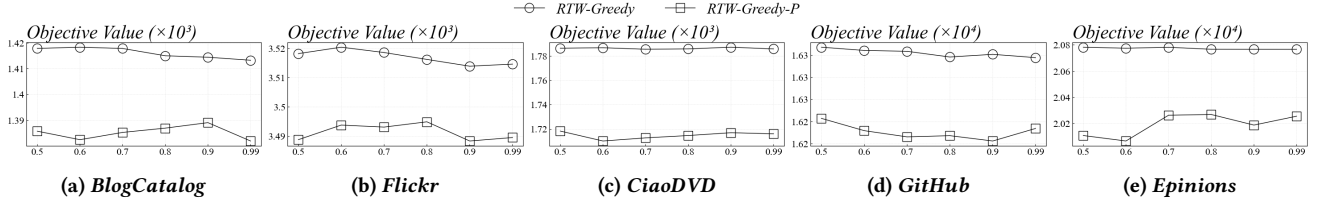


Figure 42: Objective values vs. allocation parameter λ for different methods on each dataset.

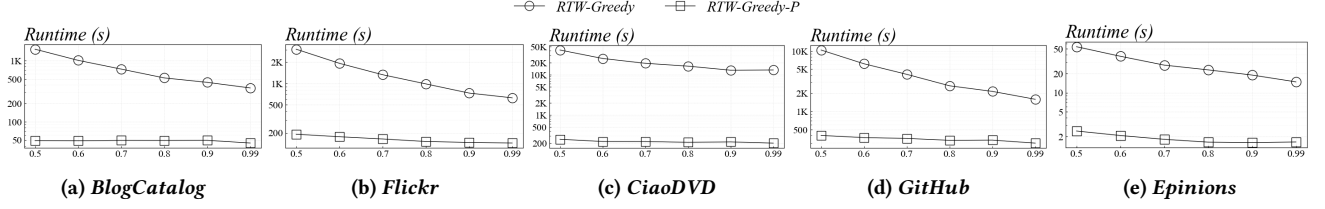


Figure 43: Runtime vs. allocation parameter λ for different methods on each dataset.

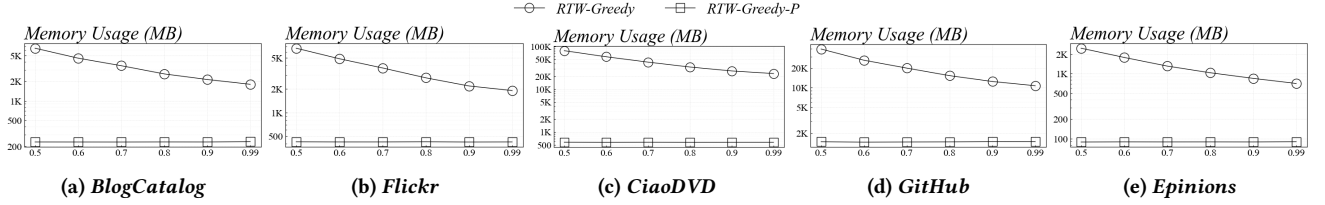


Figure 44: Memory usage (MB) vs. allocation parameter λ for different methods on each dataset.

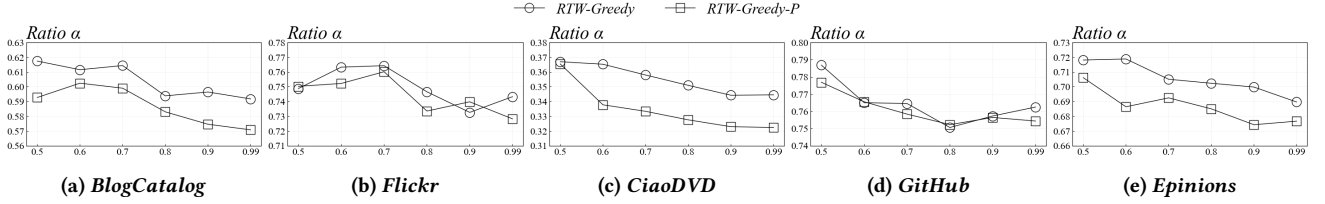


Figure 45: Solution-dependent lower-bound ratio α vs. allocation parameter λ for different methods on each dataset.

Table 6: Ablation study on runtime and memory usage.

Dataset	Runtime (s)			Memory (MB)		
	RTW-Greedy	RTW-Greedy-P	RTW-TrivialUB	RTW-Greedy	RTW-Greedy-P	RTW-TrivialUB
BlogCatalog	776.09	68.44	54.68	2,708.34	236.22	305.28
Flickr	1,332.95	182.64	157.60	2,926.72	420.37	591.21
CiaoDVD	21,977.72	308.87	339.16	33,313.59	576.10	981.56
GitHub	2,916.85	393.35	513.46	16,415.22	1,473.75	2,723.27
Epinions	13.16	1.83	0.72	1,086.16	89.12	79.60
Pokec	—	1,622.77	388.37	—	4,291.19	2,611.48
Yelp	—	3,458.04	809.91	—	4,463.35	1,734.98